Set aside

These passages were taken out of *The Thiele Machine: No Free Insight* and are kept here unchanged. Numbered theorems and sections cited below and not found here are the book's.

Contents

1 Proofs for "What it takes to be certified" 3

2 Proofs and worked runs for "Parts I and II, run on the example machine" 7

3 Proofs, the program and the worked tables for "The universal machines" 12

4 Running it 26

4.1 Taking the coat off 27

4.2 A second machine, written by hand, to check the first 27

4.3 *U* on a real guest 28

4.4 From source programs to the host 30

4.5 What wasn't built 36

5 Constructing the quoting operation 37

5.1 Numerals and syntax as data 37

5.2 Recursion without running under a lambda 38

5.3 Quoting numerals 39

5.4 Quoting terms 39

6 Computing with counters 41
6.1 Finite programs made from two instructions 41
6.2 A tape in finitely many counters 42
6.3 Packing finitely many counters into two 43
6.4 Why halting has no decider 44
6.5 Seven descriptions of the same computation 45
6.6 Searching for a finite computation 48

7 The universal program in blocks 49
7.1 Arithmetic blocks 49
7.2 Selection, comparison and fetching 50
7.3 Registers and instruction codes 51
7.4 The fifteen blocks of U 52
7.5 Why the blocks simulate a guest 53
7.6 Adding PAY 54

8 The CHSH derivation, in full 56
8.1 Setting and notation 56
8.2 The classical bound 57
8.3 Tsirelson's bound from algebra 59
8.4 The zero-marginal 5×5 moment matrix 61
8.5 The integer check 66
8.6 Level 1+AB 68
8.7 The elliptope completion 75
8.8 The four-vector construction 78
8.9 The arcsine boundary, in both directions 82
8.10 The finite local model, in both directions 83
8.11 Scope of the constructions 85
8.12 Prior work 86

9 Further reading 87
9.1 Structure, information and knowledge 87
9.2 Cost, records and thermodynamics 88
9.3 Computation, self-reference and checking 92
9.4 Correlations and environmental records 95

CHAPTER 1

Proofs for “What it takes to be certified”

Proof of the theorem “Certified is Thiele-complete plus certification” (book Theorem 15.23).

Proof. (1) Clause (a) is (A) together with “loaded states are clean”. Clause (c) is (C), with the three record kinds as the marks. Clause (b) says clean states have the record down, which with clean loads gives (B), and it gives the earned chain. Clause (d) gives (D): the claim’s three moves end with the record up from the loaded state where it holds and down from the one where it fails. Going back, the same pieces put the four clauses together again. (2) is (1) read left to right. (3) From any state with the record down, counting A to zero costs nothing and the first mark then raises the record: a raise for one mark from every clean state, where a certified raise from a clean state takes three (Theorem 15.25(3)). ∎

Proof of the theorem “CHECK is a unit-cost test” (book Theorem 15.24).

Proof. At a start the latch is down and the table is empty, so the check passes exactly when $\mathrm{eval}(\mathrm{Ge}(m), a)$ says yes, that is when $m \leq a$; a failing check sets the latch. The price table charges a CHECK 1. ∎

Proof of the theorem “What every certified Thiele-complete machine has” (book Theorem 15.25).

Proof. (1) A raise of the record isn’t made by a base move, by (a), so it’s a record move, which costs one by (c). Running a program on the base is induction on its steps, each matched by (a). (2) is (c), added up along the run. (3) Clause (b) finds a CHECK, a COMMIT and a CERTIFY at three different places, and (2) counts them. (4) is (d): the three moves certify exactly where the claim holds, the claim fails at some loaded state, and the check can’t pass there because (b) says a passing check means the claim holds; at the loaded state where it holds, the three moves raise the record. (5) Cut the run just after the move where the record rises. That shorter run

starts clean and ends with the record up, so clause (b) puts the record down right before a CERTIFY in it and up right after. The record never comes down, so it rises only once, and that CERTIFY is the move. (6) Every move is a base move or a record move, and (a) supplies base moves. ■

Proof of the theorem “The language first, and what a certificate then means” (book Theorem 15.26).

Proof. (1) Take the plain interface the language-first one becomes. (2) The checker’s soundness and the keeping proof are the ones clause (b) already supplies. (3) Clause (b) gives the chain; the checker’s soundness proof against L gives c’s meaning at the state checked; L’s keeping proof carries it, through states where the thing c is about is unchanged, to the state committed. ■

Proof of the observation “A clock is a Thiele machine and isn’t certified Thiele–complete” (book Observation 15.27).

Proof. Every move costs one, so any raise costs at least one: the toll holds for any record. The machine’s own moves are the two-counter instructions, which gives the universal base. But a certified Thiele-complete machine has a move priced 0, by Theorem 15.25(6), and this one has none. ■

Proof of the theorem “Each clause does work” (book Theorem 15.28).

Proof. (1) The base moves leave the flag alone, TICK never lowers it, the base costs 0 and TICK costs 1, which gives (a) and (c). Through any interface, (a) gives a loaded state, which is clean, so its record is down by (b), and a move that puts the record up from every state raises it alone. Clause (b) would need a CHECK, a COMMIT and a CERTIFY inside that one-move run, and one move can’t be three. (2) The base gives (a) and costs 0, which with no record moves is (c). No run ends with the record up, so (b) asks nothing of a run, and a checker that always says no is sound for any meaning. Clause (d) needs a run from some loaded state that raises the record, and none does. ■

Proof of the theorem “The four clauses are independent” (book Theorem 15.29).

Proof. (a) Nothing moves a trapped state’s flag, so a run that uses DROP ends with the flag down. A run from a clean state that ends with the flag up never used DROP; it’s a run of this chapter’s machine, and Theorem 15.11 gives its chain. DROP is priced 0 and read as a base move, so (c) holds, and (d) holds as it does for the plain machine. A state with the flag up has it down after DROP. That breaks the

permanence in (a), and permanence is about the machine alone, so no interface meets (a). (b) FORGE never lowers the flag, costs one and is read as a record move, so (a), (c) and (d) hold as before. From a clean start FORGE alone raises the flag, and a run of one move has no room for a CHECK, a COMMIT and a CERTIFY. Through any interface: (d) gives a loaded state from which the record rises, so its trap is down; (a) makes that state clean, so its record is down by (b); and FORGE alone raises it, which (b) forbids. (c) Clauses (a), (b) and (d) never mention a price, and they hold as for the plain machine. Base moves cost one, so (c) fails, and every certified Thiele-complete machine has a move priced 0 (Theorem 15.25(6)). (d) is Theorem 15.28(2). ∎

Proof of the theorem “Any unit” (book Theorem 15.30).

Proof. (1) At unit one every listed price is unchanged.

(2) Changing the size of the unit changes no state transition, claim or record. Thus the computing, earned-chain and distinguishing-claim requirements stay as they were. To multiply the prices by c, read the old ledger multiplied by c too. Every addition is then multiplied by the same amount.

In the other direction, count complete groups of c units in the ledger. There may be a remainder smaller than c at the start. A step adds either nothing or exactly one complete group, so it leaves that remainder alone and increases the group count by the divided price. The equation for that count is

$$\left\lfloor \frac{\ell + cr}{c} \right\rfloor = \left\lfloor \frac{\ell}{c} \right\rfloor + r.$$

It gives the required whole-number ledger on the same states and runs.

(3) The move for an ordinary counter instruction is a base move and costs zero at every unit. A machine charging for every move has no such move.

(4) Find the passing CHECK, matching COMMIT and raising CERTIFY in the earned chain. They occupy three different places in the run. Each costs c, and all other charges are nonnegative, giving at least $3c$. The claim was true at its check and stays true up to its commitment because what it concerns is unchanged. The record rises at that chain’s CERTIFY and never comes down; changing the price unit changes none of these events.

(5) Doubling the small machine’s prices gives unit two by (2). Its CHECK then costs two. At unit one every move must cost zero or one, so this same priced machine cannot qualify there through another choice of interface. ∎

Proof of the theorem "The small machine, the generic one and the sorted one are certified Thiele-complete" (book Theorem 15.31).

Proof. (1) Clause (a) is Theorem 15.3 together with the step line: INC, DEC and HALT never touch the flag, the flag never comes down, and a start is clean with its trap down. Clause (b) is Theorem 15.11 with checker soundness (Lemma 15.2), and a meaning about c keeps holding while c's value doesn't move. Clause (c) is the price table. Clause (d) is Theorem 15.22 with the claim "A is zero," which holds at $(0, b)$ and fails at $(5, b)$. (2) and (3) are the same argument with the property language left open. For (3), read a number as a list in the list code: 0 is the empty list, and $2^x(2y+1)$ is x followed by the list numbered y (every positive number splits that way in exactly one way, and y is smaller than the number, so the reading ends). The meaning of *sorted* is that each entry is at most the one after it. The checker reads the list from the front: yes on the empty list and on a list of one entry, and on x followed by a list that starts with y, yes exactly when $x \leq y$ and it says yes on that rest. Induction on the length of the list shows it says yes exactly when the meaning holds, and a language with one property has an exact equality test. The sorted claim holds of the list numbered 18, which is $[1, 2]$, and fails of 20, which is $[2, 1]$. (4) is (1) of Theorem 15.25 and Section 2's floor. (5) The interface over the language is the one of (1), (2) and (3) with the meanings handed over by the language, and the checker's soundness is Lemma 15.2 and its generic form. ∎

Proof of the observation "Every certified Thiele-complete machine hides its record and its ledger from its window" (book Observation 15.32).

Proof. Theorem 15.25(4) gives a loaded state and a run from it that raises the record, and (3) says that run paid at least 3. Say it ends with window (line ℓ, x, y). Two-counter instructions can steer any configuration to any other: empty A, fill it to $x+1$, empty B, fill it to y, then one DEC of A that jumps to line ℓ. Those are base moves, so from the same loaded state they reach the same window, cost nothing, and leave the record down, by (a), (b) and (c). That's two runs, one window, two records and two ledgers, which is (2). A function of the window would have to give both answers on that one window, which is (3). For (1), load $(x+1, y)$ and take just the last DEC. (4) is (3) read as a Thiele machine, whose runs are the machine's. ∎

CHAPTER 2

Proofs and worked runs for “Parts I and II, run on the example machine”

Proof of the theorem “The small machine runs the eight-state machine” (book Theorem 15.34).

Proof. (1) By cases on the move. On a live state CERTIFY runs, because a committed claim is on the channel; it raises the flag, moves the line on by one and charges one. INC B moves the line on by one and charges nothing, and it touches only B and B’s version, so the fact about A and the commitment stand and the state stays live. In a jump, each COMMIT finds its fact in the table and succeeds, charging one, and the INC and the DEC charge nothing and end at line a, whose slot is a; the flag isn’t touched. (2) After each compiled move, the state is live again, so the next compiled move starts with exactly the condition just used in (1). The window agrees after each move and each move adds its own charge. At the end of the list the windows agree and the ledger has added the sum of those charges. (3) The eight-state machine’s own toll (Section 3) says a run of its moves that takes the flag from down to up is charged at least one, and (2) carries that charge across. (4) CHECK ($A \geq 0$) passes at every value of A and the COMMIT after it passes, which takes line 1 to line 3, slot 3, with two units of cost paid; then (2) applies. A raised flag was raised by a CERTIFY, and Theorem 15.11 gives the chain. ∎

Proof of the theorem “Structural entitlement represented” (book Theorem 15.36).

Proof. (1) is Theorem 8.3(1). (2) Clause (b) applied to a run from a clean state that ends with the record up gives the chain, and a passing check means the claim’s meaning holds, which keeps holding while the thing it’s about is unchanged. (3) The covering (M6) and Theorem 8.3’s count finish it, with the record moves, which

are the paid steps, in place of the charged steps. (4) is clause (c) added up. (5) is Theorem 15.25(3). ■

Proof of the theorem “A floor with no supplied tree” (book Theorem 15.37).

Proof. Write each rival’s answers on a strip, one box per CHECK in the run. One yes-or-no question gives two possible strips; a second gives four; each further question doubles the possibilities. With c questions there are at most 2^c strips. Put rivals with the same strip in the same pile. The premise limits each pile to m rivals. Hence at most $2^c m$ rivals can be on the original list.

Now $\lceil \log_2 m \rceil$ bits suffice to number m places inside a pile, and c bits suffice to name its strip. Together they number every rival using at most $c + \lceil \log_2 m \rceil$ bits. The least number needed for $|\Omega|$ rivals cannot be larger. Subtract the inside-pile allowance, clipping a negative result to zero, to get the claimed bound of c.

Each CHECK is a record move. Counting all record moves can only increase that number, and each contributes exactly one to the ledger while base moves contribute zero. This gives the two remaining comparisons. ■

Proof of the theorem “Structural entitlement, with the narrowing made by the run” (book Theorem 15.38).

Proof. (1)–(2) Start with the prior list. At each CHECK the actual run passed, run the preceding instructions from each surviving rival and ask that same question there. Keep the rivals that pass. Nothing adds a rival, and the actual start s_0 passes every one of its own tests, so it stays throughout. A missing rival has a first test that removed it. That is the required failed CHECK, at the stated place in the run.

(3) Name the successive list sizes $q_0, q_1, \ldots, q_n$, with $q_0 = |\Omega|$ and $q_n = |\Omega'|$. The before-and-after sizes for test i are consecutive members of that list. Multiply the after-sizes by q_0, and multiply the before-sizes by q_n: each side contains exactly $q_0 q_1 \cdots q_n$. This is the displayed product identity.

(4) The share premise says $q_{i-1} \le 2^k q_i$ at every test. Apply it first to q_0, then to q_1 in the resulting bound, and keep going. After n tests it gives $q_0 \le 2^{kn} q_n$. Numbering q_n final places takes $\lceil \log_2 q_n \rceil$ bits; another kn bits can number up to 2^{kn} times as many. Thus the difference in rounded index lengths is at most kn. Each of these n tests is a CHECK the run actually passed, so it is among the paid record moves. Exact prices make the number of those moves the ledger increase. Multiplying that larger count by k preserves the bound.

(5) The run starts clean and ends with its record up. The earned-record requirement therefore finds its CHECK, COMMIT and raising CERTIFY. Those three different acts each cost one, so the ledger increased by at least three.

(6) The actual start remained on every list, so every q_i is positive. The ratios $q_0/q_1, q_1/q_2, \ldots, q_{n-1}/q_n$ multiply to q_0/q_n: each intermediate size cancels once above and once below. Taking base-two logarithms turns this product into the sum of the per-test logarithms.

To account for rounding, set $u = \log_2 q_0$ and $w = \log_2 q_n$. The integer $\lceil u - w \rceil + \lceil w \rceil$ is at least $(u - w) + w = u$, so it is at least the least integer above u, namely $\lceil u \rceil$. Rearranging gives the claimed bound by $\lceil \log_2(q_0/q_n) \rceil$. Finally, rounding each per-test logarithm up and adding gives an integer at least as large as their sum. It cannot be smaller than rounding the sum up just once. This proves the last comparison. ∎

Proof of the proposition “One CHECK can take four bits” (book Proposition 15.39).

Proof. $A \geq 15$ holds at exactly one of the sixteen starts. From $(15, 0)$ the CHECK passes and files the claim, the COMMIT finds it with A’s version unchanged, and the CERTIFY raises the record. The posterior is the one rival that passes the run’s only CHECK, so the index bits are $\lceil \log_2 16 \rceil - \lceil \log_2 1 \rceil = 4$, and the run has three record moves. ∎

Proof of the theorem “What the record adds” (book Theorem 15.40).

Proof. (1) One direction is clause (b): a run from a clean start that ends with the record up contains the chain. For the other, the chain’s CERTIFY leaves the record up, which is part of what the chain is, and by the last words of clause (a) a record that’s up stays up under every move, so whatever the run does after that CERTIFY it ends with the record up. The floor of 3 is Theorem 15.25(3). (2) Read the proof of Theorem 15.36 again and mark what each conclusion used. Its (1) used (M1) and (M2), through Theorem 8.3(1), and nothing about the machine. Its (3) used (M5) and the count $|\Omega| \leq L(T) \cdot |\Omega'|$ of (M6), and (M4) to put the record moves in place of the tree’s depth; the clean start and the raised record never entered. Its (4) is clause (c) added up along the run, from whatever state the run starts at. The decoded run being shown by some survivor is in the hypotheses and, like the second half of (M3) above, feeds none of the three. The conclusions dropped, (2) and (5), are exactly the two that used the clean start and the record. ∎

Proof of the proposition “The eight-start member” (book Proposition 15.41).

Proof. From any loaded start, the first CHECK passes exactly when a is even. If it passes, the second has room in the table and passes exactly when $a \geq 2$. Nothing has changed A, so their matching COMMIT and CERTIFY then succeed. If either check

fails, the trap prevents certification. This proves the claim for every a, b. Within the eight rivals, a can only be $0, 1, 2, 3$; of those, only 2 is even and at least 2, whatever b. From $(2, 0)$ and $(2, 1)$ the COMMIT then finds its fact with A's version unchanged, nothing having written A, and the CERTIFY raises the flag; from every other start the first failing CHECK traps the machine, and a trapped machine raises nothing. So the posterior is those two. The hypotheses: the posterior narrows the eight strictly (M1); a dropped rival whose run fails a CHECK shows receipts no survivor shows (M2); the run ends with the record up and its receipts, two passes, are the survivors' (M3); the tree has depth 2 and the run 4 record moves (M4); two survivors (M5); and the fibres, the four rivals with B at 0 for the survivor $(2, 0)$ and the four with B at 1 for $(2, 1)$, cover all eight, add up to eight, and have four members each, the leaves of the tree, so $8 \leq 4 \cdot 2$ (M6). Then $\lceil \log_2 8 \rceil - \lceil \log_2 2 \rceil = 3 - 1 = 2$; the four record moves are two CHECKs, a COMMIT and a CERTIFY, and the ledger rises by exactly those four by clause (c); and the chain is in the run. For Theorem 15.37, the run's questions are its two CHECKs, and two rivals answer them alike exactly when their a's agree on being even and on reaching 2, which puts the eight starts in four classes of two, so $m = 2$ and $\lceil \log_2 8 \rceil - \lceil \log_2 2 \rceil = 2$ is the number of CHECKs. With the two CHECKs alone there's no CERTIFY, so the record stays down, the ledger rises by 2, and the index bits are still 2. ■

Proof of the theorem "A search through n bits" (book Theorem 15.42).

Proof. Each question is a CHECK of whether one register holds a given bit, which passes exactly when it does; a failing CHECK traps. So the program certifies exactly when every answer is right, which is when v begins with the answers. The counting is the count of bit strings: the 2^m fibres of 2^k values each add up to the 2^{k+m} of the prior, and none is longer than the 2^k leaves of the tree, so the representation theorem applies with every hypothesis a theorem. The cap is the fact table: it holds sixteen facts, never evicts, and a CHECK on a full table traps (Theorem 15.16), so a seventeenth CHECK before the CERTIFY ends the run in a trap. ■

Proof of the theorem "The time tax" (book Theorem 15.43).

Proof. (1) Take the starts $(m - 1, b)$ and (m, b). At the jth move, $j \leq n < m$, at most $j - 1$ pebbles have left A, so A is at least $m - j \geq 1$ in both runs, and no DEC finds A empty in either. The two runs take the same branches and differ only by the one pebble in A. They are at the same line with the same B, and they stop at the same moment if they stop at all, though $A \geq m$ holds of one start and not the other. (2) Run the ladder from $A = a$. Each of the first $\min(a, m)$ moves takes a pebble off A and jumps two lines on. If $a \geq m$ that's all m moves, ending on line $2m + 1$. If $a < m$, the run reaches line $2a + 1$ after a moves and from there goes on

one line per move (a DEC falling through, then an INC), so after m moves it's on line $a + m + 1$, below $2m + 1$. It's all DEC and INC, so it costs 0. (3) is the price table: CHECK ($A \geq m$) passes exactly when $A \geq m$, a failing CHECK traps and the flag stays down, and the three moves cost three. (4) is the same price table with one instruction: a passing CHECK goes on to line 2 and a failing one traps and leaves the line at 1. (5) is arithmetic on (1) to (4). ∎

The worked run for the time tax, with its table. Try $m = 3$ with $A = 2$. The ladder takes a pebble on line 1 and jumps to line 3, takes the second and jumps to line 5, then finds A empty and falls through to line 6, the INC: after three moves it sits on line 6, not 7. With $A = 3$ the third DEC succeeds and the run is on line 7. Two moves in, both runs are on line 5 with B untouched, which is (1).

Moves taken	Start with two pebbles	Start with three pebbles
0	line 1, $A = 2$	line 1, $A = 3$
1	line 3, $A = 1$	line 3, $A = 2$
2	line 5, $A = 0$	line 5, $A = 1$
3	line 6, $A = 0$	line 7, $A = 0$

The reader in the theorem sees the line and B, which stays unchanged through these three moves. The A entries are shown here to explain the run; that reader is not allowed to use them. Line 6 still has an instruction: the decision time is not a claim that both runs have halted.

CHAPTER 3

Proofs, the program and the worked tables for "The universal machines"

Proof of the theorem "No host fact on an exact copy of the guest's counter" (book Theorem D.1).

Proof. A fixed program mentions finitely many properties, so Q is finite, and the translation sends the infinitely many $\mathrm{Ge}(0), \mathrm{Ge}(1), \ldots$ into it: two of them land on the same member. The guest's COMMIT asks for $\mathrm{Ge}(m)$, and its table can only hold $\mathrm{Ge}(n)$, so it traps (or its CHECK already trapped). The host sees one property twice: its CHECK passes, since the guest's would, and its COMMIT finds exactly what that CHECK wrote. ∎

Proof of the theorem "U keeps pace with its guest" (book Theorem D.2).

Proof. The one-step part is a case for each thing the guest can do at its line: stop; INC; DEC, taken or not; CHECK, passing, failing, or with a full table; COMMIT, found or not; CERTIFY, passing or not. The block invariants and the case analysis in Appendix L show where each matching stretch lands. The relation carries the bookkeeping that makes the next case go through: each count register equals the number of slots in use for its counter, each slot in use holds (property number, checked value), a slot's fact is current exactly when the matching guest fact is, a live slot's mirror is the property's number plus one and a stale one's is 0, and unused slots, unused mirrors and the dead end are 0. At loading, the counters and guest line have their specified starting values, and both tables and channels are empty, so the matching conditions hold before any step. Now take one guest instruction. The matching stretch just described ends either at the next matching point or with both machines stopped. In the first case its slot and mirror conditions are exactly what the following instruction needs, so you can do the next stretch. Repeat this for the first m guest steps. Each running stretch takes at least one host step; if neither

machine has stopped, the accumulated host count is therefore at least m. If they stop earlier, both remain stopped with the matching final fields. That gives the whole-run statement for every m.

Proof of the theorem "Halting and answers match" (book Theorem D.3).

Proof. Theorem D.2: a halted guest is matched by a stopped host, and a host that has stopped can't be back at line 1 with the guest still running, because line 1 isn't a stop. A halted machine stays halted, so the two halting points carry the same values.

Proof of the theorem "The host's flag is the guest's, and its ledger is exact" (book Theorem D.4).

Proof. (1) Flags are equal at matching points and never come down. (2) Ledgers are equal at matching points; inside the stretch of U that runs one guest instruction, at most the one paid site of that instruction runs, so the host's ledger sits between the guest's before and after it.

Proof of the theorem "The host earns its own flag" (book Theorem D.5).

Proof. By Theorem D.4 the guest's flag rises too; find the guest step where it rises, a passing CERTIFY, and the related host point just before it. The guest's channel there names a fact, and the relation says the host's channel names the matching slot fact. The host's own flag rises at its own first passing CERTIFY, and nothing paid runs between the related point and that CERTIFY, so the host's channel hasn't moved: it names that slot fact. The host machine's version of Theorem 15.11 gives the host's CHECK and COMMIT on that slot. The record kept with the slot says what pair it held and when, and the checker's meaning gives holds(p, v). The same theorem on the guest's run, up to the step where its flag rose, gives the guest's chain on the same p and c, and the record says c held v at its CHECK.

Proof of the theorem "The host is certified Thiele-complete, and U's halting is undecidable" (book Theorem D.6).

Proof. (1) The base is registers 0 and 1, run by INC and DEC; the claims are "Slot holds of register r"; the rest is the host machine's own versions of the small machine's theorems, and for non-vacuity, register 0 loaded with 1, which is $\langle 0, 0\rangle$, the pair of Zero and 0, against register 0 loaded with 0, which is no pair. Over the claim language the interface is the same one, with the meanings handed over by the language. (2) is Theorem 15.31(4) and Theorem D.5. (3) is Theorem D.3, and

the loading map reduces the small machine's halting question, undecidable by Theorem 15.5, to this one. ■

Proof of the theorem "One host runs every computably presented Thiele machine" (book Theorem D.7).

Proof. Two layers, each proved on its own. The bottom layer is the compiler: for every presented machine, its guest program's run passes through points that code M's states in order, with the guest's ledger equal to M's ledger plus the surcharge and the guest's flag equal to M's latch, and the guest's own flag stands on an earned chain whose claim is M's reading program applied to the coded state where the reading first said yes. Here is why. The guest keeps a few registers packed into its counters the way Minsky packed them: the coded state, a latch bit, the price still owed, and a register that counts the reading's steps. It starts at the reading with nothing owed, and goes round one loop. Run the compiled reading on the coded state, counting its steps. If it says yes and the latch bit is 0, run CHECK, COMMIT and CERTIFY on the claim "reading program r says yes of v," v being the packed registers with the step count as fuel, set the latch bit, and take three off what's owed, stopping at zero. Pay what's owed in PAYs. Run the compiled driver, and halt if it says stop. Otherwise run the compiled step and price for the move it picked, put the new coded state and that price in place, and go round again. Everything but the PAYs and the one chain is INC and DEC, at no cost. So at the point just after the PAYs and before the driver runs, once n moves have been made, the coded state is M's state after n moves, the latch bit is M's latch, and the ledger is M's ledger, except that the move where the latch bit was set was paid with the chain's three and the larger of $c - 3$ and 0 in PAYs instead of its price c (or, if the reading is yes at s_0, the start was charged the chain's three). The CHECK passes, because the fuel is the reading's own step count, and at such a value the fixed checker says what the reading says; the COMMIT finds that fact, since nothing wrote B in between; and the latch bit lets the chain run once. The top layer is U_P, proved the way U was in the previous theorems, with the PAY case added: it keeps pace with any guest of this kind, halting and flag matched, ledger equal at matching points, and its flag earned on a slot. Compose the two. The earned part takes one more step: the host's chain gives a guest CHECK that passed, and a guest run that ends with its flag up has exactly one passing CHECK, the compiler's CHECK of the reading on counter B. So the guest CHECK the host's chain points to is that one, and its claim is M's reading at the first state where it said yes. ■

Proof of the theorem “Exact, within two, and no better” (book Theorem D.8).

Proof. (1) and (2): the guest pays M's prices in PAYs, a unit at a time, except at the move where M's reading first says yes. There it has to run its CHECK, COMMIT and CERTIFY, three units, and it makes up the rest of the move's price in PAYs, so it pays the larger of the price c and three. The price is at least one, because the move raised a reading (the toll), so the excess is at most two, and none when $c \geq 3$. (3): a raised flag on the host machine costs at least three from a clean start, whatever the program, and M's ledger is less. (4): the guest finds the reading yes before M has made a move, runs its CHECK, COMMIT and CERTIFY there, three units on top of a ledger of nothing, and from then on its flag is up, so the reading never first says yes again and every move's price is paid in PAYs, exactly. ∎

Proof of the theorem “The host runs any guest” (book Theorem D.9).

Proof. (1) Before any move, the guest is at $g(h)$. One untrapped guest move has budget 1, enough for every guest instruction, so it takes exactly the guest's next step. It leaves the host untrapped, which lets the same argument apply to the next move. After the first move the guest has taken one step; after the second, two; and after n, exactly n. No such move changes the guest's program. (2) One pass of U_0 is a guest step, then INC A and DEC A back to line 1, so it is (1) once per pass. (3) A host program made only of OWN moves is the small machine, core for core, and Theorem 15.4 does the rest. ∎

Proof of the observation “The copy is the record” (book Observation D.10).

Proof. One move at a time. An OWN move touches neither the mirror nor the guest. A trapped GSTEP touches neither. An untrapped one sets the mirror to the old mirror or the crossing, and the guest's flag is a latch (Theorem 15.6), so if the old mirror equalled the old flag, the new mirror equals the new flag. The last sentence is this together with Theorem D.9. ∎

Proof of the theorem “The host's step collects the guest's toll” (book Theorem D.11).

Proof. (1) The guest's flag rises only on a passing CERTIFY (Theorem 15.6), which costs 1, and the budget let it run, so $b \geq 1$; the mirror was the guest's flag, so it was down, and the move sets it to the crossing. (2) is (1) at the matching move. (3) Write the host's charge and the guest's ledger increase beside every move. When a guest step runs, its price is at most the budget the host paid. When none runs,

the guest's increase is zero and the host's charge is nonnegative. Adding these comparisons along the trace gives

$$\mu(g(\text{run}(t,h))) - \mu(g(h)) \le \mu(\text{run}(t,h)) - \mu(h).$$

- Move the two starting readings to the other sides to get the stated form.

Proof of the theorem "No unpaid host certification" (book Theorem D.12).

Proof. An OWN move copies the mirror unchanged, and a trapped GSTEP does too, so the rise is an untrapped GSTEP b whose guest step crossed; a crossing is a passing CERTIFY, which costs 1 and needed the budget. The own flag moves by the small
- machine's step line.

Proof of the theorem "The host is a Thiele machine" (book Theorem D.13).

Proof. The mirror's half is Theorem D.12; the own flag's half is the small machine's
- toll on the host's core. The trace form is Section 2's universal floor.

Proof of the theorem "Rice's theorem on the host machine" (book Theorem D.14).

Proof. The reduction above. The prefix before y leaves every register but the two fresh ones as it found them, and empties those, so when H halts the built program ends in whatever state y ends in from the same input, with the same ledger, table, channel, trap and flag. When H doesn't halt, the built program stops on no input. You don't even need to know whether the looping program has Π: ask the decider
- about it, and its answer says which of y and n to put at the end.

Proof of the theorem "Rice's theorem for the small machine, from the clean start" (book Theorem D.15).

Proof. The road of Theorem D.14, with the input dropped. Take a two-counter program H with a start (a,b), and let y have Π and n lack it; as before, if the program that loops has Π, swap the roles of y and n. Build the program that does INC A a times and INC B b times, runs a copy of H with its jumps moved down (a jump to a line where H has no instruction, line 0 or past its end, goes to the line just after the copy), empties A and then B with a one-line DEC loop each, and then runs y with its jumps moved down. From the clean start the copy of H begins at (a,b), which is H's own start, so it stops exactly when H does. If it does, both counters are emptied and y runs from $(0,0)$. Every instruction before y is an INC or a DEC: it adds nothing to the ledger and touches no fact, channel, trap latch or flag. It does move the versions of A and B up, each by the number of writes that

counter took before y started, and the facts y records carry versions. But a CHECK or a COMMIT reads versions only to ask whether two versions of the same counter are equal, and adding one fixed amount to both keeps the answer. So y takes the same steps inside the built program as it takes from the clean start, with the same counters, trap latch, ledger and flag at every point, and with facts that differ only by that fixed shift in their versions. That shift is why the fact table and the channel are left out of "behaves the same from the clean start." If H doesn't stop, neither does the built program: the copy of H never reaches a HALT, and nothing before y can trap. So the built program behaves the same as y when H stops from (a, b), and the same as the two-line program that loops, INC A then DEC A back to line 1, when it doesn't. A decider for Π would then decide whether H stops from (a, b), with the consequence stated in Theorem 15.5; which of y and n goes at the end is the decider's own answer about the looping program. "Halts from the clean start" is carried by behaving the same from the clean start, held by the empty program and lacked by the looping one, so it's covered. ∎

Proof of the theorem "s-m-n, and a universal program for host programs" (book Theorem D.16).

Proof. (1) The prefix writes register 2 c times and nothing else, so it moves that register's version from 0 to c. A fact records a version, so after the prefix every later version of register 2, and the version in every fact the run records about register 2, is c more than in a clean run. A CHECK or a COMMIT reads versions only to ask whether two versions of the same register are equal, and adding c to both keeps the answer, so the two runs take the same steps. The run is V's run from the stated start, moved down by c lines. (2) is the evaluator. A finite algorithm decodes a number into a program, runs it step by step on given registers, record layer included, and, searching over the number of steps, returns the output of the first halting run. It is a finite tape algorithm, and the translation of Appendix K turns any finite tape algorithm into a finite program of INC and DEC on finitely many counters. The host runs those instructions on registers of its own, with HALT marking the returned output. That's W. ∎

Proof of the theorem "Kleene's recursion theorem on the host machine" (book Theorem D.17).

Proof. (1) Let E be the following program of INC and DEC, built the same way as W from one finite evaluator, with T's number t written in. On inputs x and c it computes the number of $\mathrm{spec}(c, P_c)$, where P_c is the program numbered c, runs T on that number to get d, and then runs the program numbered d on x. Let c_0 be the number of E, and let $e = \mathrm{spec}(c_0, E)$. By (1) of Theorem D.16, e on input x does

what E does on x and c_0. And E on c_0 builds the number of $\mathrm{spec}(c_0, P_{c_0})$, which is the number of e, since P_{c_0} is E. So e hands its own number to T, gets d, and runs d on x. (2) and (3) are (1) with T carrying out F or f. ■

Proof of the theorem “The recursion theorem, up to the record” (book Theorem D.18).

Proof. The construction is Theorem D.17’s, with a different evaluator. The first step is to see that a record is small. From a clean start, a host program that stops has a record I can read off five numbers, in this order: what it left in register 0; how many facts the table holds; what is left of the ledger once the facts, the trap and the flag are paid for, which I’ll call the replay allowance; 1 if the flag is up and 0 if not; and 1 if the trap latch is up and 0 if not. The record obeys a short rulebook, and you can read each rule off the price table. A run stops at its first trap, so its ledger is its passing CHECKs, plus its passing COMMITs, plus its passing CERTIFYs, plus one if it trapped (only those three can fail, and the failing one is charged). Its passing CHECKs are its facts, at most sixteen. Its flag is up exactly when some CERTIFY passed. So the allowance is the passing COMMITs plus the passing CERTIFYs less one for the flag, which is never negative, and the ledger is the facts, plus that allowance, plus the flag, plus the trap. A COMMIT needs a fact behind it, so a positive allowance means at least one fact. A CERTIFY needs a commitment, so a raised flag means a passing COMMIT, and so a positive allowance. And the channel is empty exactly when no COMMIT passed, which is exactly when the allowance is zero, since a CERTIFY can’t pass before a COMMIT has. The allowance counts repeated CERTIFYs as well as COMMITs; it counts the cost the replay can spend on commitments, not the original commitment events. Those are the only properties of a record the replay will use.

The second step is five blocks, one for each number. Each block reads the code of T, runs T on the number of the specialised program, runs the program that comes out on x, and returns its number. All of that happens on plain data: the blocks are programs of INC and DEC, built the way W was, from a finite evaluator translated into INC and DEC. So if T has marks of its own, they are simulated and never paid.

The third step is the payment. The evaluator puts 1 in a fresh register, which is $\langle 0, 0\rangle$, the pair of Zero and 0, so it satisfies the host’s one property, and CHECKs it once for every fact; there are at most sixteen, so the table has room. It COMMITs to that claim once for every unit in the replay allowance. It CERTIFYs once if the flag is up. It copies the first number into register 0. And if the trap latch is up, it CHECKs a register that holds 0, which fails, since 0 is no pair, and the host traps. Everything before the payments is INC and DEC and costs nothing, and the payments cost what the record costs, so the ledger is exact. Specialise the evaluator to its own number, as before. ■

Proof of the theorem "No fixed point of full behaviour" (book Theorem D.19).

Proof. Let $F(p)$ be the one-line program INC r, with r one above the number of p. To carry this out on numbers, add one to the input code, encode INC with that register, and encode the one-element instruction list. These are finite arithmetic operations of the counter construction in Appendix K. A program's number is at least every register it names: the number of an instruction is above its register, and the code of a list is above every instruction in it. So e can't name r, the register $F(e)$ writes. On input 0 the run of $F(e)$ ends with 1 in register r, and if e stops at all, it ends with 0 there. If e doesn't stop, then it already differs from $F(e)$, which does. Either way they don't behave the same. ∎

Proof of the theorem "The diagonal on the host machine" (book Theorem D.20).

Proof. (1) The flip, send programs that d calls good to the program lacking Π and the others to the one having it, is carried out by a host program by hypothesis. Theorem D.17(2) gives a program e that computes what its image computes; Π holds of e exactly when it holds of the image, and the image lacks it exactly when d calls e good, so d is wrong at e. (2) From a deciding program D build a host program that computes D's flip on numbers, by the finite-simulation search of Appendix K, reading the Boolean from the first halting run of D; then (1) applies. This one is a plain no: the flip (1) needs is built from D itself, so no premise about what can be represented is left over. ∎

Proof of the theorem "Rice's theorem for the two-counter machine, on any inputs" (book Theorem D.21).

Proof. For every input: given a two-counter instance H starting from counters a and b, build a program that turns the input of n pebbles in bin A into $(6n+5)\cdot 2^a \cdot 3^b$, runs H inside the exponents of 2 and 3, divides out every 2 and every 3 when H stops, takes away 5, divides by 6, and then runs the program that has the property. Because $6n+5$ shares no factor with 2 or 3, the coding never mistakes it for an exponent, and the input comes back whole. If H halts, the built program behaves the same as the one with the property on every input; if not, it never stops on any input, like the program that loops. As before, if the looping program has the property, put a program that lacks it at the end instead, and the decider's own answer about the looping program says which. So a decider for the property would decide two-counter halting. ∎

Proof of the theorem "Kleene for the two-counter machine, inputs written as powers of two" (book Theorem D.22).

Proof. (1) The host's three pieces, with two counters in place of registers. First, the universal program. Load a program's number e and its input x together as $2^x \cdot 3^e$ in bin A, with bin B empty. The evaluator of Appendix K, the one that searches budgets and runs the program numbered e on x, is a finite counter program on finitely many counters, and packing those counters into two by the prime-exponent construction of Appendix K.1 gives U_2: started on $2^x \cdot 3^e$ it stops with 2^y in bin A exactly when the program numbered e, started on 2^x, stops with 2^y. Second, specialising. For a number c and a program V, let spec(c, V) be c copies of a block that multiplies bin A by three, followed by V with its jumps moved down. The block is INC and DEC only, so it adds nothing to the ledger and records nothing, and started on 2^x the c copies leave $2^x \cdot 3^c$; so spec(c, V) on 2^x does what V does on $2^x \cdot 3^c$. Third, the fixed point. Let T be a program that carries out F: on 2^n, n the number of a program p, it stops with 2^m, m the number of $F(p)$. Build E from the same finite evaluator, with T's number written in, and pack it into two counters as before: on $2^x \cdot 3^c$ it computes the number of spec(c, P_c), where P_c is the program numbered c, runs T on that number to get d, and runs the program numbered d on 2^x. Let c_0 be the number of E and $e = \text{spec}(c_0, E)$. On 2^x, e does what E does on $2^x \cdot 3^{c_0}$: it builds the number of spec(c_0, P_{c_0}), which is the number of e itself since P_{c_0} is E, hands it to T, gets the number of $F(e)$, and runs $F(e)$ on 2^x. The machine is deterministic, so a packed output is unique, and for every x and y, e started on 2^x stops with 2^y exactly when $F(e)$ does. (2) is the counterexample that follows. ∎

Proof of the theorem "No plain recursion on two counters" (book Theorem D.23).

Proof. (2) from (1): let $F(p)$ be a program that multiplies bin A by the constant c determined by the length of p. Multiplication by a fixed constant is the transfer loop of Appendix K. Decoding a finite program, counting its lines, finding the least prime above a number and writing that loop are computable operations, so a program carries out F on encoded program numbers. A fixed point would multiply every plain input by the constant forbidden by (1).

For (1), suppose an n-line program did it. Split each state into the two bin counts and a finite *control*: the line (one of the n lines, or "stopped" for any line with no instruction), the trap, whether the channel is empty, and the table read entry by entry as a fact's property, its counter, and whether it is *current* (its version is its counter's version now) or stale. Only properties the program names can enter the table, at most n of them, so an entry is one of at most $4n$ kinds, and a table of at most sixteen entries is one of at most $\sum_{k \le 16} (4n)^k \le (4n+1)^{16}$ lists. So there are at

most $Q = 4(n+1)(4n+1)^{16}$ controls. The control and the two counts decide the next control and counts: a CHECK writes a current fact, a write to a counter makes that counter's facts stale, a COMMIT passes exactly when a current fact with its property and counter is in the table, a CERTIFY passes exactly when the channel isn't empty, and the ledger, the flag, the channel's contents and the versions themselves never decide a branch. Call the control with the two counts a *configuration*.

Pick a threshold $h \geq 1$ above every number k in a property $\mathrm{Ge}(k)$ the program names. At h or above, a count answers every question the program can put to it (is it zero, is it even, is it at least k, can a DEC take one off) the same way as any other count of the same parity at h or above. Call a control together with the parities of both counts a *pattern*; there are at most $4Q = N$ of them.

The forced run. A *stage start* is a configuration with one count below h, in the *exact* bin, and the other at h or above, in the *large* bin, which holds v. From it, run the program while pretending the large bin can never fall below h: follow the exact bin exactly, the large bin only by its parity, and keep a tally of the large bin's net change. This forced run depends on the control, the exact count and the parity of v, and not on v itself. The real run does exactly what the forced run does, with the large bin at v plus the tally, up to the first step where v plus the tally drops below h, the *crossing*. A move changes a count by at most one, so at the crossing the large bin holds exactly $h - 1$.

What a forced run does. One of three things, fixed by the stage start. (a) It stops. (b) It comes back to a step with the same control, the same exact count below h and the same parity of the large bin as an earlier step; it's deterministic in those, so it repeats that stretch forever, and the exact count stays bounded. (c) Neither. Then it has at most $2Qh$ steps with the exact count below h, one per combination, so from some step on the exact count stays at h or above for good. Among any $N + 1$ consecutive steps of that tail, two have the same pattern, at steps $t_1 < t_2 \leq t_1 + N$, and between them the exact count changes by some δ, even because the parities match. If $\delta < 0$, the run from t_2 repeats the run from t_1 with the exact count lowered by $|\delta|$ each time round, so after finitely many rounds it goes below h, which the tail never does. So $\delta \geq 0$, and the stretch repeats forever, lifted by δ each time round, never dipping below h. In every case but (a), then, the forced run from some step on repeats one stretch forever, and each time round the exact bin changes by a fixed g and the large bin by a fixed amount; in (b) $g = 0$, and in (c) the stretch is at most N moves long.

What the real run does, once v is large enough for that stage start, meaning the large bin can't reach the crossing before the forced run stops or its repeating stretch has gone round once in full. If the forced run stops, the real run stops the same way, with the exact bin at a count fixed by the stage start and the large one at v plus a fixed amount. If the large bin doesn't lose each time round, the real run

never crosses and never stops. If it loses and $g = 0$, the real run crosses with both bins below a bound fixed by the stage start. The remaining case is a *transfer*: each time round the large bin loses $m > 0$ and the exact one gains $g > 0$. That's case (c), so the stretch has at most N moves, each moving one bin by at most one, and $m, g \leq N$; both are even. Now the stage rule. Start the same stage with $v + m$ in place of v. The parity is the same, so the forced run is the same, and since the crossing comes after the stretch has gone round once, the bin needs exactly one more time round to get down to $h - 1$. So the real run crosses one time round later, with the same control, the formerly large bin at $h - 1$, and the other bin larger by g. With $v + km$ it crosses k times round later, the other bin larger by kg. After a transfer, the crossing is a new stage start with the bins' jobs swapped. For example, a loop that takes two pebbles from the full bin and puts four in the other has $m = 2$ and $g = 4$: two more pebbles going in buy one more trip round the loop, and four more coming out.

Choosing the input. Apart from v, a stage start is one of finitely many things: a control, an exact count below h, a parity, and which bin is which. Take one bound $V \geq h$ that's large enough, in the sense above, for every one of them, and above every count that was called fixed or bounded above: the exact bin of a stage that stops, and both bins at a crossing with $g = 0$. The configurations with one count below h and the other below V are finitely many. If the program multiplied every input by c, no two different inputs could pass through the same configuration, because everything after it, the output included, would be the same, while $cx \neq cx'$. So only finitely many inputs pass through that finite set. Pick an input x that passes through none of it and has cx above V. Since $(x, 0)$ is itself a configuration, $x \geq V$.

Following that input. The run from $(x, 0)$ starts a stage: bin B exact at 0, bin A large at x. Every later stage start along the run has its large bin at V or above, or the run would pass through the finite set. The run stops, since the program multiplies, so no stage runs forever. No stage crosses with $g = 0$, since that crossing is in the finite set. So each stage before the last is a transfer, with its own pair (m_i, g_i), and the last one stops. It stops with its exact bin below V and its large bin at its v plus a fixed amount, and the output cx is above V, so the output bin A is the large one. There are finitely many stages, because each takes at least one step of a run that stops.

The comparison. Put $D = 2 \prod_i m_i$ and run the input $x + D$. It is x plus a multiple of m_1, so by the stage rule the first stage crosses at the same configuration except that the other bin is larger by $(D/m_1)g_1 = 2g_1 \prod_{i \geq 2} m_i$, which is a multiple of m_2. So the second stage crosses the same way, with the next bin larger by $2g_1g_2 \prod_{i \geq 3} m_i$, and so on through every stage. The last stage starts with its large bin larger by $2 \prod_i g_i$, an even amount, so it stops the same way with bin A larger by exactly that. So the outputs from $x + D$ and from x differ by $2 \prod_i g_i$. But multiplication by c

makes them differ by $cD = 2c\prod_i m_i$. Cancel two:

$$c\prod_i m_i = \prod_i g_i.$$

Empty products are 1. Every factor on the right is between 1 and N, and c is a prime above N, so it divides none of them, and a prime that divides a product divides one of its factors. So c can't divide the right side. That's the contradiction. ∎

One slot, done by hand. Do one slot by hand. The property "even" has number 1. Checking it on value 4 puts $\langle 1,4\rangle = (2\cdot 4+1)2^1 = 18$ in a separate slot. To read that pair, halve 18 until what's left is odd: one halving gives 9, so the property number is 1; subtract one from 9 and halve to recover value 4. The host checks that property of that value. A second register, the mirror, holds 2: the property number plus one. COMMIT can read this second register while leaving the checked slot untouched.

When the guest changes the counter that fact concerned, nudge the slot $18 \to 19 \to 18$ and set the mirror to zero. The number is back where it was, but the version went up twice, so the host's old fact about that slot is stale. Putting a number back doesn't put an old check back. The mirror is what tells the search to leave that slot alone, and if the search tried it anyway, the version rule would refuse it.

Act	slot value	slot edition	searchable mirror
Check evenness of encoded 4	18	v	2
Guest writes its counter: INC slot	19	$v+1$	
DEC slot; clear mirror	18	$v+2$	0

The program, register by register and job by job. **The program.** I'll describe U the way Turing would, one register at a time and then one job at a time. Registers 0 and 1 hold the guest's A and B. Register 2 holds the guest's whole program as one number: each instruction gets a number (an opcode paired with its operands, INC as 0, DEC as 1, HALT as 2, CHECK as 3, COMMIT as 4, CERTIFY as 5), and the list of them is packed with $\langle\cdot,\cdot\rangle$, the empty list as 0. Register 3 is the guest's line. Registers 4 to 13 are scratch. Registers 14 and 15 count how many CHECKs on A and on B have passed. Registers 16 to 47 are *mirrors*, sixteen for each guest counter, and registers 48 to 79 are the slots, sixteen for each guest counter. The guest's table holds sixteen facts in total, across both counters, and all sixteen can be about one counter, so the host gives each counter room for all of them. The reservation is the host's, and the guest's own capacity stays sixteen in all. Register 80 is a dead end that's never written.

Each guest step goes like this. At line 1 the program copies the program number and the guest's line into scratch, putting the originals back as it goes, because copying a counter on this machine means emptying it into two others and refilling

it. If the guest's line is 0, or past the end of the program, it stops. Otherwise it peels instructions off the front of the copy, one per line, until it holds the one at the guest's line, and splits that into opcode and operands. Then a chain of DECs on the opcode sends it to one of six handlers.

- INC c: INC the guest's counter c, nudge every slot about c and zero its mirrors, add one to the guest's line, back to line 1.
- DEC c j: DEC the guest's counter c. If it was 0, add one to the guest's line. If it wasn't, nudge every slot about c, zero its mirrors, and set the guest's line to j. Back to line 1.
- HALT: stop.
- CHECK p c: let k be the count of passed CHECKs on c. If k is below 16, build $\langle$number of p, value of $c\rangle$ in slot k of c, run CHECK Slot on that slot, set its mirror to the number of p plus one, add one to the count and to the guest's line, and go back to line 1. If k is 16, run CHECK Slot on the dead end, which holds 0, which is no pair, so it traps, just as the guest's seventeenth CHECK does.
- COMMIT p c: look through the sixteen mirrors of c for one equal to the number of p plus one. At the first match, run COMMIT Slot on that slot. With no match, run COMMIT Slot on the dead end, which traps.
- CERTIFY: run CERTIFY, add one to the guest's line, back to line 1.

Why the mirrors? COMMIT has to find the slot holding the claim, and it can't read the slot to find out, because reading would make the fact stale. So beside every slot sits a mirror the program is allowed to read: the property's number plus one while the slot's fact is live, 0 once the guest has written the counter. If a guest CHECK fails its property, the host's CHECK on the slot fails too, since the slot holds exactly the pair the guest checked. If the guest table is full across both counters, the host's own sixteen-entry table is full as well, even when this counter's slot count is below sixteen; its CHECK then traps for lack of room. So both kinds of refusal come through.

The two tally lines for the surcharge. Keep two tally lines for a guest whose first three moves each cost one and whose reading first says yes after the third. The guest's totals are 1, 2, 3. This compiler pays one PAY on each of the first two moves, then its three-act chain: the host totals are 1, 2, 5. The excess is two, added once. If the third move instead costs four, the host runs the chain and one PAY there; both totals then read 6. These are counts of charged instructions, not counts of all the work needed to decode the guest. Stopping at zero throws away what the chain paid beyond the move's price. A compiler that kept it as credit against the moves after the raise would bring the surcharge back to zero once the guest had paid three

from the raising move on, and a guest whose reading starts at yes would be matched exactly from its third unit of cost on. I keep the simpler rule, and every surcharge stated below is this compiler's.

A worked replay, with its table. Try CHECK, COMMIT, CERTIFY, CERTIFY on a true claim. The ledger is four, the table holds one fact, the flag is up and the trap is down, so the replay allowance is two. The replay pays CHECK, COMMIT, COMMIT, CERTIFY: the same ledger of four and the same summary, in a different order. The theorem claims that agreement and no more.

Run	checks	commits	certificates	total paid	final flag
Original	1	1	2	4	up
Replay	1	2	1	4	up

Each table has one fact and each channel is nonempty. The counts of the individual acts differ; the selected final summary agrees. In the replay, the trap latch is up exactly when the fifth number is 1, and the flag exactly when the fourth is positive.

A worked packing, with its table. Try input 2 while the simulated counters hold 1 and 2. The pile is $17 \cdot 2 \cdot 9 = 306$. Raising the first simulated counter doubles it to 612; lowering the second divides it by three, to 204. Strip off every factor of two and three and 17 remains; subtract five and divide by six, and the original input 2 is back.

Act	packed pile	simulated counters	input factor
Load input 2; counters 1, 2	$306 = 17 \cdot 2 \cdot 3^2$	$(1, 2)$	17
Increment first counter	$612 = 17 \cdot 2^2 \cdot 3^2$	$(2, 2)$	17
Decrement second counter	$204 = 17 \cdot 2^2 \cdot 3$	$(2, 1)$	17
Strip factors 2 and 3	17	discarded	17
Subtract 5; divide by 6	2		input recovered

CHAPTER 4

Running it

Everything so far is definitions and proofs about them. Does any of it run? This appendix is me finding out. I built the small machine twice, once straight out of the definitions and once by hand from the same description, ran both, and compared the whole state after every step. Then I ran the universal programs on guests small enough to finish, and real programs through the part of the compiler that executes.

One rule holds for the whole appendix: the runs test the builds, and the proofs carry the theorems. A run shows what a build did on the inputs it was given, and no theorem in the book leans on one: every theorem in it stands on its argument, on the page.

There are two counts to keep apart as you read. The ledger counts what the price list charges. The number of interpreter steps counts the work this particular build does to carry the run out. One guest step can take millions of host steps and add one to the ledger.

Stage	What the next stage must preserve
Source orders	Variable values and completed procedure calls
Assignments and jumps	The same branches, loops and final answer
Counter blocks	Values at block boundaries; spare counters restored
Host instructions	The same result, with the opposite DEC convention translated
Packed two-counter guest	The result as an exponent in the packed counter
Fixed interpreter U_P	Guest counters at matching points; decode the exponent to read the answer

This is a path through the compiler, not a claim that one step at each stage takes the same time. The source compiler inserts no certification chain. Running the executable uses physical hardware and energy. These runs did not measure heat per host step, and the ledger is not a heat measurement. The times only tell you which runs are practical on this build. The table shows what ran and what didn't.

What	Status
The small machine	run: the executable versions agree on every tested step
The host machines, with and without PAY	run, compared the same way
U and U_P on tiny guests	run, against a separate guest implementation
U on a guest that certifies	proved and counted, not run
U_P on a guest that certifies	proved, not run
Source programs through the compiler	run as far as the host program
The two-counter guest and U_P running it	proved, not run
The presented machine carried by U_P	proved, not run

4.1 Taking the coat off

The first executable is generated straight from the formal definitions, the way you'd strip the coat off a proof and keep the program underneath. The second was written separately, by hand, from the same description. Both implement the small machine, the host with a register for every whole number, the host with PAY and the loaders, and both run the universal programs; their instruction lists the second build reads as data printed from the definitions, so it checks them by running them, not by writing them again. The compiler below has an executable runner too.

Whole numbers matter here. A machine word in one build is 63 bits wide, and the guest codes U reads go up past 2^{100}, so ordinary fixed-width arithmetic would wrap around and lie. The mathematical machine never overflows, so its executable uses integers of any size, with the arithmetic chosen to match the definitions: subtraction stops at zero and division by zero gives zero. What sits between the definitions and the running code, the mappings, the compilers and the routines that read input and write output, is code I had to trust.

4.2 A second machine, written by hand, to check the first

Comparing two builds step by step catches one of them disagreeing with the description, on the runs I tried. They're compared on the *whole* state after *every* step: the program line, every counter and register and its version, the fact table in order, the channel, the trap latch, the ledger and the flag. The comparison covers exhaustive sets of short programs, random programs and both hosts, and they agree on every

tested step. U and U_P on tiny guests are compared at fifteen checkpoints two hundred steps apart, over the first three thousand host steps of each run, instead of after every step, and the longer runs below against a separate guest implementation, once per pass of the loop and at the stop. I also broke the machine on purpose, giving it a fact table one too small, a CHECK, COMMIT and CERTIFY that cost nothing, a commitment that ignores versions, and a write that doesn't bump the version, and each break gets caught. A second set pins the boundaries this book draws in words, and those runs come out as the words say: two runs that end in the same state with different first certifications, a check that can't fail certifying from every start, a CHECK and six commitments costing seven without a flag, a write after the commitment and a flag over a claim that's now false, sixteen copies of one CHECK filling the table, a CERTIFY costing one at a ledger already at two, and a merge between two yes-states that leaves the entering state alone.

A mistake both builds share gets past the comparison, and so does a mistake in the definitions themselves, since both builds follow them.

4.3 U on a real guest

Now the universal program, which is where I expected trouble. The guest I wanted to run first was the obvious one, a guest that goes the whole way: CHECK, COMMIT, CERTIFY, flag up, run inside U on the built machine. That isn't what ran, and the reason is the way the guest is stored. U keeps the whole guest program as one number, a number that grows like two raised to the sum of the codes for the guest's instructions (for a guest of one instruction it's exactly a power of two), and it reads that number the only way a counter can, one unit at a time. So its speed depends on the code, and the code grows fast: a guest with one CHECK in it has code 2^{24}, and a guest with the whole chain has a code near 2^{106}.

So I came down to what could finish. The table below is what ran, on the executable host, compared against a separate guest implementation at every pass of the loop and at the stop: the host stopped exactly when the guest did, and at the stop showed the guest's counters, trap latch, ledger and flag.

guest	host	host steps	time
count A down from 2	U	25,953,317	
one passing CHECK	U	402,654,059	618 s
one failing CHECK	U	201,327,155	305 s
one passing CHECK	U_P	402,654,059	617 s
one failing CHECK	U_P	201,327,155	313 s

The first row's count is pinned exactly: if the build changed so that it moved by one step, the comparison would fail. The times are from one ordinary computer and are only there to give you the scale. The counts work out to about twelve host

steps for every unit of the guest's code each time U goes round its loop, which it does once for every guest step and once more to see that the guest has stopped: the failing CHECK traps on its first pass, 12×2^{24} and a little, and the passing one takes two passes, 24×2^{24} and a little. The growth is exponential in the size of the guest. A guest with one CERTIFY in it would have code 2^{32}, about fifty billion host steps, and at the measured rate of about 650,000 steps a second it would take something like twenty-two hours, nearly a day. The guest I wanted, with the whole chain, has a code near 2^{106}, which at the same twelve steps a unit is about 10^{33} host steps for each pass; at the measured rate, with this encoding on that computer, a run of its three steps comes to around 10^{20} years. So U on a guest that certifies is proved, in Appendix D.1, and has never been run. Runs this long only fit in memory at all because the build can repack the host's register store in the middle of a run; repacking changes how the registers are stored, not what they hold.

What a run that long can be is counted. Four loop shapes do most of U's work. Count from the DEC instruction that tests the loop counter; any instructions before that test are still counted separately. You can derive each shortcut from the arrows:

- In the emptying loop, a positive counter v takes one DEC back to its own test and becomes $v-1$. At zero, one DEC exits. Thus $E(0)=1$ and $E(v+1)=1+E(v)$, giving $E(v)=v+1$.
- In the copying loop, a positive DEC jumps back one line to INC on a distinct destination, which returns to the test. Each unit moved therefore takes two steps, and the zero test takes one: $C(0)=1$, $C(v+1)=2+C(v)$, so $C(v)=2v+1$.
- With two destination INCs before the test, each positive pass takes three steps. Thus $D(0)=1$, $D(v+1)=3+D(v)$ and $D(v)=3v+1$. If the two destinations are the same register, it receives two increments per pass; the step count stays three.
- In the halving loop, two successful DECs on the source and one INC on the quotient return to the same test. After $k=\lfloor v/2 \rfloor$ such passes, $3k$ steps have removed $2k$ from the source and added k to the quotient. The remaining source is zero or one. Follow that final branch instruction by instruction.

Each successful decrement or increment also raises its register's version once. The zero test does not write. So the same counts determine the version changes as well as the counter values and next line. These loops neither charge nor change the table, channel, trap or flag. Replacing one completed loop stretch by its derived effect and count therefore preserves the whole endpoint and the number of steps. Repeating that replacement throughout a run preserves both by addition.

The numerical tally below applies those four rules and steps the other instructions individually. It agrees with the three runs above that were also stepped all the way. For the guest CHECK, COMMIT, CERTIFY, using the zero predicate on A, the passing

start $(0,0)$ ends with the flag up and ledger three. From $(5,0)$ the first check fails and the guest stops with ledger one. Appendix L supplies the program blocks and their guest correspondence; the displayed totals count that particular encoding.

start	host steps
$(0,0)$	3,894,222,629,847,358,066,688,933,266,198,944
$(5,0)$	973,555,646,581,507,138,873,939,791,446,643

What that table shows is a count, not a run. Nothing stepped those 10^{33} instructions, and the row in the table at the top of this section says so.

Those big counts come from this encoding and from reading it one pebble at a time. They measure the work this interpreter does, and that's all they measure. Whether some other interpreter, or some other base, could do the same job fast is a separate construction with its own running-time analysis. Nothing measured here puts a floor under every way of doing it.

4.4 From source programs to the host

Running guests one CHECK at a time doesn't tell you whether the built machine can do anything you'd recognise as a program. You'd want to hand it gcd, or a sort, written the way people write them, and get the answer out, and writing gcd by hand in INC and DEC is no way to find out. So the last piece of the build is a compiler. The source is a small language you could write by hand: whole-number variables, addition, subtraction that stops at zero, comparisons, *if*, *while*, and procedures that return several results, with the rule that a procedure can only call ones defined before it, so there's no recursion. Each translation stage has a proof that it preserves the program's result: the procedures are inlined, the result is flattened to assignments and jumps, that is compiled to Minsky's counter machine through the counter-machine compiler, that to a program for the host machine, that to a two-counter guest, and the guest to a run of U_P.

□ **Theorem 4.1** (The compiler is correct)**.** For every source program with no recursion, and every input: the program computes y exactly when the runner (the bounded evaluator of the compiled host program) halts with y for some amount of fuel; exactly when the compiled host program halts with y; exactly when the two-counter guest halts with y; and exactly when U_P running that guest halts with y, with its first counter at 0, its ledger at 0 and its flag down. So a terminating compiled program delivers its answer and certifies nothing. To certify a claim about that answer, a target program must add a CHECK, a COMMIT and a CERTIFY; this compiler does not insert that chain. The runner also takes exactly as many steps as the host

program does, with the rest of its fuel left over, and a run that doesn't halt has used all its fuel.

Proof. Six arrows and a runner, each argued below, and the statement is their composition. Four of the arrows are between machines that take steps, and for those one argument does both directions, so I give it first.

The simulation argument. Take two machines S and T, each deterministic, where S has a next state from every state inside its program, and a relation $\sim$ between their states such that: a state of S inside its program is related only to states of T inside its program, and a state of S outside it only to states of T outside it; and from any related pair with S inside its program, one step of S is matched by one or more steps of T that land related to S's next state. Then from related starts, S halts exactly when T does, and they halt related. Forward: start with the related pair. Follow the stretch of T that matches S's first step; its endpoint is the related start needed for the second stretch. Keep joining stretches until S stops. There are only finitely many steps to match, so this gives a finite run of T whose final state is related to S's final state. The latter is outside its program, so the former is outside its own. Backward: suppose T halts after k steps and S doesn't halt. Then S takes at least $k+1$ steps without leaving its program, the matched stretches give T a run of at least $k+1$ steps all of whose stretch ends are inside its program, and T, being deterministic, has only one run from its start, which halted at step k. That's a contradiction, so S halts, and by the forward direction it halts related to where T did.

Unfolding the calls. Imagine a program written on cards. A call says "carry out the orders on that other card, then come back." Here a card may call only a lower-numbered card. Card zero therefore calls nothing. Replace every call on card one by the orders from card zero; then do card two, using the already expanded lower cards, and continue. There are finitely many cards, and every replacement has already been built. The process ends with calls replaced by actual orders.

The variables need their own places too. Give each active procedure a block of numbered boxes, its *frame*. If the caller's block starts at box b and uses n boxes, put the called procedure's block immediately after it, starting at $b+n$. A call becomes four pieces: copy the arguments into that new block; set its other boxes to zero; carry out the expanded orders using those box numbers; copy the answers back into the caller's named boxes. A local variable numbered x in the called procedure is thus the global box numbered $b+n+x$. No assignment reaches below the procedure's own block.

Why does this preserve a terminating run? Begin with the lowest-numbered procedure. It has no calls, so its expanded orders are its original orders with the boxes renamed. Assignments change the same values, tests choose the same branches, and loops repeat the same bodies. Now take a procedure whose lower-

numbered calls have already been settled. Each replacement call builds exactly the arguments and zeros that the original call would receive. The expanded lower procedure returns the same answers, and the copy-back puts them in the same caller boxes. The caller's surrounding orders then continue from the same values. Working upward through the numbered procedures settles the main program.

The comparison works backward as well. Whenever an expanded call finishes, its four pieces have run in their written order. The middle piece is the expanded lower procedure, whose terminating runs I have already matched to the original lower procedure. Its preparation and copy-back are just the original call's input and output operations. Replace that finished four-piece execution by the call it implements. The surrounding assignments, branches and completed loop repetitions then give the same original execution. A finite execution in either description therefore gives a finite execution in the other, with the same answer. Neither can terminate when the other does not.

In symbols, matching a source arrangement e to the global boxes G in a frame starting at b means $G(b+x) = e(x)$ for each local box x. The comparison above preserves that equality at the beginning and end of every translated statement.

Flattening. The flat code of a statement placed at address a: an assignment is one assignment; a sequence is the first statement's code followed by the second's; *if* b is a jump on false to the else part, the then part, a jump past the else part, and the else part; and *while* b with a body of l instructions is a jump on false to $a+l+2$, the body at $a+1$, and a jump back to a. Every jump in the code of a statement lands inside that code or at its end, the address just past it. Now follow a completed structured execution. An assignment performs the same change; a sequence joins its two runs; a branch follows the same test answer. For each loop iteration, the test falls through, the body's code takes the run to $a+l+1$, and the jump returns it to a. Repeat for the remaining iterations; the final false test jumps to the end. Thus an execution from variables e to e_1 gives a flat run from address a to the end of the code with those same final variables.

Read a completed flat run in the other direction. Because no jump escapes past the code's end, a run that leaves it must visit that end first. Assignments and branches recover their structured operations immediately; a sequence splits at the second statement's first address. For a loop, the run's first step is the test. If it is false, the run jumps to the end, and the loop's derivation is the one with a false test. If it is true, the run enters the body's code, which it can leave only through its end at $a+l+1$ (the body's own backward statement), where the jump returns it to a with a strictly shorter run left. Read the next iteration the same way. This cannot continue beyond the finite run I started with: its final test must be false, giving the last piece of the structured loop execution. So the flat program at address

1 halts exactly when the structured statement has a derivation, with the same final variables.

Counter machine. Here a DEC jumps when its register is zero and goes on otherwise. Variable x lives in register $x+1$, register 0 is kept at 0, so DEC 0 j is a plain jump to j, and registers above the variables are working space. The relation: register 0 is 0, registers 1 to n hold the flat variables, and every register above is 0. Each flat instruction becomes a block built from small loops. Follow one pass and then count how many pebbles remain to be moved: Emptying a register: each pass takes one off, and the loop exits at zero. Moving s into d: each pass takes one off s and adds one to d, so $s+d$ never changes, and at exit s is 0 and d holds the old sum. Copying: move s into two registers at once, then move one of them back. Cut-off subtraction $t-u$: take one off each while both are positive; at exit one of them is 0, and the other holds the difference, or 0. Comparison: take one off each in step; whichever reaches zero first, or both together, decides $<$ and $=$, and the block empties both. Build a larger expression from these smaller jobs. A constant is a fixed number of increments into an empty working register. A variable is copied there with its original restored. For a sum or cut-off subtraction, compute the two smaller expressions in separate working registers, combine them with the loops just described, and empty the temporary registers. Starting with the innermost pieces and working outward, the final value lands in register t, the registers above t return to zero, and those below t are left alone. A test computes its comparison in the same spare registers, chooses the true or false exit, and clears the spare registers on either exit. Combining tests follows the selected exits of their smaller tests, each of which has already restored the registers. Thus a test leaves the registers as it found them and changes only which address comes next. So an assignment block computes the value into the first working register, empties register $x+1$, moves the value in, and lands related; a jump on false runs the test's code with the true exit at the next block and the false exit at the block of j; a jump is DEC 0 j. The linker places the blocks one after another and sends every jump to the start of its target's block. That's the one-step matching. The flat machine has a next state from every address in its program, and an instruction that names a variable outside the frame just advances, so the step is total. The counter machine is deterministic. The simulation argument gives both directions: the counter program from the loaded inputs leaves its code with register $out+1$ equal to y exactly when the flat program halts with y in out.

Host. The host's DEC is the other way round: it jumps when its register is positive, taking one off, and goes on at zero. An INC x stays INC x. A DEC x j at host address a becomes three instructions: DEC x $(a+3)$, then INC 0, then DEC 0 j', where j' is the host address of j's block. If register x is positive, the first takes one off and jumps to $a+3$, the start of the next block, which is what the counter machine does. If it is

0, the run goes on to INC 0, which raises register 0 to one more than it held, and DEC 0 j' finds it positive, takes the one back off and jumps to j', which is the counter machine's jump at zero, with register 0 as it was. The relation: the same registers, at corresponding addresses. Each counter-machine step is matched by one or three host steps. The host program has no CHECK, COMMIT, CERTIFY or PAY, and INC and DEC change only a register and the line, at no cost, so every host state in the run has the start's facts, channel, trap latch, ledger and flag. The counter machine's step is total, the host is deterministic, and the simulation argument gives: the host program halts with register $out + 1$ equal to y exactly when the counter program leaves its code with that register equal to y.

Guest. The host program's registers $r_0, \ldots, r_{k-1}$ are packed into the guest's counter B as $\prod_i p_i^{r_i}$, one prime p_i per register, with counter A the spare. That is the packing of Appendix K.1, with the two counters' jobs swapped. The relation: the guest is at the start of the block for the matching instruction, B holds the code of the registers, and A is 0. An INC on register i is the stretch that multiplies B by p_i; a DEC on register i is the stretch that divides B by p_i with remainder and takes the positive branch when the remainder is 0, the zero branch otherwise, restoring B. The appendix argues each stretch: it ends, because each of its loops empties the counter it counts down; it lands with A at 0; and, by unique factorisation, it changes the exponent of p_i alone, by the right amount. The host's step is total on these programs and the guest is deterministic, so the simulation argument gives: the guest halts with B coding registers whose register $out + 1$ is y, that is, with y the exponent of that register's prime in B, exactly when the host program halts with that register at y. The guest has no record instruction, so its ledger stays 0 and its flag down.

U_P. U_P keeps pace with its guest the way U does, with the PAY case added (the proof of Theorem D.7). This guest has no PAY and no record instruction, so on it U_P runs only branches U has, and the argument of Theorems D.2 and D.3 goes through: from a freshly loaded host U_P halts exactly when the guest does, with its registers 0 and 1 holding the guest's A and B, and its ledger and flag equal to the guest's. So it halts with register 0 at 0, register 1 the code with y in it, ledger 0 and flag down.

The runner. The runner keeps the host program and the registers in binary tries, a tree indexed by the bits of a number. Read the address one bit at a time: a zero chooses one branch and a one the other. Looking up an address follows that route to its stored value, or returns the default zero if no value was stored there. Storing at an address follows the same route and changes its final value, creating missing branches as needed. Every other address either branches off before that end or names a different node, and keeps its value. Thus the tree performs the same lookup and single-address update as the register map it represents. So each round of the runner's loop does exactly what one host step does to the same registers, and the

runner stops when the line leaves the program or the fuel is spent. If it halts after k rounds, the host run of k steps is halted at the end of the program with the runner's registers, and the fuel left is the fuel given less k. If the source program has a derivation, the arrows above give a halting host run, of some length k, and the host, being deterministic, has only that run; any fuel of at least k makes the runner take those k steps and stop. If the runner hasn't halted, it used every unit of fuel on host steps that stayed inside the program, so the host is still running after that many steps, and a derivation, which would give a halting host run of some length k, would need k larger than the fuel.

Composing: the program computes y exactly when the unfolded program does, exactly when the flat program halts with y, exactly when the counter program does, exactly when the host program does, exactly when the guest halts with y coded in B, exactly when U_P halts as stated, and exactly when the runner halts with y on some fuel. ∎

The theorem says nothing about how many steps the compiled program takes next to the source; it's about answers.

The runner and the compiler are implemented, and the build has been run on a set of programs I picked to be annoying: gcd, factorial, a primality program written with procedures, an Ackermann tower, a sorting network on a list packed into one number, and a tiny interpreter for another machine written in the source language. Then come 220 random programs that halt, 160 random programs with loops that might not, programs that never halt run for a fixed budget, and programs mutated on purpose to make sure the comparison notices. Answers are compared with direct arithmetic and separate interpreters of the source and compiled programs. Ackermann's function at $(3, 5)$ is 253; the source program takes 64,768 operations to get there, and the compiled host program takes about 119 million steps. The executable runner does a few million host steps a second, so that's under a minute.

That works out to about 1,800 host steps per source operation for Ackermann, and up to about a hundred thousand for multiplication by repeated addition, because every addition copies a running total one unit at a time. On this build the work grows with the size of the numbers being handled, because every operation is one pebble at a time. How it grows with the size of the program I haven't measured. The last two stages, the two-counter guest and U_P running it, are proved and have never been run, and U_P on a compiled guest would be slow for the same reason U on a certifying guest is. Still, I like that 253. It came out of nothing but INC and DEC, and a proof that they mean what the source meant.

4.5 What wasn't built

There's no chip. The model is not a circuit, and the lack of one says nothing about it. The example was never cut into metal: there's no circuit for the small machine, no simulation of one at the level of gates, and no board ever ran any of this. The small machine exists as mathematical definitions and executable implementations that agree on every tested step. That's the whole of what "it runs" means in this book.

I'd love to see it in metal some day. But builds that can be held against each other on every step are worth a lot to me. A board with nothing to hold it up against isn't, and that's the trade I made.

CHAPTER 5

Constructing the quoting operation

The proof in Section 13 uses a program that takes the code of a term and returns the code of that code. Here's how to build it.

Draw the instruction "add one" on a card. Now make a second card describing the first card's marks, rather than carrying out the addition. The count has not risen: you have described an instruction. Describe the second card in turn and you have a description of a description. Quoting does that second job mechanically. In the language below, take the variable of index 0. Its code is a tree with three lambda nodes, then an application of variable 2 to the numeral for 0. Quoting that code builds a description of those three lambdas and that application; it does not look up variable 0 or execute the term it describes.

Nothing in the operation lets a program inspect itself by magic: its input is an ordinary tree encoded as a closed lambda term.

The construction follows Forster and Smolka (2017), whose construction for L has the same three syntax builders on Scott-encoded children, an auxiliary procedure for numerals, and a quote procedure Q with the same specification, defined by the same recursion on codes; the self-application that uses Q in Section 13 is their fixed-point lemma, after Barendregt's (1984) proof for the full lambda calculus. What follows is my rebuilding of it, with the reductions written out.

5.1 Numerals and syntax as data

For readability, the formulas below give names to bound variables. Renaming a bound variable changes nothing; the underlying syntax uses the de Bruijn indices of Definition 13.3. Applications associate to the left. Write $X \Downarrow Y$ here for a finite sequence of the weak call-by-value steps ending in the value Y.

Scott numerals are the following closed values:

$$\overline{0} = \lambda z.\lambda s.z, \qquad \overline{n+1} = \lambda z.\lambda s.s\,\overline{n}.$$

So a numeral selects either the supplied zero case or the supplied successor case, passing its predecessor to the latter. The cases below are values, so choosing one does not prematurely run the other.

The three syntax constructors, operating on already encoded children, are

$$\begin{aligned} \text{mkvar} &= \lambda n.\lambda v.\lambda a.\lambda l.(v\,n),\\ \text{mkapp} &= \lambda p.\lambda q.\lambda v.\lambda a.\lambda l.a\,p\,q,\\ \text{mklam} &= \lambda p.\lambda v.\lambda a.\lambda l.l\,p. \end{aligned}$$

Each builder is a term of the language L of Definition 13.3. For a variable of index n, an application and an abstraction, respectively, define their codes by the following resulting values:

$$\begin{aligned} \ulcorner \#n \urcorner &= \lambda v.\lambda a.\lambda l.(v\,\overline{n}),\\ \ulcorner p\,q \urcorner &= \lambda v.\lambda a.\lambda l.a \ulcorner p \urcorner \ulcorner q \urcorner,\\ \ulcorner \lambda p \urcorner &= \lambda v.\lambda a.\lambda l.l \ulcorner p \urcorner. \end{aligned}$$

In particular, $\text{mkvar}\,\overline{n}$, $\text{mkapp}\ulcorner p \urcorner\ulcorner q \urcorner$ and $\text{mklam}\ulcorner p \urcorner$ reduce to these three codes. A code is closed even when the term it describes has a free variable: the index of that variable is stored as a numeral. It is data inside the code, and the code itself has no free variable.

5.2 Recursion without running under a lambda

The quoting procedure will have to call itself on the smaller pieces of a description. There is a catch: this language works out an argument before handing it to a function. Hand over the recursive calculation itself and it starts calculating at once, before the receiving function has decided whether it needs it. That can send you straight back into the same calculation forever.

Instead, hand over a recipe waiting for an ingredient. In this language a lambda is such a recipe, and it counts as a finished value even though its body has not run. The receiving function can keep the recipe and call it when it reaches a smaller piece. In the expression below, the little recipe $\lambda a.x\,x\,f\,a$ is the part doing that job. The letter a names the argument it is waiting for; f names the work to do once the recipe and the current input have been handed over.

$$W = \lambda x.\lambda f.\lambda v.\ f\,(\lambda a.x\,x\,f\,a)\,v, \qquad \text{rec}(F) = \lambda v.W\,W\,F\,v.$$

When F and u are closed values, beta reduction gives

$$\text{rec}(F)\,u \longrightarrow W\,W\,F\,u \longrightarrow^{*} F\,(\lambda a.W\,W\,F\,a)\,u = F\,(\text{rec}(F))\,u.$$

The recursive argument is a lambda, so it's already a value. It starts work only when F applies it to an argument. That is the whole reason this construction survives the call-by-value rule.

5.3 Quoting numerals

Let $E_j = \ulcorner \#j \urcorner$ for $j = 0, 1, 2$, and let $Z = \ulcorner \overline{0} \urcorner$, obtained directly from the finite syntax of $\overline{0} = \lambda\lambda.1$. Write $\text{mklam}^k(X)$ for k nested applications of the syntax builder mklam to X; it is an abbreviation for constructing k lambda nodes, and the language gains no new operation from it.

Define

$$F_N = \lambda r.\lambda n.\ n\, Z\, (\lambda k.\text{mklam}^2(\text{mkapp}\, E_0\, (r\, k))), \qquad Q_N = \text{rec}(F_N).$$

Follow the two branches. Zero chooses the already built description Z. A number one larger than n chooses the other branch and hands it the predecessor n. That branch first asks the same procedure to describe the predecessor. Then it puts the description inside an application of variable zero and wraps two lambdas around it. Those are precisely the marks in the numeral for $n + 1$.

Zero therefore works. If the procedure describes n correctly, the wrapping just described makes it describe $n + 1$ correctly. Starting at zero and taking one more step proves the claim for every natural number. Every call on a positive number asks about a smaller one, so it reaches zero and finishes. In the notation for termination and returned value, the result is $Q_N\, \overline{n} \Downarrow \ulcorner \overline{n} \urcorner$.

Do the smallest case by hand. Call the successor case G. Then $Q_N\, \overline{1}$ steps to $F_N\, Q_N\, \overline{1}$ and on to $\overline{1}\, Z\, G$; the numeral hands its predecessor $\overline{0}$ to G, the recursive call $Q_N\, \overline{0}$ comes back as Z, and the builders return $\text{mklam}(\text{mklam}(\text{mkapp}\, E_0\, Z))$. That is the code of $\lambda\lambda.(\#0\, \overline{0})$, which is $\overline{1}$.

5.4 Quoting terms

For a proposed recursive call r, define three handlers:

$$\begin{aligned} H_V &= \lambda n.\text{mklam}^3(\text{mkapp}\, E_2\, (Q_N\, n)), \\ H_A(r) &= \lambda p.\lambda q.\text{mklam}^3(\text{mkapp}\, (\text{mkapp}\, E_1\, (r\, p))\, (r\, q)), \\ H_L(r) &= \lambda p.\text{mklam}^3(\text{mkapp}\, E_0\, (r\, p)). \end{aligned}$$

Now set

$$F_Q = \lambda r.\lambda t.t\, H_V\, (H_A(r))\, (H_L(r)), \qquad Q = \text{rec}(F_Q).$$

The handlers are lambdas and so are values. The Scott code of the input chooses one of them and hands it the encoded children. Start at the leaves and build outward.

At each node the handler builds the syntax of that node's encoding: $Q\ulcorner t\urcorner \Downarrow \ulcorner\ulcorner t\urcorner\urcorner$. There are three kinds of node to consider:

- For $t = \#n$, the first handler constructs three lambda nodes over an application of variable 2 to the quoted numeral. That is the syntax of $\lambda\lambda\lambda.(\#2\,\overline{n})$, the code of $\#n$.
- For $t = p\,q$, quote the two children first. The second handler constructs three lambdas over variable 1 applied to those two codes. That is the syntax of the application code.
- For $t = \lambda p$, the last handler constructs three lambdas over variable 0 applied to the quoted child code. That is the syntax of the abstraction code.

Each recursive call works on a proper child of the finite input tree, or on a smaller numeral. No chain of calls can keep descending through a finite tree or decreasing a natural number forever. Each builder takes finitely many beta steps, so the construction finishes and returns the code just described.

The mkapp used in Section 13 is this builder. With Q and that builder written out, every piece of the self-referential term there is on the page.

CHAPTER 6

Computing with counters

Put a few beans in one cup and leave a second empty. Write a short list of orders: add one, take one away if there is one, go to this line, stop. A finger on the page keeps your place. Carry out the orders literally. The question in this appendix is how to give the same job to another person with different equipment (a strip of paper, stacks of cards, or a different set of cups) and get the same answer whenever the first person stops.

Agree on how the input is laid out and how to read the output before either starts. One person must not quietly solve the problem while preparing the other's cups. The reductions in this appendix concern functions on natural numbers. To compute a relation $R(\vec{x}, y)$ means to halt with output y exactly when $R(\vec{x}, y)$ holds, and to produce only numerical outputs on halting runs. The machines below are deterministic, so any relation they compute is the graph of a partial function. A translation must preserve both the output and whether the run terminates. Its input and output encodings are part of the translation.

6.1 Finite programs made from two instructions

A counter holds a natural number. INC adds one and advances to the next line. DEC subtracts one and jumps to its named line when the counter is positive; at zero, it leaves the counter alone and advances. A finite program uses finitely many counters. An unconditional jump to j is INC r followed by DEC r j: it restores r's value and always jumps. A zero test is DEC r to a positive branch that first restores r; its fall-through is the zero branch. These operations preserve values, not versions. They are used here for computation, where there are no live facts to preserve.

Try copying a cup with three beans. Put two empty cups beside it and keep a supply of spare beans. Each time you remove a bean from the first cup, put one in each of the other two. When the first is empty, move the third cup's beans back into it. Now the first and second each have three, and the third is empty. Try zero beans, then five. At every pass one more of the original beans has been counted into both other cups; that is why it works for any starting count, not just the three you tried. These are the moves behind the copy $C(x, y, t)$ in the next appendix.

Emptying a counter is a DEC loop back to its own line. Moving a pile decrements its source and increments its destination until the source is empty. Copying uses two such destinations and then restores the source from one of them. Addition, multiplication by a fixed natural number, and bounded loops are finite combinations of these moves. Division by a fixed positive number b needs only b control states: decrement the input, cycle through remainders $0, \ldots, b-1$, and increment a quotient counter whenever the cycle returns to zero. On exit the quotient is in that counter and the remainder is in the control state.

6.2 A tape in finitely many counters

Cut a strip into numbered squares and put a finger on one square. Lift the squares on its left into a stack, nearest on top, and do the same on its right. Moving your finger one place right is now two acts: put the square you were reading on top of the left stack, then take the top of the right stack as the new square. Write a new mark before moving and the stack keeps the changed square. Move left and the two stacks exchange jobs. You can walk the strip either way without losing its order. An exhausted stack supplies a fresh blank square.

Now replace each stack by a count. Use digit 1 for a blank and digit 2 for a mark, with base 3. Push a mark onto an empty stack: its code is 2. Push a blank on top: its code is $3 \cdot 2 + 1 = 7$. Count seven beans in groups of three. The one left over says “blank on top”; the two groups say the remaining stack has code 2. Pop that and the marked square comes back. Multiplication pushes and division pops.

In general, number a finite tape alphabet by $1, \ldots, q$, including its blank, and put $b = q + 1$. Encode a finite stack whose top symbol is d and whose remaining stack has code n by $bn + d$; the empty stack is 0. A push is multiplication by b followed by d increments. A pop is division by b: the remainder is the top symbol and the quotient is the remaining stack. The digits are nonzero, so no nonempty stack has code 0. Popping an empty stack gives remainder 0, which is no digit; the control reads it as the blank and leaves the stack empty.

A tape configuration is a control state, the scanned symbol, and two stacks: the symbols to the left of the head, nearest first, and the symbols to its right, nearest first. The control state and scanned symbol have finitely many possibilities, so they fit in the program’s line number. To write d and move right, push d onto the left stack and pop the right stack for the new scanned symbol. A left move does the reverse. The arithmetic just described implements each transition in a finite terminating stretch of counter instructions. Extra stored blanks do not change the represented tape. A finite number of tapes uses two stacks per tape and finitely many scratch counters. Halting states translate to halting lines. Induction over tape steps proves the simulation; termination of every arithmetic stretch proves its converse.

In the other direction a tape can hold the finitely many counter values as unary strings separated by delimiters. INC appends a mark in the selected string; DEC erases one when present and otherwise detects the empty string. The current instruction is finite control. Each scan and rewrite terminates, so halting and the output are preserved. Multiple tapes may be represented on one tape by a list of tracks with marked head positions: a complete scan reads the marked symbols, a second scan writes their changes and shifts their markers. Extend the represented interval by a blank cell when needed. A fixed-length binary code for the finite alphabet implements each scan with binary symbols and finitely many intermediate control states. So this argument also applies to the single-tape binary halting question.

6.3 Packing finitely many counters into two

Try holding two counts in one pile of eighteen beans. Count how many times you can halve eighteen exactly: once. Count how many times you can divide it exactly by three: twice. Reassemble the eighteen beans after each reading. Those are the two stored counts, 1 and 2. Double the pile to 36 and only the first stored count rises. Divide 36 by three to get 12 and only the second falls. Divide by three again: four beans, storing 2 and 0.

Try taking one more off that second count. Four beans do not divide evenly into threes. The attempted division must restore the four and take the zero branch; keeping only the quotient would corrupt the first count. This is why the procedure below remembers the remainder. Two and three share no factor, so changing the number of one kind of factor leaves the other alone. Assign another prime to each extra count. An all-zero collection is represented by one bean: the empty product is 1.

Let $p_0, \ldots, p_{k-1}$ be distinct primes. At the boundary between simulated instructions, keep

$$A = \prod_{i=0}^{k-1} p_i^{r_i}, \qquad B = 0,$$

where r_i are the simulated counter values. The simulated line is finite control. The representation is unique by prime factorization. There are enough primes for any finite program: a prime divisor of $1 + \prod_i p_i$ is not among the primes already chosen, so the list can always be extended.

To simulate INC on r_i, empty A into B, adding p_i to B per decrement, then move B back to A. The result is $p_i A_{\text{old}}$ and $B = 0$. To simulate DEC, divide A by p_i using the finite remainder cycle described above. On exit $A = 0$, $B = q$ and the control holds d, where $A_{\text{old}} = p_i q + d$ and $0 \le d < p_i$. If $d = 0$, the original number was

divisible by p_i, so $r_i > 0$. Move B to A and take the positive branch. Its exponent of p_i has decreased by one and every other exponent is unchanged. If $d > 0$, then $r_i = 0$. Restore A by moving B back with p_i increments per unit, add d, and take the zero branch. Both branches restore $B = 0$. All loops terminate because their source counter decreases. Induction gives the represented run at every simulated instruction boundary, and no simulated instruction can conceal an infinite internal run.

This is universality with an encoding. With input n in the first simulated counter and the others zero, the packed input is 2^n when $p_0 = 2$. It is supplied to the two-counter machine as its input; the construction does not claim that two counters starting at $(n, 0)$ can first manufacture 2^n. That is precisely the distinction Schroeppel's theorem in Section 15 turns on, and the undecidability used by the one-counter theorem in Appendix E ranges over packed starts too. For a tuple of inputs, supply its product of prime powers. If a numerical output is needed in ordinary notation, the external decoding reads the designated prime's exponent. A host with finitely many additional counters can perform this decoding itself by repeated fixed-prime division.

6.4 Why halting has no decider

Write two responses on a card: "If the predictor says STOP, keep going forever. If it says KEEP GOING, stop." Suppose somebody offers a predictor that always finishes and always gets a program's eventual behaviour right. Give it this very card, with these instructions for using its answer. Work through both possible answers. STOP makes the card keep going; KEEP GOING makes it stop. Waiting longer for the predictor does not create a third correct answer: its promise was to finish with one of the two.

The card can be a program because each response is a finite instruction, and waiting for and branching on the predictor's answer are computable operations. Here is how to include the input explicitly. Finite tape programs can be written as finite binary words, and a program can interpret such a word by looking up the next transition and maintaining the simulated tape. Invalid descriptions may be assigned one fixed halting program. Suppose a terminating algorithm $H(p, x)$ decided whether the program described by p halts on x. Build a program D which, on input p, computes $H(p, p)$, loops forever if the answer is yes, and halts if it is no. Let d describe D. Either answer to $H(d, d)$ contradicts what $D(d)$ does. So no such deciding algorithm exists.

Translate a tape program and its input by the stack construction and then prime packing. Both translations are finite algorithms, and the translated two-counter run halts exactly when the tape run halts. A decider for two-counter halting would

then give H. The same reduction, followed by Theorem 15.4, applies to the small machine. Halting instances can be enumerated by simulating every description and input for successively larger finite budgets. If the nonhalting instances could also be enumerated, interleaving those two enumerations would decide each instance. This proves the impossibility underlying the reduction formulation of Theorem 15.5.

6.5 Seven descriptions of the same computation

Here are the translations behind Theorem E.6. Each one keeps the partial function, including where it runs forever. The constructions are the familiar ones behind the results of Minsky, Shepherdson and Sturgis, and Forster, Larchey-Wendling and their coauthors; the encodings and the reason each translation works are given here.

Binary stacks. A binary stack is a finite word with push and pop operations, with a distinct empty case. Encode the empty word as 1 and pushing $d \in \{0,1\}$ as $n \mapsto 2n + d$. Division by two pops it, with 1 tested before division. Finite counters implement these operations. Conversely two stacks hold a tape as above; encode every symbol of a larger alphabet by a fixed-length binary block. A push or pop of that block takes a fixed finite number of stack operations, and the control stores a partially read block. A tape simulates counters, so the translations connect binary stack machines and counter machines in both directions.

The two decrement conventions. Some counter machines jump on zero and fall through after a positive decrement; the convention of this book does the reverse. To translate a conditional decrement, send its two outcomes to separate short blocks that unconditionally jump to the intended destinations. In this book's convention an unconditional jump is the restoring INC/DEC pair above. Under jump-on-zero that pair falls through, because the DEC finds the counter positive; there the jump is a DEC on one extra counter that nothing ever increments, so it is always zero. So either convention implements the other's instruction, with the same final values on the original counters and one finite stretch per step, and the translation into jump-on-zero spends one counter more. Relabel each target to the beginning of its translated block. This proves the equivalence of the two Minsky instruction conventions without identifying their individual steps.

FRACTRAN. A program is an ordered finite list of positive rational numbers. At a positive integer N, multiply by the first fraction whose product with N is an integer; halt if none applies. Counter machines simulate this by fixed-denominator division, multiplication and an ordered scan of the finite list. For the reverse translation, assign distinct primes p_r to counters and q_i to control states. Represent a counter configuration by $q_i \prod_r p_r^{n_r}$. Every translated instruction uses a fresh intermediate state prime, so its state marker never cancels out of a fraction. For INC r going to j, use $t_i p_r / q_i$, followed by the transfer q_j / t_i. Try it with a counter represented

by factors of 2, a starting line marked by 5, a destination marked by 7 and an intermediate marker 11. Ten beans, $5 \cdot 2$, mean line 5 with counter value 1. The fraction $22/5$ takes ten to 44; $7/11$ takes 44 to 28. Divide the result by its line marker 7: four remains, two factors of 2. The line has changed and the counter has risen by one. The intermediate 11 keeps the two parts of the instruction separate. For DEC r with positive target j and zero target k, use, in this priority order,

$$\frac{t_i}{q_i p_r}, \qquad \frac{u_i}{q_i},$$

with transfers q_j/t_i and q_k/u_i. All state and intermediate primes are distinct from the counter primes. Exactly one state marker is present. Fractions belonging to other states cannot apply. At state i, the first decrement fraction applies exactly when $n_r > 0$; otherwise the second applies. The next fraction transfers the intermediate marker to the destination marker. A halting state has no outgoing fraction. Each counter step then takes two fraction steps, and the prime exponents preserve all counter values. Read the output as the exponent of the output counter's prime.

Partial recursive functions. Compare two orders. "Try the first ten tickets" gives you a stopping point before you begin. "Try tickets in order until one passes" does not. If each test finishes, the second order can still keep you working forever because every ticket fails. If one test itself never finishes, you never reach the next ticket. Those are the differences between a bounded loop and the search used here. Start with zero, successor and projections; close under composition, primitive recursion and minimization. In minimization, search $0, 1, 2, \ldots$ for the first argument where the tested function returns zero; every earlier test must terminate with a positive result. A counter program implements the initial functions directly. Disjoint scratch counters preserve arguments across composition. For primitive recursion it keeps the recursion index and accumulated result, increments the index, and runs the step program until it reaches the supplied bound. For minimization it runs the tests successively; an undefined test leaves the run there forever, as the definition requires. Induction on these constructions supplies a finite counter program for every partial recursive function.

Conversely encode a counter configuration by nested pairs $\langle m, n \rangle = (2n + 1)2^m$, with a separate finite code for the line. Addition, multiplication, powers, bounded division and bounded search are primitive recursive: their loops have a supplied natural bound and can be written as recursion on that bound. Pair decoding is bounded by the encoded positive number. The next configuration and the halt test are then primitive recursive for any fixed finite program. The configuration after t steps is primitive recursion on t, keeping halted configurations fixed. Minimize the total halt test over t, then read the output counter. This is the program's partial function, and it is undefined exactly when the program never halts.

The language L. Fold two instruction cards shut. One says "put three beans in the output cup and stop"; the other says "keep adding beans forever." A zero input is to choose the first card. If your rule makes you carry out both cards before choosing, you never deliver the three beans. You have to keep the instructions folded until the choice is made. An abstraction does that job in weak call-by-value: its body waits until the abstraction is applied. The Scott numeral definitions and the delayed recursion construction in Appendix J give zero, successor, predecessor, case distinction and guarded recursion as terms. A case on a numeral n can be delayed explicitly as

$$n\,(\lambda d.u)\,(\lambda p.\lambda d.(v\,p))\,(\lambda a.a)\,:$$

at zero it evaluates u, and at a successor it evaluates v on the predecessor. Both branches are values until selected, so call-by-value does not evaluate the unused branch. Composition is application; primitive recursion repeats a step on the predecessor; minimization is a guarded search that evaluates each test before moving to its successor. Build the term in the same order as the function: begin with its initial functions, then use the corresponding term operation each time composition, recursion or minimization is used. Each operation consumes the results of the already built pieces exactly as the function definition requires. On numerical inputs, a computation that finishes therefore returns the Scott numeral of its answer. If a required subcomputation never finishes, evaluation waits for it; if minimization never finds a successful test, its search keeps going. In either case there is no resulting value.

In the other direction, encode a finite term as a finite string or as the nested-pair code of its syntax tree. Reading the outer constructor, finding a variable index, renaming bound indices and substituting a finite term are terminating traversals of finite trees. A tape algorithm performs them with an explicit stack of unvisited children. For weak call-by-value evaluation it evaluates an application's function and argument, then substitutes the argument value in the abstraction's body; it never reduces inside an unapplied abstraction. Each operation is a finite tape computation. Induction on a terminating evaluation gives a terminating tape run with its value; reading a terminating run backwards gives the evaluation tree. Scott numerals are decoded by counting successor constructors. So an L computation of a numerical relation is a tape computation of it.

The tape and counter translations, the two counter conventions, binary stacks, FRACTRAN, partial recursive functions and L now lie in one connected family of translations. Composing the translations preserves their input/output relation. None of them keeps time, space, or a ledger of what was paid just because it keeps a partial function. Where I need a ledger kept, I build that in on purpose, as the universal program of Appendix L does; time and space I don't claim at all.

6.6 Searching for a finite computation

Give yourself one token and run one instruction of the program you want to test. If it has not stopped, reset its starting arrangement, give yourself two tokens and try again. Then three. A program that stops after seven instructions is found on the seven-token attempt. Seeing six attempts fail says only that it has not stopped within six instructions; it does not tell you it runs forever.

The evaluators used in Appendix D.4 work by that search. A program and a natural budget determine a finite simulation. Enumerate budgets $0, 1, 2, \ldots$, and stop at the first one that exhibits a halting run, returning its output. If no such run exists, the search continues forever. Encoding a finite program, decoding an instruction, and performing one step use the finite operations above, so this search is itself a partial recursive function and has a finite counter program. The host can execute that program instruction for instruction, using as many registers as the construction requires. Its computation uses only INC, DEC and HALT, so it adds nothing to the ledger and leaves the fact table, channel and flag alone. This supplies the evaluators and program transformations used in the host's s-m-n and fixed-point arguments.

CHAPTER 7

The universal program in blocks

Put the guest's instructions on one sheet and the host's on another, with a finger marking the current line on each. The guest takes one instruction. The host takes as many of its smaller instructions as it needs, then returns to the head of its sheet. Compare the two arrangements there: counts, live facts, commitment, flag and ledger. The host is allowed to do more work between comparisons. It is not allowed to lose a fact, revive a dead one, or charge another unit for its bookkeeping. The lists below let you carry out that comparison instruction by instruction.

Here is the finite list of instructions that is U. A block is a list, not an extra instruction: replace every block name by its list, place the lists consecutively, and resolve each label to its instruction number. The first instruction is at address 1. Within a block, o is its first address and $o + k$ its offset k. Unless a branch says otherwise, execution continues into the next block. The only final instructions are INC, DEC, HALT, CHECK, COMMIT and CERTIFY, with the meanings and prices of Definition 15.3 on the host's registers (Definition D.2).

7.1 Arithmetic blocks

Write $J(a, p)$ for the two-instruction list INC a; DEC a p. It jumps to p and restores the value of a. The following lists specify the other arithmetic blocks. Semicolons concatenate lists; a named block in a list is placed at the next available address. A target such as $o + 3$ refers to the enclosing block's start, not the nested block's start.

Block	Expansion at address o
$Z(x)$	DEC x o
$J_0(x, p)$	DEC x $(o + 3)$; $J(x, p)$; INC x
$M(x, y)$	$J(x, o + 3)$; INC y; DEC x $(o + 2)$
$M_2(x, y, z)$	$J(x, o + 4)$; INC y; INC z; DEC x $(o + 2)$
$C(x, y, t)$	$Z(y)$; $Z(t)$; $M_2(x, y, t)$; $M(t, x)$
$H(a, x, p)$	$Z(a)$; $M(x, a)$; $J(a, o + 8)$; INC x; DEC a $(o + 11)$; $J(a, p)$; DEC a $(o + 7)$

Block	Expansion at address o
$P(a,x,y)$	$Z(a)$; $M_2(x,a,a)$; INC a; $M(a,x)$; $J(a,o+22)$; $M_2(x,a,a)$; $M(a,x)$; DEC y $(o+13)$
$N(a,x,y)$	$J_0(x,0)$; $Z(y)$; $J(a,o+8)$; INC y; $H(a,x,o+7)$

Their lengths, in the order $Z, J_0, M, M_2, C, H, P, N$, are $1,4,4,5,11,12,23,20$. The source and destination of a move are distinct; M_2 may have equal destinations, as in P. In C the three registers are distinct; in H, $a \neq x$; in P and N, all three are distinct.

Each claim about these blocks follows from a decreasing counter. Z decrements to zero. M adds the old source to the destination and empties the source; M_2 does it to both destinations. So C leaves x unchanged, writes its old value to y, and leaves $t = 0$. J_0 tests zero without changing the value: a positive counter is decremented and then restored, while zero takes the jump.

In H, first move the old x into a. Each complete pass removes two from a and adds one to x. So it leaves $a = 0$ and $x = \lfloor x_{\text{old}}/2 \rfloor$; an even old value jumps to p, an odd one falls through. In P, the first four blocks make $x = 2x_{\text{old}} + 1$ and $a = 0$. Each subsequent pass doubles x while decrementing y, so it ends at $x = (2x_{\text{old}} + 1)2^{y_{\text{old}}}$, with $a = y = 0$. In N, zero jumps to address 0, which holds no instruction, so the run stops. A positive number is repeatedly halved until odd, counting the even divisions in y. So N sends $x = \langle m, n \rangle$ to $x = n$, $y = m$, $a = 0$. These invariants prove both the returned values and termination on the stated inputs.

7.2 Selection, comparison and fetching

The selector $D(x, t; p_0, \ldots, p_{k-1})$ has length $3k$. For $i = 0, \ldots, k-1$, place at offset $3i$ the three instructions DEC x $(o + 3i + 3)$; $J(t, p_i)$. If the entering value is $i < k$, it takes target p_i with $x = 0$. If it is at least k, it falls through with x reduced by k. This follows by counting the decrements before the first zero.

The equality block $E(x, y; t_1, t_2, u; p)$ first makes copies $C(x, t_1, u)$ and $C(y, t_2, u)$, occupying offsets 0 to 21. Its remaining list is

$$
\begin{aligned}
22: &\quad \text{DEC } t_1 \ (o+29),\\
23: &\quad J_0(t_2, p),\\
27: &\quad J(u, o+32),\\
29: &\quad \text{DEC } t_2 \ (o+22),\\
30: &\quad J(u, o+32),\\
32: &\quad Z(t_1);\ Z(t_2).
\end{aligned}
$$

The listed offsets count expanded instructions, so the block has length 34. The temporary registers are distinct from each other and from x, y. The two copies count down together. If they reach zero together, the block jumps to p; if one reaches zero first, it clears both and falls through. So it preserves x, y, leaves its temporary counters zero at either exit, and branches exactly on equality.

For fetching, define $F(q, l; w, k, h, a, t; p_0, p_e)$ by the list

$$\begin{aligned} 0:&\ C(q, w, t),\\ 11:&\ C(l, k, t),\\ 22:&\ J_0(k, p_0),\\ 26:&\ \text{DEC } k\ (o + 27),\\ 27:&\ J_0(w, p_e),\\ 31:&\ N(a, w, h),\\ 51:&\ J_0(k, o + 56),\\ 55:&\ \text{DEC } k\ (o + 27). \end{aligned}$$

Its length is 56. Register q holds a list encoded by 0 for empty and $\langle \text{head}, \text{tail} \rangle$ for a nonempty list, and l is a line number. The scratch registers are distinct and separate from q, l. A zero line takes p_0. Otherwise decrementing the line copy once makes $k = l - 1$. Each pass removes the next list head into h and its tail into w. If $k = 0$, that head is the selected instruction and the block exits; otherwise it decrements k and repeats. An exhausted list takes p_e. The original program and line values are restored by the copies. Every call terminates: each complete pass decreases k, or exits.

7.3 Registers and instruction codes

Put $T_i = i + 4$ for $0 \le i < 10$. For bank $c \in \{0, 1\}$ and slot $0 \le k < 16$, put

$$R_c = c, \quad Q_c = 14 + c, \quad V_{c,k} = 16 + 16c + k, \quad S_{c,k} = 48 + 16c + k.$$

Register 2 is the program, register 3 its line, and register 80 is the dead end. V are the mirrors, S the slots, and Q the counts of successful checks in each bank. Property codes are 0 for zero, 1 for even, and $n + 2$ for at least n. The code of an instruction is the pair of its opcode and operand: INC has opcode 0 and operand c; DEC has opcode 1 and operand $\langle c, j \rangle$; HALT has opcode 2 and operand 0; CHECK has opcode 3 and operand $\langle c, p \rangle$; COMMIT has opcode 4 and that same operand form; CERTIFY has opcode 5 and operand 0. A program is their list code. These rules specify the loaded number in register 2.

A bump of slot S and mirror V is INC S; DEC S to the following instruction; $Z(V)$. It has length 3. It leaves the slot's value alone, increases its version by two, and

empties the mirror. Let B_c concatenate the sixteen bumps of bank c, in increasing slot order; its length is 48.

7.4 The fifteen blocks of U

The following table fixes the order, start address and expanded length of every major block. Labels in the definitions below mean these addresses; H without arguments denotes the head at 1 and K the halt block at 98.

Block	Start	Length
Head	1	97
Halt	98	11
INC bank selection	109	8
INC bank 0	117	52
INC bank 1	169	52
DEC bank selection	221	28
DEC bank 0	249	60
DEC bank 1	309	60
CHECK bank selection	369	28
CHECK bank 0	397	1022
CHECK bank 1	1419	1022
COMMIT bank selection	2441	28
COMMIT bank 0	2469	628
COMMIT bank 1	3097	628
CERTIFY	3725	4

The head is $F(2,3;T_0,T_1,T_2,T_3,T_4;K,K)$; $Z(T_0)$; $N(T_3,T_2,T_5)$; $D(T_5,T_4;109,221,98,369,2441,3725)$; $J(T_4,K)$. Fetching supplies an instruction code in T_2. Unpacking leaves its operand in T_2 and its opcode in T_5. The six-way selector chooses its handler. The final jump handles an invalid opcode, though a loaded program has none. The halt block is $Z(T_0);\cdots;Z(T_9)$ followed by HALT.

INC's bank selector is $D(T_2,T_4;117,169)$; $J(T_4,K)$. Each other bank selector first runs $N(T_3,T_2,T_6)$, leaving the property code or jump target in T_2 and the bank in T_6, then selects its two bank blocks with $D(T_6,T_4;\text{bank }0,\text{bank }1)$; $J(T_4,K)$.

The INC block for bank c is INC R_c; B_c; INC 3; $J(T_4,H)$. The DEC block, starting at o, is DEC R_c $(o+5)$; INC 3; $Z(T_2)$; $J(T_4,H)$; B_c; $Z(3)$; $M(T_2,3)$; $J(T_4,H)$. A zero counter follows the first return, advancing the guest line without bumping slots. A positive counter jumps over it, bumps the bank and installs the target line.

A CHECK slot block for (c, k) is the following 60 instructions after expansion:

$$C(R_c, T_8, T_4);\ C(T_2, T_9, T_4);\ P(T_3, T_8, T_9);\ M(T_8, S_{c,k});$$
$$\text{CHECK Slot } S_{c,k};\ Z(V_{c,k});\ M(T_2, V_{c,k});$$
$$\text{INC } V_{c,k};\ \text{INC } Q_c;\ \text{INC 3};\ J(T_4, H).$$

The CHECK bank block at o is $C(Q_c, T_7, T_4)$, then a selector on T_7 whose sixteen targets are $o + 62 + 60k$, then CHECK Slot 80 and $J(T_4, K)$, then the sixteen slot blocks. Its length is $11 + 48 + 1 + 2 + 16 \cdot 60 = 1022$. The selected unused slot starts at zero, so the move into it installs exactly the pair of property code and guest value. The host's own table has capacity sixteen: if it is full, its CHECK traps even when this bank's count is below sixteen. So the capacity is shared between the banks, just as it is in the guest. A count of sixteen in one bank already fills the table, so the dead-end check and the full-table trap are the same case.

A COMMIT slot block is COMMIT Slot $S_{c,k}$; $Z(T_2)$; INC 3; $J(T_4, H)$, of length 5. The COMMIT bank block at o first increments T_2, making the wanted mirror value one more than the property code. It then runs, for $k = 0, \ldots, 15$, $E(V_{c,k}, T_2; T_7, T_8, T_4; o + 548 + 5k)$. After those comparisons come COMMIT Slot 80 and $J(T_4, K)$, followed by the sixteen slot blocks. Its length is $1 + 16 \cdot 34 + 1 + 2 + 16 \cdot 5 = 628$. The comparison reads the mirror, leaving the slot untouched. Its first match commits that slot's current fact; absence of a match reaches the dead end and traps. The CERTIFY block is CERTIFY; INC 3; $J(T_4, H)$, of length 4.

These expansions determine every instruction and target. Adding the fifteen lengths gives 3728. There are sixteen ordinary CHECK sites and one dead-end CHECK in each bank, sixteen ordinary COMMIT sites and one dead-end COMMIT in each bank, and one CERTIFY site: $2 \cdot 17 + 2 \cdot 17 + 1 = 69$ paid sites. Every other instruction costs nothing.

7.5 Why the blocks simulate a guest

Run a case that catches more than a wrong sum. Start the guest with four beans in A and run CHECK Even A. Put its fact on a card saying "even, version 0." The host copies the value into an unused slot as the pair (Even, 4), whose code is $(2 \cdot 4 + 1)2^1 = 18$, and checks that slot. Put one unit on each ledger.

Now let the guest add one bean and take it away. Its pile is four again, but its version is 2. The old fact still says version 0. On the host, each of those guest writes bumps the slot by one and back: 18 to 19 to 18. The slot's value ends where it began, while its version rises and its mirror is cleared. Try COMMIT Even A using the old check. Both machines must refuse, even though four is even. Both pay one more unit for that failed attempt. A translation that merely compares the piles at the end would miss the failure this experiment is designed to catch.

Start again and leave out the two bean moves. CHECK, COMMIT, CERTIFY must succeed on both sheets, raising both flags for exactly three units. The longer list of host instructions adds nothing to the ledger for copying or looking up a line. These two runs test the reason for keeping both values and versions. The argument for all runs has to keep the following correspondence at every return to the head.

At the head, registers 0, 1 and 3 give the guest counters and line. Each successful guest CHECK has one host slot fact holding its property and checked value. Its mirror is the property code plus one while the guest fact is current, and zero after the guest counter has been written. The matching host slot fact is current in exactly the first case. Count registers say how many slots have been used in their bank; unused slots and mirrors are zero. Tables have corresponding entries in the same order, and channels name corresponding entries. The ledgers and flags agree. Loading a guest establishes every part of this relation.

Fetching and dispatching terminate and preserve that relation's stored values. For INC or a positive DEC, the guest writes its counter; the selected bank's bumps invalidate exactly its former slot facts and clear their mirrors. A zero DEC writes nothing and bumps nothing. These cases add nothing to either ledger. For CHECK, an unused slot receives the exact pair tested by the guest. Both checkers answer the same, and the tables have the same length. A passing check appends the corresponding fact and updates its mirror and count. A false check or a full table traps both machines, after the same single paid act. For COMMIT, a nonzero mirror matches exactly a current guest fact with the requested property and counter. Its host slot has not been read or changed, so the corresponding COMMIT succeeds. If none matches, both machines trap. Again exactly one unit is paid. For CERTIFY, channels are either both empty or both hold corresponding commitments, so the instruction either traps both or raises both flags, for one unit.

All arithmetic between these paid acts terminates by the block invariants. A normal case returns to the head; a halting or trapped case stops. No arithmetic block has a paid instruction. This proves the one-step correspondence and bounds every intermediate host ledger between the guest's old and new ledgers. Induction proves the run correspondence of Theorems D.2 and D.4. It also proves the reverse halting implication: an infinite guest run entails infinitely many nonempty host stretches, and a running guest instruction cannot produce a stopped host except in its own halting or trapping case.

7.6 Adding PAY

For U_P, use opcode 6 with operand 0 for PAY, append a PAY handler to the six handlers, and append its block after CERTIFY. The block is PAY; INC 3; $J(T_4, H)$, of length 4. The head's selector now has seven entries and is three instructions

longer. Every block after the head then starts three addresses later. Resolve targets to those new addresses; the head itself remains at address 1. The new PAY block starts at 3732. The total length is $3728 + 3 + 4 = 3735$ and there are $69 + 1 = 70$ paid sites. The interpretation of Slot uses the priced host's property language from Appendix D.2; the arithmetic blocks and their invariant do not depend on the particular meaning of a property code. A PAY guest step is one PAY host step surrounded by arithmetic that costs nothing, so the same ledger argument applies.

CHAPTER 8

The CHSH derivation, in full

Appendix H tells the story; this appendix does the algebra. Each item gives the statement with its hypotheses and the steps you need to check it. Nothing here says that a matrix is a physical state. Where quantum strategies come in, I say exactly how they stand to the matrices.

8.1 Setting and notation

Start with four piles of tally sheets, one for each pair of settings Alice and Bob can be given. On each round their answers either agree or disagree. Give agreement a score of $+1$ and disagreement a score of -1. In each pile, add the scores and divide by the number of rounds. That pile's average is its *correlator*. It says how strongly the two answers agree for that pair of settings.

For example, three agreements and one disagreement give a total of two over four rounds, hence an average of one half. The four averages make a little square table: Alice's setting chooses the row, Bob's chooses the column. The score I will study adds three cells of that table and subtracts the bottom-right one. Here are the names for those quantities.

Definition 8.1 (Settings, outcomes, correlators)**.** Alice receives a setting $x \in \{0,1\}$ and Bob a setting $y \in \{0,1\}$. Each answers a bit, recoded as an outcome $a, b \in \{+1,-1\}$ (bit 0 becomes $+1$, bit 1 becomes -1). The correlator E_{xy} is the average of the product ab over the rounds with settings (x,y). The CHSH quantity is

$$S = E_{00} + E_{01} + E_{10} - E_{11}.$$

I collect the four correlators into the 2×2 matrix

$$E = \begin{pmatrix} E_{00} & E_{01} \\ E_{10} & E_{11} \end{pmatrix},$$

whose rows are indexed by Alice's setting and whose columns by Bob's. The rows of E are (E_{00}, E_{01}) and (E_{10}, E_{11}); the columns are (E_{00}, E_{10}) and (E_{01}, E_{11}).

Definition 8.2 (Counters)**.** There are eight counts: for each setting pair, same_{xy} counts rounds whose two answer bits matched and diff_{xy} counts rounds whose bits differed. A round adds one to exactly one of the eight. Write

$$N_{xy} = \text{same}_{xy} + \text{diff}_{xy}, \qquad D_{xy} = \text{same}_{xy} - \text{diff}_{xy}, \qquad E_{xy} = \frac{D_{xy}}{N_{xy}}.$$

A matched round contributes $ab = +1$ and a differing round $ab = -1$, so D_{xy} is the sum of the products and D_{xy}/N_{xy} is their average. Since $|D_{xy}| \le N_{xy}$, every correlator from counters lies in $[-1, 1]$. When a bucket is empty, its correlator is defined as 0. The score is $E_{00} + E_{01} + E_{10} - E_{11}$.

8.2 The classical bound

Write a fixed plan on a card: Alice's answer to setting zero, her answer to setting one, and Bob's two answers. Each answer is a sign, plus or minus. Before multiplying anything, look at Bob's pair. Either the signs agree or they differ. If they agree, adding them gives plus or minus two and subtracting them gives zero. If they differ, the sum is zero and the difference is plus or minus two. One of those two quantities vanishes; the other cannot exceed two in size. The next calculation uses exactly that split.

Lemma 8.1 (Deterministic strategies)**.** Let $a_0, a_1, b_0, b_1 \in \{+1, -1\}$. Then $a_0b_0 + a_0b_1 + a_1b_0 - a_1b_1$ is 2 or -2. □

Proof. Factor: $a_0b_0 + a_0b_1 + a_1b_0 - a_1b_1 = a_0(b_0 + b_1) + a_1(b_0 - b_1)$. If $b_0 = b_1$ then $b_0 + b_1 = \pm 2$ and $b_0 - b_1 = 0$; if $b_0 \neq b_1$ the other way round. In both cases exactly one of the two brackets is ± 2 and the other is 0, so the expression is ± 2 times a sign, which is $+2$ or -2. There are sixteen sign choices, and every one of them gives exactly ± 2. None gives 0 or ± 4. ■

You can also just list the sixteen sign choices and add each one up. Nothing in the argument needs real numbers: the signs are whole numbers, so the bound holds the same way whether you read the correlators as rationals (what a counter gives) or as reals.

A fixed card is only the first case. You can shuffle among several cards, perhaps using some more often than others. Every card still tells each player an answer using that player's setting alone. The shuffle changes how often each card contributes; it does not change the score that a particular card would produce. Since each card's score lies between minus two and two, their average must stay between those same ends. The following definition writes out that weighted shuffle.

Definition 8.3 (Finite factorizable model)**.** Four real correlators $E_{00}, E_{01}, E_{10}, E_{11}$ are factorizable when there are a natural number $\lambda_{\max}$, weights $p(\lambda)$ and response signs $A(x,\lambda)$, $B(y,\lambda)$ for $\lambda = 0, \ldots, \lambda_{\max}$ such that: $0 \le p(\lambda) \le 1$; $\sum_{\lambda=0}^{\lambda_{\max}} p(\lambda) = 1$; every $A(x,\lambda)$ and every $B(y,\lambda)$ is -1 or $+1$; and for all x, y, $E_{xy} = \sum_{\lambda=0}^{\lambda_{\max}} p(\lambda) A(x,\lambda) B(y,\lambda)$. In words: a hidden card λ is drawn with probability $p(\lambda)$, and each player reads an answer off the card using only their own setting. Only the correlators are modelled.

□ **Theorem 8.2** (The classical bound for a finite local model)**.** If E is factorizable, then $-2 \le E_{00} + E_{01} + E_{10} - E_{11} \le 2$.

Proof. Substitute the factorization: $S = \sum_\lambda p(\lambda) S_\lambda$, where S_λ is the deterministic expression of Lemma 8.1 at the signs $A(\cdot,\lambda), B(\cdot,\lambda)$. Each $S_\lambda \in [-2, 2]$, the weights ■ are nonnegative and sum to 1, so $-2 = \sum_\lambda p(\lambda)(-2) \le S \le \sum_\lambda p(\lambda) \cdot 2 = 2$.

Remark 8.1 (Scope of the classical bound)**.** Fine's 1982 theorem is an equivalence between several descriptions of local models and the Bell inequalities. The preceding theorem proves one direction: a finite factorizable model obeys the bound. The converse of this single inequality is false: $(1, 1, -1, 1)$ has $S = 0$, but it breaks another of the sign variants ($E_{00} + E_{01} - E_{10} + E_{11} = 4$), so no local model gives it. The correlation-form equivalence needs each correlator between -1 and 1 and all eight inequalities; with those, Section I.10 builds the local model. The preceding theorem is stated for finitely many hidden cards ($\lambda \le \lambda_{\max}$), and that loses nothing. Whatever the hidden variable is and however it is drawn, the correlators see it only through its four answer signs, so lumping together the cards that carry the same four signs gives at most sixteen cards and the same four correlators. All the lumping needs is a probability for each of the sixteen sign patterns.

The finite-tally statement concerns one fixed strategy replayed on every recorded round.

Definition 8.4 (A tally consistent with one local plan)**.** For bits $a_0, a_1, b_0, b_1 \in \{0, 1\}$ and a counter record w, w is locally consistent with (a_0, a_1, b_0, b_1) when, for each setting pair (x, y): if $a_x = b_y$ then $\mathrm{diff}_{xy} = 0$, otherwise $\mathrm{same}_{xy} = 0$; and every bucket is sampled, $N_{00}, N_{01}, N_{10}, N_{11} > 0$.

Theorem 8.3 (A tally above 2 is not one replayed strategy)**.** Let w be a counter record whose rational score $S(w)$ is greater than 2. Then for all bits a_0, a_1, b_0, b_1, w is not locally consistent with (a_0, a_1, b_0, b_1). Every locally consistent record has $S(w) = 2$ or $S(w) = -2$; the first statement is that fact turned around. □

Proof. Under local consistency each sampled bucket is pure: if $a_x = b_y$ every round in the bucket matched, so $E_{xy} = N_{xy}/N_{xy} = +1$; otherwise every round differed and $E_{xy} = -1$. In signs, $E_{xy} = \alpha_x \beta_y$ with $\alpha_x = (-1)^{a_x}$ and $\beta_y = (-1)^{b_y}$. The score is then the deterministic expression of Lemma 8.1, which is 2 or -2. A score above 2 contradicts this. ■

The theorem rules out one explanation, a single four-bit plan used on every round. It says nothing about a local model that redraws its hidden state every round; Appendix H gives a four-round coin-flip tally with score 4. The clause that every bucket is sampled does work here. An empty bucket reads as 0, and the plan that answers 0 to everything, never asked the pair 11, would score $1 + 1 + 1 - 0 = 3$.

8.3 Tsirelson's bound from algebra

Now put the cards aside and look just at four numbers in the square. I want to bound their three-plus-one-minus score from their squared sizes. The useful trick is to write the proposed allowance minus the squared score as a pile of squares. A square cannot be negative. If the difference really is that pile, the score cannot outrun the allowance.

The identity below is the receipt for that trick. The letters a, b, c, d are the four entries read across the first row, then across the second. It holds for any four real numbers. After proving it, I will put in the two bounds on row length that this part of the checker supplies.

Lemma 8.4 (The six-square identity)**.** For all real a, b, c, d, □

$$4(a^2 + b^2 + c^2 + d^2) - (a + b + c - d)^2 = (a - b)^2 + (a - c)^2 + (a + d)^2 + (b - c)^2 + (b + d)^2 + (c + d)^2.$$

Proof. Expand the square on the left: $(a + b + c - d)^2 = a^2 + b^2 + c^2 + d^2 + 2ab + 2ac - 2ad + 2bc - 2bd - 2cd$. So the left side is

$$3(a^2 + b^2 + c^2 + d^2) - 2ab - 2ac + 2ad - 2bc + 2bd + 2cd.$$

On the right, each variable appears in exactly three of the six brackets (a in $a-b$, $a-c$, $a+d$; b in $a-b$, $b-c$, $b+d$; c in $a-c$, $b-c$, $c+d$; d in $a+d$, $b+d$, $c+d$), so the squared terms add to $3(a^2+b^2+c^2+d^2)$. The cross terms are $-2ab$ from $(a-b)^2$, $-2ac$ from $(a-c)^2$, $+2ad$ from $(a+d)^2$, $-2bc$ from $(b-c)^2$, $+2bd$ from $(b+d)^2$ and $+2cd$ from $(c+d)^2$. Both sides agree term by term.

Since the right side is a sum of squares, $S^2 \le 4\sum_{xy} E_{xy}^2$ for any four reals. ■

□ **Theorem 8.5** (Row-bound form of Tsirelson's bound)**.** For all real $E_{00}, E_{01}, E_{10}, E_{11}$ with $E_{00}^2 + E_{01}^2 \le 1$ and $E_{10}^2 + E_{11}^2 \le 1$, $S^2 \le 8$. The absolute form $|S| \le \sqrt{8} = 2\sqrt{2}$ follows.

Proof. Adding the two row bounds gives $\sum_{xy} E_{xy}^2 \le 2$. By Lemma I.3, $S^2 \le 4\sum_{xy} E_{xy}^2 \le 8$. ■

□ **Theorem 8.6** (Tightness)**.** The point $E_{00} = E_{01} = E_{10} = 1/\sqrt{2}$, $E_{11} = -1/\sqrt{2}$ satisfies both row bounds and has $S = \sqrt{8}$.

Proof. Each row has squared length $1/2 + 1/2 = 1$, and $S = 4/\sqrt{2} = 2\sqrt{2} = \sqrt{8}$. ■

Remark 8.2 (Where equality can happen)**.** Equality $S^2 = 8$ under the row bounds forces equality in both steps of the proof above: the six squares must all vanish, which gives $a = b = c = -d$, and the rows must be tight, which gives $2a^2 = 1$. So the point of Theorem I.5 and its negative are the only places the bound is reached. A tally's correlators are ratios of whole numbers and $2a^2 = 1$ has no such solution, so no tally reaches the bound; the integer check's passing tallies stay strictly below it and come as near as you like. (Take rational a, b with $a^2 + b^2 = 1$, from a Pythagorean triple, and the correlators $(a, b, b, -a)$: they pass the check of Section I.5 with no slack, and $S = 2(a+b)$, which gets as close to $2\sqrt{2}$ as you like as a/b approaches 1.)

□ **Lemma 8.7** (A rational constant above the bound)**.** $\sqrt{8} < 5657/2000$.

Proof. Both numbers are positive, so it suffices to compare squares. $5657^2 = 32{,}001{,}649$ and $8 \cdot 2000^2 = 32{,}000{,}000$, so $(5657/2000)^2 =$

$32{,}001{,}649/4{,}000{,}000 > 8 = (\sqrt{8})^2$. If $\sqrt{8} \geq 5657/2000$ held, squaring both nonnegative sides would give $8 \geq (5657/2000)^2$, a contradiction. ∎

The constant is the four-decimal round-up of $2\sqrt{2} = 2.8284271\ldots$; the gap is about 7.3×10^{-5}. It lets rational code test "above the Tsirelson bound" without a square root. It plays no part in the integer check of Section I.5.

8.4 The zero-marginal 5×5 moment matrix

The four averages fill only the Alice–Bob part of a larger table. Give that table five labels: a constant reading 1, Alice's two possible readings, and Bob's two. Each cell records an average product of its row and column readings. A reading multiplied by itself is 1, so the diagonal is all ones. The four averages measured fill the cross-player cells and their reflected partners.

Other cells are still empty. This particular check fills them with zero: each player's average answer against the constant reading, and the cells between that player's two settings. These are choices of the check; the four cross-player averages alone did not supply them. I will later let the two within-player cells vary and ask whether any filling works. For now, write down this one fixed filling and test it.

Definition 8.5 (The pinned level-1 matrix)**.** Take rows and columns I, A_0, A_1, B_0, B_1. Put 1 on the diagonal and the four correlators in the Alice–Bob cells, and set the four one-party entries (each player's own average answer) and the two within-party entries (A_0 against A_1, B_0 against B_1) to zero. The resulting matrix is:

$$M(E) = \begin{array}{c} \\ I \\ A_0 \\ A_1 \\ B_0 \\ B_1 \end{array} \begin{array}{c} \begin{array}{ccccc} I & A_0 & A_1 & B_0 & B_1 \end{array} \\ \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & E_{00} & E_{01} \\ 0 & 0 & 1 & E_{10} & E_{11} \\ 0 & E_{00} & E_{10} & 1 & 0 \\ 0 & E_{01} & E_{11} & 0 & 1 \end{pmatrix} \end{array}$$

What do I ask of this table? Choose five numbers, one beside each row and one beside the matching column. For each cell, multiply its entry by its row's number and its column's number, then add all twenty-five results. Do this with any five numbers you like, positive, negative or zero. I require the total never to be negative. That requirement is called *positive semidefiniteness*, abbreviated PSD.

Write the five chosen numbers as a column v. The superscript T turns a column into a row, so ordinary matrix multiplication writes that same weighted total as

$v^{\mathrm{T}}Mv$. The condition is therefore $v^{\mathrm{T}}Mv \geq 0$ for every choice of v. The table is symmetric too: reflect it across its diagonal and the entries agree.

The first row and column have only one nonzero entry, the corner 1. Their contribution is the square of the first chosen number. The remaining four rows and columns can be tested on their own. Grouping them gives

$$M(E) = \begin{pmatrix} 1 & 0 \\ 0 & G \end{pmatrix}, \qquad G = \begin{pmatrix} I_2 & E \\ E^{\mathrm{T}} & I_2 \end{pmatrix}.$$

Here I_2 is the two-by-two identity table: ones on its diagonal and zeros elsewhere. The superscript on E again swaps rows and columns. The weighted total for $M(E)$ is the first number's square plus the weighted total for G. If every total for G is nonnegative, adding a square keeps it nonnegative. If some total for G is negative, choose the first number to be zero and the same negative total appears for $M(E)$. Thus the two tables pass or fail together.

The next operation is completing a square, with several numbers at once. For one number, $u^2 + 2uw + rw^2$ becomes $(u+w)^2 + (r-1)w^2$. The first square cannot hurt, and choosing $u = -w$ makes it vanish. So the whole expression stays nonnegative for every choice exactly when the remaining part does. The matrix formula below does the same job for two groups of numbers.

Here *positive definite* means that the weighted total for P is strictly positive for every nonzero choice of weights. In particular, P has an inverse, written P^{-1}: applying it undoes multiplication by P. The matrix left after completing the square is called the *Schur complement*. Its name is less useful here than the operation that produces it.

□ **Lemma 8.8** (Schur complement, the standard form)**.** Let P be symmetric positive definite. The symmetric block matrix $\begin{pmatrix} P & Q \\ Q^{\mathrm{T}} & R \end{pmatrix}$ is PSD if and only if $R - Q^{\mathrm{T}}P^{-1}Q$ is PSD.

Proof. For u, w of the right sizes,

$$\begin{pmatrix} u \\ w \end{pmatrix}^{\mathrm{T}} \begin{pmatrix} P & Q \\ Q^{\mathrm{T}} & R \end{pmatrix} \begin{pmatrix} u \\ w \end{pmatrix} = (u + P^{-1}Qw)^{\mathrm{T}} P\,(u + P^{-1}Qw) + w^{\mathrm{T}}(R - Q^{\mathrm{T}}P^{-1}Q)\,w.$$

The first term is never negative because P is positive definite, and it is zero at $u = -P^{-1}Qw$. So the form is nonnegative for all (u, w) exactly when the second term is nonnegative for all w. ■

The identity itself holds for any invertible symmetric P. One direction (block PSD gives the complement PSD) needs only that: put $u = -P^{-1}Qw$. The other direction is where P has to be positive definite. For the pinned matrix I don't need the lemma in the abstract: I carry out its one instance with $P = I_2$, $Q = E$, $R = I_2$, where the complement is

$$I_2 - E^{\mathrm{T}}E = \begin{pmatrix} 1 - E_{00}^2 - E_{10}^2 & -(E_{00}E_{01} + E_{10}E_{11}) \\ -(E_{00}E_{01} + E_{10}E_{11}) & 1 - E_{01}^2 - E_{11}^2 \end{pmatrix}.$$

The diagonal of $E^{\mathrm{T}}E$ holds the squared lengths of the columns of E, which is why the conditions below are column conditions.

Definition 8.6 (Column contractive)**.** Write $p = 1 - E_{00}^2 - E_{10}^2$, $q = 1 - E_{01}^2 - E_{11}^2$ and $s = E_{00}E_{01} + E_{10}E_{11}$. The correlators are column contractive when

$$p \geq 0, \qquad q \geq 0, \qquad p\,q - s^2 \geq 0.$$

Lemma 8.9 (The 2×2 test)**.** If $\alpha \geq 0$, $\delta \geq 0$ and $\alpha\delta - \beta^2 \geq 0$, then $\alpha u^2 + 2\beta uw + \delta w^2 \geq 0$ for all real u, w. □

Proof. If $w = 0$ the form is $\alpha u^2 \geq 0$. Otherwise put $t = u/w$, so the form is $w^2(\alpha t^2 + 2\beta t + \delta)$. If $\alpha = 0$, then $-\beta^2 \geq 0$ forces $\beta = 0$ and the bracket is $\delta \geq 0$. If $\alpha > 0$, $\alpha t^2 + 2\beta t + \delta = \alpha(t + \beta/\alpha)^2 + (\alpha\delta - \beta^2)/\alpha$, a sum of two nonnegative terms. ■

Theorem 8.10 (Contractive implies PSD)**.** If $p \geq 0$, $q \geq 0$ and $pq - s^2 \geq 0$, then $M(E)$ is positive semidefinite. □

Proof. Write $v = (v_0, x_0, x_1, y_0, y_1)$. Reading off the entries of $M(E)$,

$$v^{\mathrm{T}}M(E)\,v = v_0^2 + x_0^2 + x_1^2 + y_0^2 + y_1^2 + 2x_0(E_{00}y_0 + E_{01}y_1) + 2x_1(E_{10}y_0 + E_{11}y_1).$$

Complete the squares in x_0 and x_1:

$$\begin{aligned} v^{\mathrm{T}}M(E)\,v = v_0^2 &+ (x_0 + E_{00}y_0 + E_{01}y_1)^2 + (x_1 + E_{10}y_0 + E_{11}y_1)^2 \\ &+ p\,y_0^2 - 2s\,y_0y_1 + q\,y_1^2. \end{aligned}$$

To check the last three terms, expand the two squares: they contribute $(E_{00}y_0 + E_{01}y_1)^2 + (E_{10}y_0 + E_{11}y_1)^2$ beyond the cross terms, which is $(E_{00}^2 + E_{10}^2)y_0^2 + 2s\,y_0y_1 + (E_{01}^2 + E_{11}^2)y_1^2$; subtracting it from $y_0^2 + y_1^2$ leaves $p\,y_0^2 - 2s\,y_0y_1 + q\,y_1^2$.

The first three terms are squares. The last is the 2×2 form with $\alpha = p$, $\beta = -s$, $\delta = q$, nonnegative by Lemma 8.9. ■

□ **Lemma 8.11** (Discriminant). If $a + 2bt + ct^2 \geq 0$ for every real t, then $b^2 \leq ac$.

Proof. If $c < 0$ the quadratic tends to $-\infty$, so $c \geq 0$. If $c = 0$, the line $a + 2bt$ is nonnegative for every t only if $b = 0$, and then $b^2 = 0 = ac$. If $c > 0$, evaluate at $t = -b/c$: $a - b^2/c \geq 0$, so $b^2 \leq ac$. ■

□ **Theorem 8.12** (PSD implies contractive). If $M(E)$ is positive semidefinite, then $p \geq 0$, $q \geq 0$ and $pq - s^2 \geq 0$.

Proof. Feed the quadratic form three test vectors, written in the order (I, A_0, A_1, B_0, B_1). Each is the choice $x = -Ey$ that zeroes both completed squares in the proof of Theorem 8.10, so only $y^{\mathrm{T}}(I_2 - E^{\mathrm{T}}E)y$ is left.

First, $v = (0, -E_{00}, -E_{10}, 1, 0)$ gives $v^{\mathrm{T}}Mv = p$, so $p \geq 0$.

Second, $v = (0, -E_{01}, -E_{11}, 0, 1)$ gives $v^{\mathrm{T}}Mv = q$, so $q \geq 0$.

Third, for each real t, $v(t) = (0, -(E_{00}t + E_{01}), -(E_{10}t + E_{11}), t, 1)$ gives $v(t)^{\mathrm{T}}Mv(t) = p\,t^2 - 2s\,t + q \geq 0$. Apply Lemma 8.11 with $a = q$, $b = -s$, $c = p$: $s^2 \leq pq$. ■

□ **Corollary 8.13** (The equivalence). For all real $E_{00}, E_{01}, E_{10}, E_{11}$, the symmetric matrix $M(E)$ is positive semidefinite if and only if the correlators are column contractive.

Proof. Theorem 8.10 is one direction and Theorem 8.12 the other. ■

The equivalence above is about the columns. The bound can go through either the rows or the columns: the six-square identity only needs the four squares to add to at most 2, and the two row bounds give that, and so do $p \geq 0$ and $q \geq 0$.

□ **Theorem 8.14** (PSD implies Tsirelson). If the pinned matrix $M(E)$ is symmetric and positive semidefinite at $E_{00}, E_{01}, E_{10}, E_{11}$, then $S^2 \leq 8$ and $|S| \leq \sqrt{8}$.

Proof. Feed the form the vector that is 1 at A_0, $-E_{00}$ at B_0, $-E_{01}$ at B_1 and 0 elsewhere. Reading off the entries, the form is $1 + E_{00}^2 + E_{01}^2 - 2E_{00}^2 - 2E_{01}^2 = 1 - E_{00}^2 - E_{01}^2$ (the B_0B_1 cell is 0), so $E_{00}^2 + E_{01}^2 \leq 1$. The vector that is 1 at A_1, $-E_{10}$ at B_0 and $-E_{11}$ at B_1 gives $E_{10}^2 + E_{11}^2 \leq 1$ the same way. Then Theorem I.4 applies. ∎

Here are the three conditions and the score at a few points, all by direct arithmetic. In this table, $a = 3/5$, $b = 4/5$ and $t = 1/\sqrt{2}$.

point $(E_{00}, E_{01}, E_{10}, E_{11})$	S	p	q	$pq - s^2$	**contractive**
$(0, 0, 0, 0)$	0	1	1	1	yes
$(0.6, 0.6, 0.6, -0.6)$	2.4	0.28	0.28	0.0784	yes
$(a, b, b, -a)$	2.8	0	0	0	yes, with no slack
$(t, t, t, -t)$	$2\sqrt{2}$	0	0	0	yes, with no slack
$(1, 1, 1, 1)$, all-ones plan	2	-1	-1		no
$(1, 0, 1, 0)$, a classical mixture	2	-1	1		no
$(1, 1, 1, -1)$, PR box	4	-1	-1		no

Every deterministic plan has all four correlators equal to ± 1, so $p = 1 - 1 - 1 = -1$ and every one of the sixteen fails.

Remark 8.3 (What PSD of the pinned matrix means)**.** $M(E)$ is the level-1 moment matrix with the two within-party entries $\langle A_0A_1 \rangle$ and $\langle B_0B_1 \rangle$ pinned to zero. Its PSD is sufficient for the correlators to admit some PSD completion (Section I.7), and it is not necessary: the all-ones plan has a completion but fails here. So a check built on $M(E)$ is sound for the completion set and not complete for it, and it rejects some honest classical tallies. That correlators with a PSD completion are exactly those reachable by quantum measurements is Tsirelson's theorem, and both halves are proved here in finite dimension. For real and for complex amplitudes, a strategy's correlators are inner products of unit vectors (with complex amplitudes, use real vectors of twice the length) and its level-1 moment matrix is PSD, so they have a PSD completion. The other half, that every PSD completion comes from a quantum strategy, comes from this construction: the strategy is built on two four-level systems from the completion's Gram vectors (Section I.8).

8.5 The integer check

The machine has whole-number tallies. The three conditions above were written using averages, which are fractions. You can test the same conditions without ever dividing: first make sure each denominator is positive, then multiply it away. Multiplying by a positive number cannot turn a passing inequality into a failing one or a failing one into a passing one.

For the first column, the condition starts as one minus two squared averages. Multiplying by the product of their squared denominators leaves three whole-number products. I call that result A. The second column gives B the same way. The cross-column term is a sum of two products of averages; multiplying by all four denominators leaves a whole number I call C. The last condition becomes $C^2 \le AB$, because both sides have had the same positive denominator removed. Four positive bucket sizes and these three comparisons make seven checks. Here are the products to calculate.

Definition 8.7 (The seven-integer check)**.** From the counters of Definition I.2, with all arithmetic in $\mathbb{Z}$, let

$$\begin{aligned} A &= N_{00}^2 N_{10}^2 - D_{00}^2 N_{10}^2 - D_{10}^2 N_{00}^2, \\ B &= N_{01}^2 N_{11}^2 - D_{01}^2 N_{11}^2 - D_{11}^2 N_{01}^2, \\ C &= D_{00} D_{01} N_{10} N_{11} + D_{10} D_{11} N_{00} N_{01}. \end{aligned}$$

The check is the conjunction of seven facts, in this order: $0 < N_{00}$, $0 < N_{01}$, $0 < N_{10}$, $0 < N_{11}$, $0 \le A$, $0 \le B$, and $C \cdot C \le A \cdot B$.

There is no division, no real number, no square root and no S in it.

□ **Lemma 8.15** (The seven facts are the three conditions with denominators cleared)**.** Assume $N_{00}, N_{01}, N_{10}, N_{11} > 0$ and put $E_{xy} = D_{xy}/N_{xy}$. Then, with p, q, s as in Definition I.6,

$$p = \frac{A}{N_{00}^2 N_{10}^2}, \qquad q = \frac{B}{N_{01}^2 N_{11}^2}, \qquad s = \frac{C}{N_{00} N_{01} N_{10} N_{11}},$$

$$pq - s^2 = \frac{AB - C^2}{(N_{00} N_{01} N_{10} N_{11})^2}.$$

Because every denominator is positive, $0 \le A \iff p \ge 0$, $0 \le B \iff q \ge 0$ and $C^2 \le AB \iff s^2 \le pq$.

Proof. For p: $1 - D_{00}^2/N_{00}^2 - D_{10}^2/N_{10}^2$; multiply by $N_{00}^2 N_{10}^2$ to get $N_{00}^2 N_{10}^2 - D_{00}^2 N_{10}^2 - D_{10}^2 N_{00}^2 = A$. For q the same with columns 01 and 11, multiplied by $N_{01}^2 N_{11}^2$. For s:

$$\frac{D_{00}}{N_{00}}\frac{D_{01}}{N_{01}} + \frac{D_{10}}{N_{10}}\frac{D_{11}}{N_{11}} = \frac{D_{00}D_{01}N_{10}N_{11} + D_{10}D_{11}N_{00}N_{01}}{N_{00}N_{01}N_{10}N_{11}} = \frac{C}{N_{00}N_{01}N_{10}N_{11}}.$$

Then $pq = AB/(N_{00}^2 N_{10}^2 N_{01}^2 N_{11}^2) = AB/(N_{00}N_{01}N_{10}N_{11})^2$ and $s^2 = C^2/(N_{00}N_{01}N_{10}N_{11})^2$; subtract. Multiplying an inequality by a positive number preserves it in both directions. ∎

The check has the following guarantee:

Theorem 8.16 (Soundness of the integer check)**.** If a tally w passes the seven-integer check, then its four correlators $E_{xy} = D_{xy}/N_{xy}$ are column contractive. Consequently its pinned matrix is positive semidefinite. □

Proof. The first four tests say that every bucket contains a round, so each average is its signed total divided by a positive count. The next two tests say that A and B are nonnegative. Dividing each by its positive denominator gives $p \geq 0$ and $q \geq 0$. The last test says $AB - C^2 \geq 0$. Divide that difference by the positive square of the product of all four counts; the result is $pq - s^2 \geq 0$, as the preceding calculation shows. Those are exactly the three column conditions, and Theorem 8.10 shows that they make the matrix pass the nonnegative-total test. ∎

Corollary 8.17 (A passed check implies the Tsirelson bound)**.** If a tally w passes the seven-integer check, the correlators of w satisfy $S^2 \leq 8$. It is the composition of Theorem I.10, Corollary I.7 and Theorem I.8. □

Two of the seven facts carry the bound by themselves. With every pair sampled, $0 \leq A$ and $0 \leq B$ are $p \geq 0$ and $q \geq 0$, so the four squared correlators add to at most 2 and Lemma I.3 gives $S^2 \leq 8$. The fact $C \cdot C \leq A \cdot B$ is what positive semidefiniteness adds, and the bound doesn't use it. For the form this takes on the small machine, where a raised flag has to stand on a passing check, see Appendix H.

A worked tally. Take $(\text{same}, \text{diff}) = (4,1),(4,1),(4,1),(1,4)$ for the pairs $00, 01, 10, 11$. Then $N_{xy} = 5$ for all four, $D_{00} = D_{01} = D_{10} = 3$, $D_{11} = -3$, and the correlators are $0.6, 0.6, 0.6, -0.6$ with $S = 2.4$. Compute:

$$A = 25 \cdot 25 - 9 \cdot 25 - 9 \cdot 25 = 625 - 225 - 225 = 175,$$
$$B = 25 \cdot 25 - 9 \cdot 25 - 9 \cdot 25 = 175,$$
$$C = 3 \cdot 3 \cdot 5 \cdot 5 + 3 \cdot (-3) \cdot 5 \cdot 5 = 225 - 225 = 0.$$

All seven facts hold: four positive N, $0 \le 175$ twice, and $0 \le 175 \cdot 175$. Dividing back, $p = 175/625 = 0.28$ and $s = 0$, the second row of the table in Section I.4.

Two more, for contrast. One round per pair from the all-ones plan, $(1,0)$ four times, gives $N = D = 1$ everywhere and $A = 1 - 1 - 1 = -1 < 0$: the check fails. The tally $(4,1),(9,1),(9,1),(1,4)$ gives correlators $3/5, 4/5, 4/5, -3/5$, with $N = 5, 10, 10, 5$ and $D = 3, 8, 8, -3$: $A = 25 \cdot 100 - 9 \cdot 100 - 64 \cdot 25 = 0$, $B = 100 \cdot 25 - 64 \cdot 25 - 9 \cdot 100 = 0$ and $C = 3 \cdot 8 \cdot 10 \cdot 5 + 8 \cdot (-3) \cdot 5 \cdot 10 = 0$. It passes with no slack, at $S = 2.8$.

8.6 Level 1+AB

The first table had one place for each player's answer. Enlarge it by giving each Alice–Bob product a place too. There are four such products, so the five places become nine. A cell still asks for the average product of its row and column labels. Some of those products reduce to entries I already have; five more numbers are needed for the particular filling below. Call them γ_1 through γ_5. The longer labels say which products they stand for.

The five additional parameters name the moments $\gamma_1 = \langle A_0 A_1 B_0 \rangle$, $\gamma_2 = \langle A_0 A_1 B_1 \rangle$, $\gamma_3 = \langle A_0 B_0 B_1 \rangle$, $\gamma_4 = \langle A_1 B_0 B_1 \rangle$ and $\gamma_5 = \langle A_0 A_1 B_0 B_1 \rangle$. These are labels for the entries; the definition below does not construct operators.

Definition 8.8 (The level-1+AB matrix)**.** The 9×9 real matrix on rows and columns $I, A_0, A_1, B_0, B_1, A_0B_0, A_0B_1, A_1B_0, A_1B_1$, with the following entries:

	I	A_0	A_1	B_0	B_1	A_0B_0	A_0B_1	A_1B_0	A_1B_1
I	1	0	0	0	0	E_{00}	E_{01}	E_{10}	E_{11}
A_0	0	1	0	E_{00}	E_{01}	0	0	γ_1	γ_2
A_1	0	0	1	E_{10}	E_{11}	γ_1	γ_2	0	0
B_0	0	E_{00}	E_{10}	1	0	0	γ_3	0	γ_4
B_1	0	E_{01}	E_{11}	0	1	γ_3	0	γ_4	0
A_0B_0	E_{00}	0	γ_1	0	γ_3	1	0	0	γ_5
A_0B_1	E_{01}	0	γ_2	γ_3	0	0	1	$-\gamma_5$	0
A_1B_0	E_{10}	γ_1	0	0	γ_4	0	$-\gamma_5$	1	0
A_1B_1	E_{11}	γ_2	0	γ_4	0	γ_5	0	0	1

Two constraints are fixed by this layout. First, the four-body cells carry opposite signs: $+\gamma_5$ at (A_0B_0, A_1B_1) and $-\gamma_5$ at (A_0B_1, A_1B_0). In a real moment matrix those two cells are $\langle A_0A_1B_0B_1\rangle$ and $\langle A_0A_1B_1B_0\rangle$, so the tie says $\langle A_0A_1(B_0B_1 + B_1B_0)\rangle = 0$, which for a real matrix is $\langle\{A_0, A_1\} \otimes \{B_0, B_1\}\rangle = 0$. The level-1 pinnings don't imply it. Take a classical strategy: Alice's two answers a_0, a_1 are independent fair signs, and Bob copies them, $b_0 = a_0$ and $b_1 = a_1$. Every pinned cell of its moment matrix is 0 and both four-body cells are 1, so no choice of the nine numbers gives its matrix. So the matrix here is level 1+AB cut down to the slice $\langle\{A_0, A_1\} \otimes \{B_0, B_1\}\rangle = 0$, and that is what "level 1+AB" names in this book. The slice holds whenever A_0 and A_1 anticommute. The soundness theorems below are about this matrix, so they stand as stated. Second, the matrix keeps the level-1 pinning: every cell whose label reduces to a marginal or to $\langle A_0A_1\rangle$ or $\langle B_0B_1\rangle$ is zero, for instance (A_0, A_0B_0), (A_0B_0, A_0B_1) and (A_0B_0, A_1B_0). The displayed matrix is symmetric.

I want to test this larger table with the same operation as before: put a weight beside each row and column and add the weighted cells. The first three weights will each be collected into one completed square. Everything involving the other six will be left over. That leftover is what I call the residual; it is the part whose sign I still need to settle.

Write a vector as $(v_I, v_{A_0}, v_{A_1}, v_{B_0}, v_{B_1}, v_{00}, v_{01}, v_{10}, v_{11})$, where v_{xy} is the coordinate of A_xB_y. Put

$$
\begin{aligned}
L_1 &= E_{00}v_{00} + E_{01}v_{01} + E_{10}v_{10} + E_{11}v_{11},\\
L_2 &= E_{00}v_{B_0} + E_{01}v_{B_1} + \gamma_1 v_{10} + \gamma_2 v_{11},\\
L_3 &= E_{10}v_{B_0} + E_{11}v_{B_1} + \gamma_1 v_{00} + \gamma_2 v_{01}.
\end{aligned}
$$

Definition 8.9 (The residual)**.**

$$\begin{aligned} R = {} & v_{B_0}^2 + v_{B_1}^2 + v_{00}^2 + v_{01}^2 + v_{10}^2 + v_{11}^2 \\ & + 2v_{B_0}(\gamma_3 v_{01} + \gamma_4 v_{11}) + 2v_{B_1}(\gamma_3 v_{00} + \gamma_4 v_{10}) \\ & + 2\gamma_5(v_{00}v_{11} - v_{01}v_{10}) - L_1^2 - L_2^2 - L_3^2. \end{aligned}$$

The condition called column contractivity at level 1+AB is $R \geq 0$ for all six real inputs.

□ **Lemma 8.18** (Three squares plus the residual)**.** For every vector,

$$v^{\mathrm{T}} M_9\, v = (v_I + L_1)^2 + (v_{A_0} + L_2)^2 + (v_{A_1} + L_3)^2 + R.$$

Proof. The coordinates v_I, v_{A_0}, v_{A_1} appear in the form only through their diagonal squares and the cross terms $2v_I L_1$, $2v_{A_0} L_2$, $2v_{A_1} L_3$, read off rows I, A_0, A_1 of the matrix. Completing each square adds $L_1^2 + L_2^2 + L_3^2$, which the residual subtracts. What is left in R is the diagonal of the remaining six coordinates and the cross terms among them, read off the lower right 6×6 block. ■

□ **Theorem 8.19** (PSD at level 1+AB)**.** For all real E_{xy} and $\gamma_1, \ldots, \gamma_5$, the matrix of Definition I.7 is positive semidefinite if and only if the residual R is nonnegative for every choice of its six real inputs.

Proof. If $R \geq 0$ everywhere, Lemma 8.18 writes the form as three squares plus R. Conversely, given the six residual inputs, choose $v_I = -L_1$, $v_{A_0} = -L_2$, $v_{A_1} = -L_3$; the three squares vanish and PSD gives $R \geq 0$. This is the level-1 argument of Section I.4 one size up. ■

As a quadratic form in $(v_{B_0}, v_{B_1}, v_{00}, v_{01}, v_{10}, v_{11})$, numbered 1 to 6, the residual has the symmetric coefficient matrix H with these upper entries. Each entry can be read back off R by collecting the coefficient of $v_i v_j$.

$$
\begin{aligned}
H_{11} &= 1 - E_{00}^2 - E_{10}^2, & H_{22} &= 1 - E_{01}^2 - E_{11}^2,\\
H_{12} &= -(E_{00}E_{01} + E_{10}E_{11}), & H_{13} &= -E_{10}\gamma_1,\\
H_{14} &= \gamma_3 - E_{10}\gamma_2, & H_{15} &= -E_{00}\gamma_1,\\
H_{16} &= \gamma_4 - E_{00}\gamma_2, & H_{23} &= \gamma_3 - E_{11}\gamma_1,\\
H_{24} &= -E_{11}\gamma_2, & H_{25} &= \gamma_4 - E_{01}\gamma_1,\\
H_{26} &= -E_{01}\gamma_2, & H_{33} &= 1 - E_{00}^2 - \gamma_1^2,\\
H_{34} &= -(E_{00}E_{01} + \gamma_1\gamma_2), & H_{35} &= -E_{00}E_{10},\\
H_{36} &= \gamma_5 - E_{00}E_{11}, & H_{44} &= 1 - E_{01}^2 - \gamma_2^2,\\
H_{45} &= -\gamma_5 - E_{01}E_{10}, & H_{46} &= -E_{01}E_{11},\\
H_{55} &= 1 - E_{10}^2 - \gamma_1^2, & H_{56} &= -(E_{10}E_{11} + \gamma_1\gamma_2),\\
H_{66} &= 1 - E_{11}^2 - \gamma_2^2.
\end{aligned}
$$

The top left 2×2 block is $I_2 - E^{\mathrm{T}}E$ from Section I.4.

The full check does not decide $R \geq 0$ directly. It tests the stronger fact that H is positive definite, by two Schur steps and a Sylvester test, all fraction-free.

Lemma 8.20 (Fraction-free Schur step)**.** For a symmetric $n \times n$ matrix h and any vector v, with $S_{ij} = h_{11}h_{ij} - h_{1i}h_{1j}$ for $i, j \geq 2$ (here S is the Schur matrix; the score plays no part in this lemma), □

$$h_{11}\, v^{\mathrm{T}} h\, v = \Big(\sum_j h_{1j} v_j\Big)^2 + \sum_{i,j \geq 2} S_{ij} v_i v_j.$$

So if $h_{11} > 0$ and the $(n-1) \times (n-1)$ matrix S gives a nonnegative form, then h does too. The cascade uses this identity at sizes six and five.

Proof. Expand the square: $h_{11}^2 v_1^2 + 2h_{11}v_1 \sum_{j\geq 2} h_{1j}v_j + (\sum_{j\geq 2} h_{1j}v_j)^2$. The left side is $h_{11}^2 v_1^2 + 2h_{11}v_1 \sum_{j\geq 2} h_{1j}v_j + h_{11} \sum_{i,j\geq 2} h_{ij}v_i v_j$. The difference is $\sum_{i,j\geq 2}(h_{11}h_{ij} - h_{1i}h_{1j})v_i v_j$. Dividing by $h_{11} > 0$ keeps the sign. ■

Lemma 8.21 (Sylvester for 4×4)**.** Let h be symmetric 4×4 with leading principal minors d_1, d_2, d_3, d_4. If d_1, d_2, d_3 are strictly positive and $d_4 \geq 0$, then $v^{\mathrm{T}} h\, v \geq 0$ for all v. □

Proof. Use the fraction-free factorization

$$d_1^2 d_2 d_3 \, v^{\mathrm{T}} h \, v = d_1 d_2 d_3 \, P_1^2 + d_1 d_3 \, Q_1^2 + Q_2^2 + d_1^2 d_2 d_4 \, v_4^2,$$

where, writing $e_{ij} = h_{11} h_{ij} - h_{1i} h_{1j}$,

$$\begin{aligned} P_1 &= \sum_j h_{1j} v_j, \\ Q_1 &= d_2 v_2 + e_{23} v_3 + e_{24} v_4, \\ Q_2 &= d_1 d_3 v_3 + (d_2 e_{34} - e_{23} e_{24}) v_4. \end{aligned}$$

The identity is Lemma 8.20 three times over, each time on the matrix the last one left. With $f_{ij} = e_{22} e_{ij} - e_{2i} e_{2j}$ for $i, j \geq 3$, the lemma gives $d_1 \, v^{\mathrm{T}} h \, v = P_1^2 + \sum_{i,j \geq 2} e_{ij} v_i v_j$, then $e_{22} \sum_{i,j \geq 2} e_{ij} v_i v_j = Q_1^2 + \sum_{i,j \geq 3} f_{ij} v_i v_j$, then $f_{33} \sum_{i,j \geq 3} f_{ij} v_i v_j = Q_2^2 + (f_{33} f_{44} - f_{34}^2) \, v_4^2$. The corners are minors of h, by one fact about determinants: the $k \times k$ leading minor of e is h_{11}^{k-1} times the $(k+1) \times (k+1)$ leading minor of h. To see it, multiply rows 2 to $k+1$ of that block of h by h_{11}, which multiplies its determinant by h_{11}^k, then subtract h_{1i} times the first row from row i, which changes nothing; those rows now read $0, e_{i2}, \ldots, e_{i,k+1}$, so the determinant is h_{11} times the leading minor of e. So $e_{22} = d_2$ and $f_{33} = e_{22} e_{33} - e_{23}^2 = d_1 d_3$, and the same fact applied to e, whose corner is d_2, gives $f_{33} f_{44} - f_{34}^2 = d_2 \det e = d_1^2 d_2 d_4$. Multiply the first equation by $d_1 d_2 d_3$ and substitute the other two. With $d_1, d_2, d_3 > 0$ and $d_4 \geq 0$ the right side is nonnegative and the factor on the left is positive. ∎

The cascade for the full check runs on H. Step one: $H_{11} > 0$ and the 5×5 matrix $S_{ij}^{(6)} = H_{11} H_{ij} - H_{1i} H_{1j}$, $2 \leq i, j \leq 6$, gives a nonnegative form. Step two: $S_{22}^{(6)} > 0$ and the 4×4 matrix $S_{ij}^{(5)} = S_{22}^{(6)} S_{ij}^{(6)} - S_{2i}^{(6)} S_{2j}^{(6)}$, $3 \leq i, j \leq 6$, gives a nonnegative form. Step three: the four leading minors of $S^{(5)}$ are positive. Together these conditions make the residual nonnegative, and so the nine-by-nine matrix positive semidefinite.

Definition 8.10 (What the integer check computes)**.** The supplied moments arrive as bucket pairs: $\gamma_k = N_{g_k} / D_{g_k}$ with $N_{g_k} = \mathrm{same}_{g_k} - \mathrm{diff}_{g_k}$ and $D_{g_k} = \mathrm{same}_{g_k} + \mathrm{diff}_{g_k}$. Watch the letters: for these five buckets N names the numerator and D the denominator: the difference and the count, the reverse of Section I.1. Let $K = (N_{00} N_{01} N_{10} N_{11} D_{g_1} D_{g_2} D_{g_3} D_{g_4} D_{g_5})^2$. The check computes the 21 integers $K H_{ij}$, the 15 integers of $S^{(6)}$ at scale K^2 and the 10 integers of $S^{(5)}$ at scale K^4, each by $h_{11} h_{ij} - h_{1i} h_{1j}$ on integers. The decider is the conjunction of: $N_{xy} > 0$ for the four pairs; $D_{g_k} > 0$ for $k = 1..5$; $-D_{g_k} < N_{g_k} < D_{g_k}$ for $k = 1..5$; and six strict positivities, namely $K H_{11} > 0$, $K^2 S_{22}^{(6)} > 0$, and the four leading minors of the

integer $S^{(5)}$. The full check combines the seven-integer check of Definition 8.7 with this decider.

Each cleared integer, read as a real, equals the corresponding real quantity times the stated positive power of K. A $k \times k$ leading minor of a matrix scaled by $c > 0$ scales by c^k. So every integer positivity holds exactly when the real one does.

Two parts of the full check are there to fail early, and no proof below needs them. A passing decider makes H positive definite (Remark 8.4), and the top left two-by-two block of H has p and q on its diagonal and $-s$ off it, so $p > 0$, $q > 0$ and $pq > s^2$, and the seven-integer check passes. And with H positive definite, the nine-by-nine form is zero only at zero: it is three squares plus R (Lemma 8.18), so where it is zero the six inputs of R are zero, then so are L_1, L_2 and L_3, and then so are v_I, v_{A_0} and v_{A_1}. So the nine-by-nine matrix is positive definite; with ones on its diagonal, that puts every γ_k strictly between -1 and 1, which is the bucket bound $-D_{g_k} < N_{g_k} < D_{g_k}$.

Theorem 8.22 (The full 1+AB check implies PSD)**.** Let the counts make every N_{xy} and every D_{g_k} positive, and let the decider just defined pass on them. Then the matrix of Definition I.7 at the four correlators and $\gamma_k = N_{g_k}/D_{g_k}$ is symmetric and positive semidefinite. □

Proof. The bucket conjuncts make every denominator positive, the scaling remark just above carries the six integer positivities over to the real cascade, and Lemma 8.20 twice plus Lemma 8.21 give $R \geq 0$, and so positive semidefiniteness by Theorem I.12. ■

Remark 8.4 (Interior only)**.** Every test in the cascade is strict, so a passing check certifies that H is positive definite, which is stronger than semidefinite. A point where H is PSD but singular cannot pass, and neither can a moment with $|\gamma_k| = 1$. For example, the tally of Section I.5 with correlators $(3/5, 4/5, 4/5, -3/5)$ has $H_{11} = 1 - 9/25 - 16/25 = 0$, so the full 1+AB check fails on it for every choice of γ, though the level-1 check of Section I.5 accepts it. The argument establishes soundness only; no converse is stated.

A worked example of the cascade in exact rational arithmetic. Take the correlators $(0.6, 0.6, 0.6, -0.6)$ and $\gamma = (0, 0, 0, 0, -1/2)$; as buckets, γ_1 to γ_4 are $(1, 1)$, giving $0/2$, and γ_5 is $(1, 3)$, giving $-2/4$, both inside the strict bounds. Then $H_{11} = H_{22} = 7/25$, $H_{12} = 0$, and every entry H_{1j}, H_{2j} with $j \geq 3$ is zero. So

$S^{(6)}_{22} = (7/25)^2 = 49/625$, and $S^{(5)}$ is a positive multiple of the lower right block of H:

$$S^{(5)} = \frac{343}{781250}\begin{pmatrix} 32 & -18 & -18 & -7 \\ -18 & 32 & 7 & 18 \\ -18 & 7 & 32 & 18 \\ -7 & 18 & 18 & 32 \end{pmatrix},$$

where $50H_{33} = 32$, $50H_{34} = 50H_{35} = -18$, $50H_{36} = 50(-1/2 + 0.36) = -7$, $50H_{45} = 50(1/2 - 0.36) = 7$, and $50H_{46} = 50H_{56} = 18$. The leading minors of the integer matrix are 32, 700, 15000 and 140625, all positive. With $H_{11} = 7/25 > 0$ and $S^{(6)}_{22} > 0$, all six tests pass, and with Section I.5's $A = B = 175$, $C = 0$ the full check passes. At $\gamma_5 = 0$ instead, the same block becomes $I_4 - \mathbf{e}\mathbf{e}^{\mathrm{T}}$ with $\mathbf{e} = (0.6, 0.6, 0.6, -0.6)$, whose third leading minor is negative ($\frac{1}{25^3}$ times -1250), and the check fails.

That last fact is general. The zero-moment check takes no γ buckets; it combines the level-1 check with the integer form of $E^2_{00} + E^2_{01} + E^2_{10} + E^2_{11} \le 1$ with denominators cleared by $(N_{00}N_{01}N_{10}N_{11})^2$.

□ **Theorem 8.23** (The zero-moment check sits inside the classical bound)**.** If the zero-moment check accepts a tally w, then the four correlators of w satisfy $E^2_{00} + E^2_{01} + E^2_{10} + E^2_{11} \le 1$ and $|S| \le 2$.

Proof. The check's extra conjunct, divided back by the positive factor $(N_{00}N_{01}N_{10}N_{11})^2$, is $r = E^2_{00} + E^2_{01} + E^2_{10} + E^2_{11} \le 1$. Then Lemma I.3 gives ■ $S^2 \le 4r \le 4$.

The level-1 half of that check follows from $r \le 1$ on its own: p and q are each at least $1 - r$, and $pq - s^2 = 1 - r + (E_{00}E_{11} - E_{01}E_{10})^2$.

This is exactly the level-1+AB condition at $\gamma = 0$. There the residual splits into the level-1 form in (v_{B_0}, v_{B_1}) and $|\mathbf{v}|^2 - \langle \mathbf{e}, \mathbf{v}\rangle^2$ in $\mathbf{v} = (v_{00}, v_{01}, v_{10}, v_{11})$, where $\mathbf{e} = (E_{00}, E_{01}, E_{10}, E_{11})$. The second piece is nonnegative for every $\mathbf{v}$ exactly when $r \le 1$: if $r \le 1$, Cauchy–Schwarz gives $\langle \mathbf{e}, \mathbf{v}\rangle^2 \le r\,|\mathbf{v}|^2 \le |\mathbf{v}|^2$; if $r > 1$, the choice $\mathbf{v} = \mathbf{e}$ gives $r - r^2 < 0$.

So the zero-moment check certifies a strict part of the classical region: no point above $|S| = 2$, and none of the sixteen deterministic points either, since each has $|\mathbf{e}|^2 = 4$. A point above 2 needs supplied moments. The matrix is positive semidefinite at correlators $(3/5, 3/5, 3/5, -3/5)$, $\gamma = (0, 0, 0, 0, -1/2)$, with $S = 12/5$. The cleanest way to see it is the γ_5 slice, $\gamma_1 = \cdots = \gamma_4 = 0$. There the residual splits into the level-1 form in (v_{B_0}, v_{B_1}) and the form of $J - \mathbf{e}\mathbf{e}^{\mathrm{T}}$ in the four v_{xy}, where J is I_4 with γ_5 at $(00, 11)$ and $-\gamma_5$ at $(01, 10)$. For $-1 < \gamma_5 < 1$, J

is positive definite, and the second form is nonnegative exactly when $\mathbf{e}^{\mathrm{T}} J^{-1}\mathbf{e} \le 1$: apply Lemma I.6 to the block matrix $\begin{pmatrix} 1 & \mathbf{e}^{\mathrm{T}} \\ \mathbf{e} & J \end{pmatrix}$ once with the corner 1 as P and once, rows reordered, with J as P. Inverting the two 2×2 blocks of J and multiplying out, that is

$$(1-\gamma_5)\big[(E_{00}+E_{11})^2+(E_{01}-E_{10})^2\big] + (1+\gamma_5)\big[(E_{00}-E_{11})^2+(E_{01}+E_{10})^2\big] \le 2(1-\gamma_5^2).$$

At this point the left side is $\frac{3}{2}\cdot 0 + \frac{1}{2}\cdot(\frac{36}{25}+\frac{36}{25}) = \frac{36}{25} = 1.44$ and the right side is $2\cdot\frac{3}{4} = 1.5$, and the level-1 part holds ($p = q = 0.28$, $s = 0$). The slice test is an illustration, not a third check: on the counters $(4,1),(4,1),(4,1),(1,4)$ and the bucket $(1,3)$, the same comparison with the denominators multiplied out says yes, and the full check of Definition 8.10 accepts the same counters with γ_1 to γ_4 given as the buckets $(1,1)$, as the worked example above shows.

Both 1+AB checks, the full check and the zero-moment check, contain the seven-integer check as a conjunct, so a passing 1+AB check also passes the level-1 check and inherits $S^2 \le 8$ from Section I.5. This follows directly by composing the definitions and the bound.

8.7 The elliptope completion

Go back to the two within-player cells I filled with zero. Leave them blank this time. Keep all four measured cross-player averages fixed, and ask whether there is any way to fill those two blanks that makes the table pass. A failure with zeros no longer settles the question; another filling may work.

Call the first missing number x and the second y, and copy each into its reflected cell so the table stays symmetric. The collection of averages that admit a passing filling has a name, the *elliptope* completion set. Membership means that at least one choice of those two numbers works. It does not require every choice to work, or require the zero choice to work.

Definition 8.11 (Completion and membership)**.** Complete the matrix of Section I.4 by setting the within-party entries to free reals: x in the (A_0, A_1) cells and y in the (B_0, B_1) cells, marginals still zero. The correlators are elliptope realizable when some x, y make the completed matrix symmetric and PSD.

$$M(E;x,y) = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & x & E_{00} & E_{01} \\ 0 & x & 1 & E_{10} & E_{11} \\ 0 & E_{00} & E_{10} & 1 & y \\ 0 & E_{01} & E_{11} & y & 1 \end{pmatrix}$$

Its quadratic form at $v = (v_0, \ldots, v_4)$ is $\sum_k v_k^2 + 2xv_1v_2 + 2yv_3v_4 + 2E_{00}v_1v_3 + 2E_{01}v_1v_4 + 2E_{10}v_2v_3 + 2E_{11}v_2v_4$. The pinned matrix is the case $x = y = 0$, so PSD of $M(E)$ implies membership.

□ **Theorem 8.24** (Deterministic plans are inside)**.** If $a_0^2 = a_1^2 = b_0^2 = b_1^2 = 1$, then $(a_0b_0, a_0b_1, a_1b_0, a_1b_1)$ is elliptope realizable, with $x = a_0a_1$ and $y = b_0b_1$.

Proof. With these entries the form equals $v_0^2 + (a_0v_1 + a_1v_2 + b_0v_3 + b_1v_4)^2$. Expanding the second square gives $a_0^2v_1^2 + \cdots$ on the diagonal, which is $v_1^2 + \cdots$ by the unit squares, and the cross terms $2a_0a_1v_1v_2 = 2xv_1v_2$, $2a_0b_0v_1v_3 = 2E_{00}v_1v_3$, and so on for all six pairs. A sum of two squares is never negative. ■

The all-ones plan is one instance. The classical mixture $(1, 0, 1, 0)$ has a completion at $x = 1$, $y = 0$, with form $v_0^2 + (v_1 + v_2 + v_3)^2 + v_4^2$.

□ **Theorem 8.25** (Convexity and finite mixtures)**.** For $0 \le \lambda \le 1$, if E and F are elliptope realizable, so is $\lambda E + (1-\lambda)F$. For finitely many members with nonnegative weights summing to at most 1, the weighted sum is realizable. Every finite mixture of deterministic plans is realizable.

Proof. The form is affine in (E, x, y) for each fixed v. Take the completion $(\lambda x_1 + (1-\lambda)x_2, \lambda y_1 + (1-\lambda)y_2)$; the form at the mixture is the same mixture of two nonnegative forms. For several members, weight each one's completed matrix by its assigned share and add them. If the shares total less than one, give the missing share to the zero-correlator point with both missing entries zero; its completed matrix is the identity. The diagonal of the weighted matrix is then one, its cross entries are the desired weighted correlators, and every weighted total is a sum of nonnegative totals with nonnegative weights. It therefore supplies the required completion. The last statement combines this with Theorem 8.24. ■

Theorem 8.26 (Tsirelson's bound holds on the completion set)**.** If E is elliptope realizable then $S^2 \le 8$. □

Proof. For a symmetric PSD matrix the bilinear form obeys Cauchy-Schwarz, $\mathcal{B}(u,w)^2 \le \mathcal{Q}(u)\,\mathcal{Q}(w)$, because $\mathcal{Q}(u+tw) = \mathcal{Q}(u) + 2t\,\mathcal{B}(u,w) + t^2\mathcal{Q}(w) \ge 0$ for all t and Lemma 8.11 applies. Take $u_1 = e_{A_0}$ and $w_+ = e_{B_0} + e_{B_1}$: $\mathcal{B}(u_1, w_+) = E_{00} + E_{01}$, $\mathcal{Q}(u_1) = 1$, $\mathcal{Q}(w_+) = 2 + 2y$. Take $u_2 = e_{A_1}$ and $w_- = e_{B_0} - e_{B_1}$: $\mathcal{B}(u_2, w_-) = E_{10} - E_{11}$, $\mathcal{Q}(w_-) = 2 - 2y$. So $c_1^2 \le 2 + 2y$ and $c_2^2 \le 2 - 2y$ with $c_1 = E_{00} + E_{01}$, $c_2 = E_{10} - E_{11}$, and both budgets are nonnegative. Then $c_1^2 c_2^2 \le (2+2y)(2-2y) = 4 - 4y^2 \le 4$, so $c_1 c_2 \le 2$, and $S^2 = (c_1 + c_2)^2 = c_1^2 + 2c_1c_2 + c_2^2 \le (2+2y) + 4 + (2-2y) = 8$. ■

The completion can make a row of E longer than 1 (the all-ones plan has rows of squared length 2), which is why this proof does not use the row bounds of Section I.4.

Corollary 8.27 (The PR box is outside)**.** $(1, 1, 1, -1)$ is not elliptope realizable, since its $S^2 = 16 > 8$. □ ■

Theorem 8.28 (Strictly beyond the classical bound)**.** $(3/5, 3/5, 3/5, -3/5)$ is elliptope realizable and has $S = 12/5 > 2$. □

Proof. At $x = y = 0$ the form equals $v_0^2 + (v_1 + \frac{3}{5}v_3 + \frac{3}{5}v_4)^2 + (v_2 + \frac{3}{5}v_3 - \frac{3}{5}v_4)^2 + \frac{7}{25}v_3^2 + \frac{7}{25}v_4^2$. Check: the two middle squares give v_3^2 coefficient $\frac{9}{25} + \frac{9}{25} = \frac{18}{25}$, plus $\frac{7}{25}$ is 1; the v_3v_4 terms are $+\frac{18}{25}$ and $-\frac{18}{25}$ and cancel; the cross terms are $2 \cdot \frac{3}{5}v_1v_3$, $2 \cdot \frac{3}{5}v_1v_4$, $2 \cdot \frac{3}{5}v_2v_3$ and $-2 \cdot \frac{3}{5}v_2v_4$, as the entries require. ■

By Theorem I.1 no finite factorizable model has $S > 2$, so this point is not a finite mixture of plans, while Theorem 8.25 puts every such mixture inside. The completion set then strictly contains the finite classical set, and the PR box lies outside it. It sits inside the no-signalling set, for the plain reason that every correlator in it is between -1 and 1 (the vector that is 1 at A_x and ∓ 1 at B_y gives $2 \mp 2E_{xy} \ge 0$), and any four such numbers come from the no-signalling box that answers (a, b) at settings (x, y) with probability $(1 + ab\,E_{xy})/4$, each player's own answer alone a fair coin.

The elliptope gate is an integer decider with a soundness proof. Its completion entries arrive as bucket pairs, and an empty bucket is read as $0/1$, so unlike Section I.5 it does not require every pair to be sampled.

Definition 8.12 (The two branches)**.** The positive-minor branch: with the completion supplied as buckets and all entries scaled to integers by a positive common factor K, the four leading minors of the 4×4 block on A_0, A_1, B_0, B_1 are strictly positive. The factorization branch: with the completion and a rational certificate supplied (six subdiagonal entries over a common positive denominator, four pivots $d_k \geq 0$ over another), the ten entry equations of $M = LDL^{\mathrm{T}}$ hold in cross-multiplied integer arithmetic. The full gate is their disjunction.

□ **Theorem 8.29** (Soundness of the gate)**.** If the full gate returns yes on a counter record, any completion buckets and any certificate, then the four correlators of the record are elliptope realizable. On the PR box tally the gate returns no for every choice of the remaining arguments.

Proof. In both branches it is enough to show the 4×4 block on A_0, A_1, B_0, B_1 gives a nonnegative form, because the I row and column are 1 on the diagonal and 0 elsewhere. The first branch: dividing by the positive factor K (a $k \times k$ minor scales by K^k) gives four positive leading minors of the real block, and Lemma 8.21 applies. The second: cross-multiplying by positive denominators changes no equation, so $M = LDL^{\mathrm{T}}$ holds over the rationals with L unit lower triangular, and the form is $v^{\mathrm{T}}LDL^{\mathrm{T}}v = \sum_k d_k P_k^2$ with $P = L^{\mathrm{T}}v$ and every $d_k \geq 0$. The PR statement is ■ soundness followed by Corollary 8.27.

Examples by direct computation: the all-ones plan passes through the factorization branch at $x = y = 1$ with pivots $(1, 0, 0, 0)$; the classical mixture $(1, 0, 1, 0)$ passes through the same branch at $x = 1$, $y = 0$ with pivots $(1, 0, 0, 1)$; the $12/5$ point passes through the positive-minor branch at $x = y = 0$; and $(3/5, 4/5, 4/5, -3/5)$ passes through the factorization branch at $x = y = 0$ with pivots $(1, 1, 0, 0)$, since $B_0 = \frac{3}{5}A_0 + \frac{4}{5}A_1$ and $B_1 = \frac{4}{5}A_0 - \frac{3}{5}A_1$ reproduce every entry.

That's all the gate is: no property of the small machine is built from it.

8.8 The four-vector construction

Put two equal-length pencils down with their ends together, and draw a line along one of them. Keep that pencil still and turn the other. Use the square corner of a

sheet of paper to bring the moving pencil's tip straight across to the line, and mark where it lands. The mark moves from a full pencil-length forward, through the joined ends when the pencils make a square corner, to a full pencil-length backward. Call those readings 1, 0 and -1. That is the signed shadow I want. Between those positions the reading is the cosine of the angle. A dot product of unit vectors measures that shadow.

Now ask for four directions, two labelled Alice and two Bob, with a prescribed shadow for each Alice–Bob pair. Try this request: Alice's first points exactly along both of Bob's; Alice's second points along Bob's first and exactly against his second. Each request alone is easy. Put them together and your hands cannot do it: Bob's pencils would have to point both the same way and opposite ways. Four numbers in range need not describe four directions that can coexist.

Write those four readings as $x = E_{00}$, $y = E_{01}$, $z = E_{10}$ and $w = E_{11}$. I haven't specified the shadow between Alice's own two directions, or between Bob's own two. Those are the two blanks, t and u, in the following table. Can they be filled so that the whole table comes from directions? The pencils let you ask it in a plane; the construction will allow four coordinates. A completion is a positive-semidefinite matrix

$$G = \begin{pmatrix} 1 & t & x & y \\ t & 1 & z & w \\ x & z & 1 & u \\ y & w & u & 1 \end{pmatrix}.$$

The extra row and column labelled I in Section I.7 have diagonal 1 and all other entries zero, so they do not change this condition.

Why positivity? Lay arrows end to end, allowing a backward arrow when its weight is negative. Square the length from the start to the finish. Expanding that square adds up the arrows' pairwise shadows, with their weights. It cannot be negative. A proposed shadow table that gives a negative answer for some choice of weights cannot come from arrows. That is the PSD test. A Gram matrix is simply the table of all those pairwise dot products.

The converse has to build the arrows. Pick the first direction as a baseline. Give every other arrow the component along it that its shadow demands, then build the remaining components perpendicular to it. What remains to be fitted is a smaller shadow table. Here is that construction, including its boundary. If its first diagonal entry is zero, positivity of $h_{11}a^2 + 2h_{1j}ab + h_{jj}b^2$ for every a, b forces $h_{1j} = 0$: with $b = 1$, a nonzero h_{1j} would let the linear term go negative without bound. Delete that zero row and column, construct vectors for the smaller matrix, and put back

the zero vector. If $h_{11} > 0$, set

$$K_{ij} = h_{ij} - \frac{h_{1i}h_{1j}}{h_{11}} \quad (i, j > 1).$$

In the quadratic form of h, take the first coordinate to be $-\sum_{j>1} h_{1j}v_j/h_{11}$. The remaining form is $v^{\mathrm{T}}Kv$, so K is PSD. Repeat the construction on this smaller matrix. Each repetition removes a row and column, so it ends at a single nonnegative entry, represented by its square root, or at the empty table. Working back up supplies vectors r_i with $r_i \cdot r_j = K_{ij}$. Then $u_1 = (\sqrt{h_{11}}, 0)$ and $u_i = (h_{1i}/\sqrt{h_{11}}, r_i)$ have Gram matrix h. The dimension is at most the number of rows. Applied to G, this gives four unit vectors a_0, a_1, b_0, b_1 in $\mathbb{R}^4$, with the required cross inner products.

Three dimensions are always enough. The table fixes the cross entries and leaves two entries of G open, $a_0 \cdot a_1$ and $b_0 \cdot b_1$. Hold the first and let the second be t. The values of t that keep G PSD form an interval: if two values do, so does every value between, because the quadratic form at a mixture is the same mixture of the two forms; the set is closed, because a form that is nonnegative at every value of a convergent sequence is nonnegative at its limit; and it is bounded, because a PSD matrix with ones on its diagonal has every entry between -1 and 1. At an end of that interval G is singular: if it were positive definite there, it would stay positive definite under every small enough change of t, on both sides. Take G at an end. A singular PSD G has a nonzero c with $c^{\mathrm{T}}Gc = 0$, and $c^{\mathrm{T}}Gc$ is the squared length of $\sum_i c_i v_i$ for the four vectors v_i, so that sum is zero and the four vectors lie in a space of at most three dimensions. Coordinates in an orthonormal basis of that space keep every inner product, so a_0, a_1, b_0, b_1 can be taken in $\mathbb{R}^3$.

The next question is harder: how do directions become a prescription for a quantum measurement? Start with two signed cards in a row. One move swaps them; another changes the sign of the card in the right-hand place. Start at $(+,+)$. Swap then change the right sign: $(+,-)$. Start again, change the right sign then swap: $(-,+)$. Every sign is reversed. Try unequal numbers on the cards: the two orders still give negatives of each other. "Anticommute" means exactly that relation between the two moves. Do either move twice and everything is back.

Write the swap as X, the sign change as Z, and doing nothing as I:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \qquad I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

For the larger construction, label four places by the two-bit words 00 through 11 and put a signed number at each. An X in one bit-position swaps each pair of places differing at that bit; a Z reverses the signs at places whose bit there is 1. The two positions give the following three moves, written as real 4×4 matrices:

$$\Gamma_1 = X \otimes I, \quad \Gamma_2 = Z \otimes X, \quad \Gamma_3 = Z \otimes Z.$$

The tensor product has entries $(A \otimes B)_{(i,k),(j,l)} = A_{ij}B_{kl}$. Each Γ_i is symmetric and squares to the identity. For $i < j$, the first position where the two products differ has X against Z, and $XZ = -ZX$; all other positions commute. So $\Gamma_i\Gamma_j = -\Gamma_j\Gamma_i$. For a unit vector a, the symmetric matrix $A(a) = \sum_i a_i\Gamma_i$ then satisfies $A(a)^2 = \sum_i a_i^2 I = I$. Its two-outcome measurement has projectors $(I + A(a))/2$ and $(I - A(a))/2$: they are symmetric, square to themselves, are orthogonal, and sum to the identity.

Take $|\Phi_4\rangle = 4^{-1/2}\sum_{k=1}^{4}|k\rangle \otimes |k\rangle$. Expanding this sum gives

$$\langle\Phi_4|A \otimes B|\Phi_4\rangle = \frac{1}{4}\sum_{i,j} A_{ij}B_{ij} = \frac{1}{4}\operatorname{tr}(A^{\mathrm{T}}B).$$

For $i \neq j$, cyclicity of the trace and anticommutation give $\operatorname{tr}(\Gamma_i\Gamma_j) = -\operatorname{tr}(\Gamma_i\Gamma_j) = 0$; for $i = j$ the trace is 4. So the correlator of $A(a)$ and $A(b)$ is $a \cdot b$. Give Alice $A(a_0), A(a_1)$ and Bob $A(b_0), A(b_1)$. Their strategy realizes x, y, z, w. All three Γ_i have trace zero, so the one-party expectations vanish too. The same real matrices and state are also a complex strategy.

With complex amplitudes two levels on each side are enough. Add $Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$ to X and Z: the three are self-adjoint, square to I and anticommute in pairs, and they have trace zero. Give Alice $a_1X + a_2Y + a_3Z$ for her vector a, and Bob $b_1X - b_2Y + b_3Z$ for his vector b. The expansion above holds for complex entries too, with Φ_2 and a factor $\frac{1}{2}$, and the transpose of Alice's matrix is $a_1X - a_2Y + a_3Z$, since Y's transpose is $-Y$. So the correlator is $\frac{1}{2}\operatorname{tr}\big((a_1X - a_2Y + a_3Z)(b_1X - b_2Y + b_3Z)\big) = a \cdot b$.

What would that mean at a bench? Prepare the stated shared state, choose one of the two specified measurements on each side, and write down the two signs returned. For each pair of settings, count agreements as $+1$ and disagreements as -1. Their average is the correlator. The model predicts the four averages x, y, z, w; a finite tally fluctuates around its expectation. The signed-card moves let you check the measurement algebra by hand. Realizing that state and those measurements in a device, and comparing its tallies with the predictions, is the physical experiment.

Conversely, in a normalized quantum state ψ with self-adjoint measurements $A_x^2 = B_y^2 = I$ on separate systems, put $a_x = (A_x \otimes I)\psi$ and $b_y = (I \otimes B_y)\psi$. They are unit vectors and $\operatorname{Re}\langle a_x, b_y\rangle = E_{xy}$. Replace each complex coordinate by its real and imaginary parts. This preserves those real inner products, and their Gram matrix is a completion. A mixed state gives the same construction by taking the direct sum of the vectors of its pure components, weighted by the square roots of their probabilities. So the completion condition and finite-dimensional quantum realizability are equivalent.

The necessary direction does not require finite-dimensional vectors. If a positive normalized expectation ω on an algebra has self-adjoint A_x, B_y, with $\omega(A_x^2), \omega(B_y^2) \leq 1$, then the real symmetric matrix with entries $\operatorname{Re}\,\omega(X_iX_j)$ is

PSD: its quadratic form is $\omega(C^*C) \geq 0$ for the real linear combination C of the four observables. Increasing each diagonal entry to 1 preserves PSD. When Alice's observables commute with Bob's, its cross entries are precisely the real correlators. This supplies the same G. Including the unit as a fifth observable also gives the level-1 moment matrix with its marginals. The sufficient direction already used only two four-level systems.

8.9 The arcsine boundary, in both directions

Keep Bob's two directions hinged at one end. Alice's first direction has to make a specified angle with each. How far can Bob's hinge open while that is possible? Hold Alice's direction upright and sweep each of Bob's around it at its required angle. Point them towards the same side to close the hinge as far as it will go; point them towards opposite sides to open it as far as it will go. There is a whole interval of openings between those limits. Alice's second direction asks for another interval.

Draw both intervals on one strip marked from 0 to 180 degrees. If they miss, no hinge setting satisfies both. If they overlap, choose an opening in the overlap and construct each Alice direction at that opening. The algebra below permits each its own perpendicular component in four dimensions. The pencils show why the shared opening is the question; they need not fit the entire construction on a tabletop. In the impossible request above, one interval is just 0 degrees and the other just 180. There is nowhere to put the hinge.

Assume $x, y, z, w \in [-1, 1]$, and put $s_x = \arcsin x$, with analogous notation for the other three entries. A unit vector making angles $\alpha, \beta \in [0, \pi]$ with two unit vectors can exist at an angle δ between those two exactly when

$$|\alpha - \beta| \leq \delta \leq \min(\alpha + \beta, 2\pi - \alpha - \beta).$$

To see this, decompose the two vectors along and perpendicular to the first. Their inner product ranges from $\cos\alpha\cos\beta - \sin\alpha\sin\beta$ to $\cos\alpha\cos\beta + \sin\alpha\sin\beta$, by Cauchy–Schwarz; every value in that interval is attained by choosing the angle between their perpendicular components. Taking arccosines gives the displayed interval, including zero perpendicular components.

Apply this to Alice's first vector and Bob's two. Since $\alpha = \pi/2 - s_x$ and $\beta = \pi/2 - s_y$, Bob's angle must lie in

$$I_0 = [\, |s_x - s_y|,\ \pi - |s_x + s_y| \,].$$

Alice's second vector gives $I_1 = [\, |s_z - s_w|,\ \pi - |s_z + s_w| \,]$. Each interval is nonempty because $|a - b| + |a + b| = 2\max(|a|, |b|) \leq \pi$ for $a, b \in [-\pi/2, \pi/2]$. They

intersect exactly when

$$|s_x - s_y| + |s_z + s_w| \le \pi, \qquad |s_z - s_w| + |s_x + s_y| \le \pi.$$

Using $|a| + |b| = \max(|a+b|, |a-b|)$, these are the four inequalities

$$|s_x + s_y + s_z - s_w| \le \pi, \qquad |s_x + s_y - s_z + s_w| \le \pi,$$
$$|s_x - s_y + s_z + s_w| \le \pi, \quad |-s_x + s_y + s_z + s_w| \le \pi.$$

So a completion implies all four.

For the converse, choose any $\delta \in I_0 \cap I_1$ and set $b_0 = e_1$, $b_1 = \cos\delta\, e_1 + \sin\delta\, e_2$. If $0 < \delta < \pi$, put

$$c_0 = \frac{y - x\cos\delta}{\sin\delta}, \qquad c_1 = \frac{w - z\cos\delta}{\sin\delta},$$
$$a_0 = x e_1 + c_0 e_2 + \sqrt{1 - x^2 - c_0^2}\, e_3,$$
$$a_1 = z e_1 + c_1 e_2 + \sqrt{1 - z^2 - c_1^2}\, e_4.$$

The interval conditions say exactly that the two radicands are nonnegative: after multiplication by $\sin^2\delta$, the first is $1 + 2xy\cos\delta - x^2 - y^2 - \cos^2\delta \ge 0$, and likewise for z, w. These four vectors have unit length and the prescribed cross inner products. If $\delta = 0$, the intervals force $x = y$ and $z = w$; take $b_0 = b_1 = e_1$, $a_0 = x e_1 + \sqrt{1-x^2}e_3$, $a_1 = z e_1 + \sqrt{1-z^2}e_4$. If $\delta = \pi$, they force $y = -x$ and $w = -z$; use the same Alice vectors and $b_1 = -b_0$. Their Gram matrix supplies the completion in every case. Together with the preceding construction, this proves the stated quantum characterization.

8.10 The finite local model, in both directions

Make sixteen cards, each with four boxes labelled A_0, A_1, B_0, B_1, and put every possible arrangement of plus and minus signs on them, one arrangement per card. Draw a card before choosing which setting to ask each player about. Alice reads her chosen box; Bob reads his. Neither needs the other's setting. How often each card is drawn is the only choice left in this model.

Try to get perfect agreement for the first three pairs and perfect disagreement for the fourth. Any card that ever gets drawn would need $A_0 = B_0$, $A_0 = B_1$ and $A_1 = B_0$, which makes $A_1 = B_1$ too. Changing how often you draw the cards cannot fix it. For less extreme requests the question is quantitative: can you assign nonnegative weights to the cards, adding to one, that give all four requested averages? The following construction supplies those weights exactly when all eight CHSH inequalities hold and the correlators are in $[-1, 1]$.

There is a useful way to assemble the deck. First allow only Bob's first question and make a distribution of the three answers A_0, A_1, B_0. Then do the same for his second question. Both smaller decks must draw Alice's answer pairs in the same proportions. The adjustable number t is Alice's agreement-minus-disagreement average across her two boxes. Choose it in both intervals

$$J_0 = [\,-1+|x+z|,\ 1-|x-z|\,], \qquad J_1 = [\,-1+|y+w|,\ 1-|y-w|\,].$$

Each interval is nonempty. They overlap exactly when $|x+z|+|y-w| \le 2$ and $|y+w|+|x-z| \le 2$, which expand to the eight CHSH inequalities. For $a_0, a_1, b \in \{-1,1\}$ set

$$P_0(a_0,a_1,b) = \frac{1 + t a_0 a_1 + x a_0 b + z a_1 b}{8},$$
$$P_1(a_0,a_1,b) = \frac{1 + t a_0 a_1 + y a_0 b + w a_1 b}{8}.$$

When $a_0 = a_1$, nonnegativity of P_0 is $1+t \ge |x+z|$; when $a_0 = -a_1$, it is $1-t \ge |x-z|$. These are precisely $t \in J_0$. The same check for P_1 uses J_1. Summing signs shows that both distributions have mass 1 and the desired pair correlators. Their common Alice marginal is $P_A(a_0,a_1) = (1 + t a_0 a_1)/4$.

To join the two smaller decks, first draw Alice's pair with its common probability. Among the first deck's cards bearing that pair, draw Bob's first answer; among the second deck's cards bearing the same pair, draw his second. Those conditional draws give a four-answer card. A pair that never occurs needs no card. Written as probabilities, this procedure is

$$P(a_0,a_1,b_0,b_1) = \begin{cases} P_0(a_0,a_1,b_0)\,P_1(a_0,a_1,b_1)/P_A(a_0,a_1), & P_A(a_0,a_1) > 0, \\ 0, & P_A(a_0,a_1) = 0. \end{cases}$$

If $P_A = 0$, nonnegativity forces both corresponding triple distributions to be zero. Otherwise summing over either Bob answer cancels the denominator. So P has the two triple distributions as marginals and total mass 1. Sample one of its sixteen answer quadruples and let each party answer with its assigned sign. This is a finite mixture of deterministic local plans with exactly the four required correlators.

The other direction is one line per card. On a single card, $A_0(B_0+B_1) + A_1(B_0 - B_1)$ is ± 2, because one bracket is 0 and the other is ± 2. Each of the other seven CHSH expressions is this one with the boxes relabelled or a sign flipped, so it is ± 2 on every card too, and any weighting of the cards keeps all eight averages between -2 and 2 (Theorem I.1 is the first of them). So a deck exists exactly when the correlators are in $[-1,1]$ and the eight inequalities hold.

8.11 Scope of the constructions

The constructions above pin down the correlators. What follows is where they stop: they aren't a completeness claim for the integer check, they say nothing about full behaviours with marginals, and they say nothing about data from a lab.

The arcsine boundary. For two settings and two outcomes per side, Tsirelson, Landau (1988) and Masanes (2003) give theorems characterizing the correlators reachable by quantum measurements: $|\arcsin E_{00} + \arcsin E_{01} + \arcsin E_{10} - \arcsin E_{11}| \leq \pi$ and its sign variants. The characterization holds as an if and only if with the completion set and, through Tsirelson's representation, with the quantum set. The point $(3/5, 4/5, 4/5, -3/5)$ is on the boundary: $\arcsin \frac{3}{5} + \arcsin \frac{4}{5} = \frac{\pi}{2}$ by the 3-4-5 triangle, so the sum is π; Tsirelson's point is on it too. The necessary direction holds in any dimension, for any expectation on an algebra of observables in which Alice's commute with Bob's.

Equality with the quantum set. That the completion set of Section I.7 is exactly the set of quantum correlators is Tsirelson's representation theorem, and it is proved in finite dimension: for both real and complex amplitudes, with two four-level systems enough for every point, and two two-level ones with complex amplitudes. That PSD of the level-1 moment matrix is necessary for quantum behaviour is proved for strategies in finite dimension, with real amplitudes and with complex ones, and in any dimension, marginals included, for any expectation on an algebra of observables.

Achievability. Theorem I.5 is a point of $\mathbb{R}^4$. A state and measurements that reach it are explicit: the two-qubit state $(|00\rangle + |11\rangle)/\sqrt{2}$ with $A_0 = Z$, $A_1 = X$, $B_0 = (Z + X)/\sqrt{2}$ and $B_1 = (Z - X)/\sqrt{2}$ has correlators $1/\sqrt{2}$, $1/\sqrt{2}$, $1/\sqrt{2}$, $-1/\sqrt{2}$ and $S = 2\sqrt{2}$, and every strategy in finite dimension has $S^2 \leq 8$, with real amplitudes and with complex ones. Both are arithmetic about the model. How close a real laboratory comes, and what that costs in loopholes closed, is the story told in Appendix H.

Completeness of the checks. The 1+AB cascade and the elliptope gate are proved sound. The cascade has no proved converse; the gate's factorization branch accepts every rational PSD completion, by the row-peeling of Section I.8 with each pivot kept. The level-1 integer check is the exception: on the small machine the same seven integer facts hold exactly when every pair was sampled and the pinned matrix is positive semidefinite (Theorem H.4), so that check is complete for what it tests. The 1+AB cascade is strict and misses boundary points. The two 1+AB checks don't sit inside one another. The full check accepts the $12/5$ tally above, which the zero-moment check refuses, and it refuses the tally with $E_{00} = 1$ and the other three correlators 0, where $H_{11} = 0$, which the zero-moment check accepts.

Finite levels. For full behaviours, marginals included, the NPA levels are relaxations. Chaturvedi (arXiv 2607.14569, 2026) argues, in a preprint, that no finite NPA level characterizes the complete quantum set even in this simplest Bell scenario. I haven't gone through that proof, and nothing here rests on it.

Statistics and physics. Nothing here says how many rounds make a tally informative, and nothing says the counters came from honest play; a program can INC whatever counts it likes.

8.12 Prior work

Tsirelson's 1980 paper gives the bound $2\sqrt{2}$ for quantum correlations, and his theorem says that quantum two-outcome correlators are inner products of real unit vectors. Landau (1988) and Masanes (2003, arXiv quant-ph/0309137) each give a condition that, by their proofs, characterizes the quantum correlators exactly for two settings and two outcomes. Wehner (2006, arXiv quant-ph/0510076) gets Tsirelson bounds, CHSH among them, from a semidefinite program on the 4×4 Gram matrix of those vectors. Navascués, Pironio and Acín (2007, arXiv quant-ph/0607119; 2008, arXiv 0803.4290) built the hierarchy of moment-matrix tests; their 2008 paper applies a Schur complement to the level-1 matrix of this scenario, recovers the arcsine condition, and discusses the 1+AB level. I went back to each of those papers for these points, and that is what I found in them. The results are theirs. Two of them are proved again here, in finite dimension: Tsirelson's representation and the arcsine characterization; the rest of their proofs are not redone. Echenim and Mhalla give a detailed derivation of Tsirelson's bound (arXiv 2306.12535), and exact rational sum-of-squares certificates go back at least to Peyrl and Parrilo (2008). So the moment-matrix route to $2\sqrt{2}$ is standard. What is specific here is narrower: an integer certificate computed from outcome counts, and the argument from "the counts pass" to PSD of the pinned matrix and on to $S^2 \leq 8$. I did not find that chain elsewhere, and I make no claim beyond it. On the host machine, the same chain is earned: a raised flag from a clean start stands on a passing check of the packed counts, and the check passes exactly when the pinned matrix is positive semidefinite and every pair was sampled (Appendix H, Theorems H.4 and H.5). The converse direction of the integer check, integers from PSD, is proved there too, and there too I make no claim beyond the chain.

CHAPTER 9

Further reading

The works below were read for the book and are not named in its finished text or in this volume. They are kept here with their annotations, so that nothing read for the book is lost.

9.1 Structure, information and knowledge

Hans Albert. Wikipedia. Article. German philosopher (1921–2023), with Popper a leading critical rationalist; Traktat über kritische Vernunft (1968).

A. J. Ayer (1956). The Problem of Knowledge. Macmillan, London, p. 34. The other definition Gettier targets.

Max Black. Wikipedia. Article. The Russian-born British-American philosopher, at Cornell from 1946, and his 1952 paper on two spheres.

Roberto Bonola (1912). Non-Euclidean Geometry: A Critical and Historical Study of Its Development. Open Court; Dover reprint, 1955. The attempts on the parallel postulate from Proclus through the Islamic geometers and Saccheri, and Farkas Bolyai's letter to his son.

G. K. Chesterton (1929). The Thing. The fence passage is quoted in the American Chesterton Society's Lecture 57: The Thing.

Gösta Grahne (1998). Updates and counterfactuals. Journal of Logic and Computation 8(1), pp. 87–117. DOI: 10.1093/logcom/8.1.87. Revision of a static world versus update of a changing one.

Jonathan Jenkins Ichikawa and Matthias Steup (2001, substantive revision 2026). The Analysis of Knowledge. Stanford Encyclopedia of Philosophy. Entry. Dharmottara's mirage and Peter of Mantua's running Plato.

Rosalie Iemhoff (2008, substantive revision 2019). Intuitionism in the Philosophy of Mathematics. Stanford Encyclopedia of Philosophy. Entry. Kreisel's creating-subject axioms CS1–CS3 (1967), CS2 being that the creating subject never forgets what it has proven, and Troelstra's strengthening (1969).

Petrus Mantuanus (d. 1400). Logica. Medieval manuscripts in Oxford libraries, Bodleian Library. Catalogue entry. Peter of Mantua as a logician of the late fourteenth century.

Per Martin-Löf (1996). On the Meanings of the Logical Constants and the Justifications of the Logical Laws. Nordic Journal of Philosophical Logic 1(1), pp. 11–60. Lectures given in Siena in 1983. The passage on proof as an act is on p. 28. Lecture text.

Meta Platforms (2021). Introducing Meta: A Social Technology Company. Company announcement, 28 October 2021. Announcement.

Benjamin Morison (2014, revised 2026). Sextus Empiricus. Stanford Encyclopedia of Philosophy. Entry. The source for his dates, his medicine and his skepticism.

J. J. O'Connor and E. F. Robertson (n.d.). Theaetetus of Athens. MacTutor History of Mathematics. Entry.

J. J. O'Connor and E. F. Robertson (n.d.). Paul Richard Halmos. MacTutor History of Mathematics. Biography.

Benoît Rittaud and Albrecht Heeffer (2014). The Pigeonhole Principle, Two Centuries before Dirichlet. The Mathematical Intelligencer 36(2), pp. 27–29. Jean Leurechon's use of the principle in 1622.

Göran Sundholm (1998). Proofs as Acts and Proofs as Objects: Some questions for Dag Prawitz. Theoria 64(2–3), pp. 187–216. DOI: 10.1111/j.1755-2567.1998.tb00187.x.

Göran Sundholm (2014). Constructive Recursive Functions, Church's Thesis, and Brouwer's Theory of the Creating Subject: Afterthoughts on a Parisian Joint Session. In Constructivity and Computability in Historical and Philosophical Perspective, Logic, Epistemology, and the Unity of Science, Springer, pp. 1–35. DOI: 10.1007/978-94-017-9217-2_1.

Timothy Yenter and Ezio Vailati (2003, revised 2024). Samuel Clarke. Stanford Encyclopedia of Philosophy. Entry. Clarke as the British philosopher and theologian who defended Newton's natural philosophy in his letters with Leibniz.

9.2 Cost, records and thermodynamics

János Aczél, Bruno Forte and C. T. Ng (1974). Why the Shannon and Hartley entropies are "natural." Advances in Applied Probability 6(1), pp. 131–146. DOI: 10.2307/1426210. Ng's 1971 Waterloo doctorate under Aczél: Mathematics Genealogy Project.

Roberto M. Amadio, Nicolas Ayache, François Bobot, Jaap P. Boender, Brian Campbell, Ilias Garnier, Antoine Madet, James McKinna, Dominic P. Mulligan, Mauro Piccolo, Randy Pollack, Yann Régis-Gianas, Claudio Sacerdoti Coen, Ian Stark and Paolo Tranquilli (2014). Certified Complexity (CerCo). Foundational and Practical Aspects of Resource Analysis, FOPARA 2013, LNCS 8552, pp. 1–18. DOI: 10.1007/978-3-319-12466-7_1.

Walter J. Boyne. Wikipedia entry: United States Air Force officer, aviation historian, director of the National Air and Space Museum (acting from 1982, director 1983–1986). Entry.

Léon Brillouin (1951). Maxwell's Demon Cannot Operate: Information and Entropy. I. Journal of Applied Physics 22(3), pp. 334–337. DOI: 10.1063/1.1699951.

Arthur Charguéraud and François Pottier (2019). Verifying the Correctness and Amortized Complexity of a Union-Find Implementation in Separation Logic with Time Credits. Journal of Automated Reasoning 62, pp. 331–365; first published online in 2017. DOI: 10.1007/s10817-017-9431-7.

Gavin E. Crooks (1999). Entropy production fluctuation theorem and the nonequilibrium work relation for free energy differences. Physical Review E 60(3), pp. 2721–2726. DOI: 10.1103/PhysRevE.60.2721. A path's chance against its reverse's is e to the dissipated work.

Joe DeBlasio (2025). Discussion: removal of one-6962-log requirement. Chromium ct-policy mailing list, 17 November 2025. Post.

Jérémie Decouchant, David Kozhaya, Vincent Rahli and Jiangshan Yu (2022). Damysus: streamlined BFT consensus leveraging trusted components. Proceedings of the Seventeenth European Conference on Computer Systems (EuroSys '22), pp. 1–16. DOI: 10.1145/3492321.3519568.

Encyclopaedia Britannica. Donation of Constantine. Web page. Composed in the eighth century; the consensus view puts it in the 750s or 760s.

Encyclopaedia Britannica. Leo Szilard. Web page. Born in Budapest in 1898; Berlin doctorate 1922 and the Institute of Theoretical Physics there.

Ethereum Foundation (n.d.). The Merge. ethereum.org roadmap page, on the transition of 15 September 2022. Page.

Ethereum Foundation (n.d.). Timeline of all Ethereum forks (Tangerine Whistle, EIP-150, and the Berlin upgrade, block 12,244,000). ethereum.org. Page.

Ethereum Foundation. Devcon editions (Devcon 0, Berlin 2014; Devcon 1, London 2015; Devcon 2, Shanghai 2016). devcon.org. Page.

Ethereum Foundation. Ethereum execution specifications, Prague fork, src/ethereum/forks/prague/vm/instructions/storage.py and vm/gas.py, at commit ecb68f21a7d28db101c769878184f6facb319da1. Repository.

Federal Aviation Administration (2008). Revisions to Cockpit Voice Recorder and Digital Flight Data Recorder Regulations. Final rule, Federal Register 73(46), 7 March 2008, pp. 12542–12573. The two-hour cockpit voice recorder.

Federal Aviation Administration (2026). 25-Hour Cockpit Voice Recorder (CVR) Requirement, New Aircraft Production. Final rule, Federal Register 91(21), 2 February 2026, pp. 4447–4462. Federal Register text. The thirty-minute recorders of 1966, the two-hour rule, and the 25-hour standard of ICAO (2016) and EASA (from 2021).

Richard FitzNigel (c. 1180). Dialogus de Scaccario (The Dialogue of the Exchequer). Text and translation ed. Charles Johnson, Nelson's Medieval Classics, 1950. The widths of the cuts on Exchequer tallies.

Fox-IT (2012). Black Tulip: Report of the investigation into the DigiNotar Certificate Authority breach. Version 1.0. Report.

Tom Furtwangler (2017). The vaccine vial monitor: the world's smartest sticker. PATH, 6 November 2017. Article. The stickers first sold in 1996, for oral polio vaccine.

General Conference on Weights and Measures (2018). Resolution 1 of the 26th meeting: On the revision of the International System of Units (SI), effective 20 May 2019.

C. H. Jenkinson (1910). Early wooden tallies relating to Surrey. Surrey Archaeological Collections 23, pp. 203–207. PDF. Hazel; the payer kept the stock, the Exchequer the foil; abolished by statute in 1826; the burning that started the fire.

Joint Committee for Guides in Metrology (2012). International vocabulary of metrology: basic and general concepts and associated terms (VIM), 3rd edition, JCGM 200:2012, entry 2.41, "metrological traceability." The definition is on p. 29; "does not ensure . . . that there is an absence of mistakes" is Note 5 to 2.41, p. 30. BIPM text.

Mikhail Kuter, Marina Gurskaya and Albert Gurskii (2025). Genoa 1340, Double Entry, and 'Gold Coasting'. De Computis, Revista Española de Historia de la Contabilidad 22(1), pp. 1–33. DOI: 10.26784/issn.1886-1881.22.1.11927.

Klaus-Jörn Lange, Pierre McKenzie and Alain Tapp (2000). Reversible Space Equals Deterministic Space. Journal of Computer and System Sciences 60(2), pp. 354–367. DOI: 10.1006/jcss.1999.1672.

Ben Laurie, Adam Langley and Emilia Kasper (2013). Certificate Transparency. RFC 6962. RFC 6962; DOI: 10.17487/RFC6962.

Ben Laurie, Eran Messeri and Rob Stradling (2021). Certificate Transparency Version 2.0. RFC 9162. RFC 9162; DOI: 10.17487/RFC9162.

A. Liapounoff (1907). Problème général de la stabilité du mouvement, traduit du russe par Édouard Davaux. Annales de la Faculté des Sciences de Toulouse, 2e série, 9, pp. 203–474. Numdam. The French translation the 1992 English one was made from.

Sinisa Matetic, Mansoor Ahmed, Kari Kostiainen, Aritra Dhar, David Sommer, Arthur Gervais, Ari Juels and Srdjan Capkun (2017). ROTE: rollback protection for trusted execution. Proceedings of the 26th USENIX Security Symposium, pp. 1289–1306. USENIX page.

Clyde Lee Miller (2009; revised 2021). Cusanus, Nicolaus [Nicolas of Cusa]. Stanford Encyclopedia of Philosophy. SEP entry.

Yûto Murashita, Ken Funo and Masahito Ueda (2014). Nonequilibrium equalities in absolutely irreversible processes. Physical Review E 90, 042110. DOI: 10.1103/PhysRevE.90.042110. The bound on average entropy production by $-\ln(1-\lambda)$, with free expansion as the first example.

Satoshi Nakamoto (2008). Bitcoin: A Peer-to-Peer Electronic Cash System. Paper.

Devon O'Brien (2018). Certificate Transparency Enforcement in Google Chrome. Chromium ct-policy mailing list, 6 February 2018. Announcement.

Garson O'Toole (2016). There Ain't No Such Thing as a Free Lunch, TANSTAAFL. Quote Investigator, 27 August 2016. Article.

Pavel Pakshin, Elena Panteley and Natalia Pakshina (2017). The Legacy of A. M. Lyapunov to Systems and Control Theory: 110 years of the Toulouse edition. IFAC World Congress 2017, open-track proposal. Davaux's translation, reviewed and corrected by Lyapunov.

Lídia del Rio, Johan Åberg, Renato Renner, Oscar Dahlsten and Vlatko Vedral (2011). The thermodynamic meaning of negative entropy. Nature 474, pp. 61–63. DOI: 10.1038/nature10123; arXiv: 1009.1630.

Jürgen Schnakenberg (1976). Network theory of microscopic and macroscopic behavior of master equation systems. Reviews of Modern Physics 48, pp. 571–585. DOI: 10.1103/RevModPhys.48.571. The entropy a Markov chain hands its surroundings, as the net rate across each pair times the log of the forward-to-backward rate ratio, which at a steady state is the whole entropy production; the biased walk is its smallest case.

Securities and Exchange Commission (2022). Electronic Recordkeeping Requirements for Broker-Dealers, Security-Based Swap Dealers, and Major Security-Based Swap Participants. Release No. 34-96034, 12 October 2022. Final rule. Adds an audit-trail alternative to the write-once requirement of Rule 17a-4(f) in place since 1997.

Caroline Shenton (2012). The Day Parliament Burned Down. Oxford University Press. The tallies burned in the furnaces under the House of Lords, 16 October 1834.

Michael Stock, Pauline Barat, Richard S. Davis, Alain Picard and Martin J. T. Milton (2015). Calibration campaign against the international prototype of the kilogram in anticipation of the redefinition of the kilogram part I: comparison of the international prototype with its official copies. Metrologia 52(2), pp. 310–316. DOI: 10.1088/0026-1394/52/2/310.

Wei Tang (2019). EIP-2200: Structured Definitions for Net Gas Metering. Ethereum Improvement Proposals. eips.ethereum.org/EIPS/eip-2200.

This Day in Aviation (n.d.). 30 October 1935: Boeing Model 299 crash at Wright Field. Page.

Kent Weare (2016). Ethereum Security Alert Issued, Ethereum Foundation Responds with “From Shanghai, With Love”. InfoQ, 19 September 2016. Article.

World Health Organization (2011). PQS performance specification: Vaccine Vial Monitor. WHO/PQS/E06/IN05.2.

9.3 Computation, self-reference and checking

Keith Adams and Ole Agesen (2006). A Comparison of Software and Hardware Techniques for x86 Virtualization. Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS XII), pp. 2–13. DOI: 10.1145/1168857.1168860. Binary translation of instructions such as POPF, which on x86 runs in user mode without trapping. Intel’s VT-x and AMD-V in 2005 and 2006: Wikipedia, x86 virtualization.

American Standards Association (1963). American Standard Code for Information Interchange, ASA X3.4-1963.

Henk Barendregt. Wikipedia. Article. A Dutch logician known for his work in lambda calculus and type theory.

Gerd Behrmann, Ansgar Fehnker, Thomas Hune, Kim Larsen, Paul Pettersson, Judi Romijn and Frits Vaandrager (2001). Minimum-cost reachability for priced timed

automata. Hybrid Systems: Computation and Control (HSCC 2001), Lecture Notes in Computer Science 2034, pp. 147–161.

N. G. de Bruijn (1972). Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church–Rosser theorem. Indagationes Mathematicae (Proceedings) 75(5), pp. 381–392. DOI: 10.1016/1385-7258(72)90034-0. Variables written as numbers that count the binders back, now called de Bruijn indices.

Stanley Burris and Marcel Jackson (2010, substantive revision 2026). George Boole. Stanford Encyclopedia of Philosophy. Entry.

Stanley Burris and Javier Legris (2009, substantive revision 2021). The Algebra of Logic Tradition. Stanford Encyclopedia of Philosophy. Entry.

G. K. Chesterton. Wikipedia. Article. An English writer, referred to as the “prince of paradox”.

Donald W. Davies (2004). Corrections to Turing’s Universal Computing Machine. In B. Jack Copeland (ed.), The Essential Turing, pp. 103–124. Oxford University Press. Davies at the National Physical Laboratory from 1947 and the word *packet*: Wikipedia, Donald Davies.

Charles Dickens (1843). A Christmas Carol. Chapman and Hall, London. Stave Three, the game of Yes and No.

William Henry Eccles and Frank Wilfred Jordan (1920). Improvements in ionic relays. British patent GB148582A, filed 21 June 1918, published 5 August 1920. Patent text. The 1918 date in the text is the filing date. The circuit as the flip-flop, the basis for electronic memory: History of Information, entry; and Wikipedia, Flip-flop (electronics), on the Eccles–Jordan trigger circuit as the first electronic latch, later the basic storage element of digital circuitry.

László Fejes Tóth (1953). Lagerungen in der Ebene, auf der Kugel und im Raum. Berlin: Springer.

László Fejes Tóth (1964). Regular Figures. Oxford: Pergamon Press.

Jean-Christophe Filliâtre, Léon Gondelman and Andrei Paskevich (2014). The spirit of ghost code. Computer Aided Verification (CAV 2014), Lecture Notes in Computer Science 8559, pp. 1–16. DOI: 10.1007/978-3-319-08867-9_1.

Nicholas Griffin, ed. (2003). The Cambridge Companion to Bertrand Russell. Cambridge University Press. G. G. Berry as a junior librarian at the Bodleian, p. 63.

Jean-Pierre Marquis (1996, substantive revision 2019). Category Theory. Stanford Encyclopedia of Philosophy. Entry. The borrowed words “category” and “functor”.

Ross M. McConnell, Kurt Mehlhorn, Stefan Näher and Pascal Schweitzer (2011). Certifying algorithms. Computer Science Review 5(2), pp. 119–161. DOI: 10.1016/j.cosrev.2010.09.009.

Jason Megill. The Lucas-Penrose Argument about Gödel's Theorem. Internet Encyclopedia of Philosophy. Article.

Glen Mével, Jacques-Henri Jourdan and François Pottier (2019). Time credits and time receipts in Iris. ESOP 2019, LNCS 11423, pp. 3–29. DOI: 10.1007/978-3-030-17184-1_1.

Jeff Miller. Earliest Uses of Symbols of Set Theory and Logic. MacTutor History of Mathematics. Page. Peano's epsilon for *est*, 1889.

J. J. O'Connor and E. F. Robertson (n.d.). The four colour theorem. MacTutor History of Mathematics. Entry. (De Morgan's letter to Hamilton of 23 October 1852 is quoted from here.)

Panu Raatikainen (2013, revised 2025). Gödel's Incompleteness Theorems. Stanford Encyclopedia of Philosophy. Entry. Source for the cautions about what the Gödel sentence says.

Erich Reck (2008, substantive revision 2025). Dedekind's Contributions to the Foundations of Mathematics. Stanford Encyclopedia of Philosophy. Entry. Peano acknowledged Dedekind's priority for the axioms.

Neil Robertson, Daniel Sanders, Paul Seymour and Robin Thomas (1997). The four-colour theorem. Journal of Combinatorial Theory, Series B 70(1), pp. 2–44.

Jürgen Schmidhuber (2003). Gödel machines: self-referential universal problem solvers making provably optimal self-improvements. Technical Report IDSIA-19-03. arXiv: cs/0309048.

Marcel-Paul Schützenberger (1961). On the definition of a family of automata. Information and Control 4(2–3), pp. 245–270.

Dana Scott (1962). A system of functional abstraction. Lectures delivered at the University of California, Berkeley, 1962–63; unpublished. The origin of the Scott encoding, as given in Mogensen–Scott encoding, Wikipedia, Article.

Dana Scott. Carnegie Mellon University's School of Computer Science, faculty Turing Award recipients: the 1976 award jointly with Michael Rabin for their work on nondeterministic machines. His doctorate at Princeton under Alonzo Church, domain theory and models of the lambda calculus: Wikipedia, Dana Scott.

Emin Gün Sirer, Willem de Bruijn, Patrick Reynolds, Alan Shieh, Kevin Walsh, Dan Williams and Fred B. Schneider (2011). Logical attestation: an authoriza-

tion architecture for trustworthy computing. SOSP 2011, pp. 249–264. DOI: 10.1145/2043556.2043580.

Ken Thompson. Heidelberg Laureate Forum, laureate profile: the 1983 Turing Award with Dennis Ritchie for operating systems theory and the implementation of Unix. Thompson's design and implementation of the original Unix at Bell Labs: Wikipedia, Ken Thompson.

Alfred North Whitehead and Bertrand Russell (1910). Principia Mathematica, vol. I. Cambridge University Press. Introduction, chapter II.

Robin Wilson (2002). Four Colours Suffice? Gresham College lecture, 21 October 2002. Lecture. De Morgan's letter of 23 October 1852 to the Irish mathematician William Rowan Hamilton.

9.4 Correlations and environmental records

M.-C. Chen et al. (2019). Emergence of classical objectivity of quantum Darwinism in a photonic quantum simulator. Science Bulletin 64, p. 580.

M. A. Ciampini, G. Pinna, P. Mataloni and M. Paternostro (2018). Experimental signature of quantum Darwinism in photonic cluster states. Physical Review A 98, 020101.

Mauro Dorato and Jan Faye (2026). Niels Bohr as a Physicist of Principle. arXiv: 2609.04297. Quotes Bohr's Unity of Knowledge (1954) on irreversible amplification and permanent marks on a photographic plate, citing p. 73.

Lluís Masanes (2003). Necessary and sufficient condition for quantum-generated correlations. arXiv: quant-ph/0309137v1. The paper gives his affiliation as the Universitat de Barcelona.

Miguel Navascués, Stefano Pironio and Antonio Acín (2007). Bounding the Set of Quantum Correlations. Physical Review Letters 98, 010401. DOI: 10.1103/PhysRevLett.98.010401; arXiv: quant-ph/0607119v2.

Miguel Navascués, Stefano Pironio and Antonio Acín (2008). A convergent hierarchy of semidefinite programs characterizing the set of quantum correlations. New Journal of Physics 10, 073013. arXiv: 0803.4290v1.

Marcin Pawłowski, Tomasz Paterek, Dagomir Kaszlikowski, Valerio Scarani, Andreas Winter and Marek Żukowski (2009). Information causality as a physical principle. Nature 461, pp. 1101–1104. Pawłowski's affiliation, the University of Gdańsk, is on the paper: arXiv: 0905.2292.

Erwin Schrödinger. Wikipedia. Article. Austrian physicist; the wave equation of 1926.

B. S. Tsirel'son (1987). Quantum analogues of the Bell inequalities. The case of two spatially separated domains. Journal of Soviet Mathematics 36(4), pp. 557–570. DOI: 10.1007/BF01663472.

T. Unden, D. Louzon, M. Zwolak, W. H. Zurek and F. Jelezko (2019). Revealing the emergence of classicality using nitrogen-vacancy centers. Physical Review Letters 123, 140402.

COLOPHON

This book is set in Charter, drawn by Matthew Carter in 1987 for the laser printers of its day, 10.5 on 14 point, on a page 189 by 246 millimetres. It was built from its source with pdfTeX. The cells are drawn in TikZ.

Its own count, taken by the build: 30 numbered statements, of which 31 are closed by a proof on the page, and the rest stated and not proved: . One cell for each, in the order they come, filled where the proof closed:



